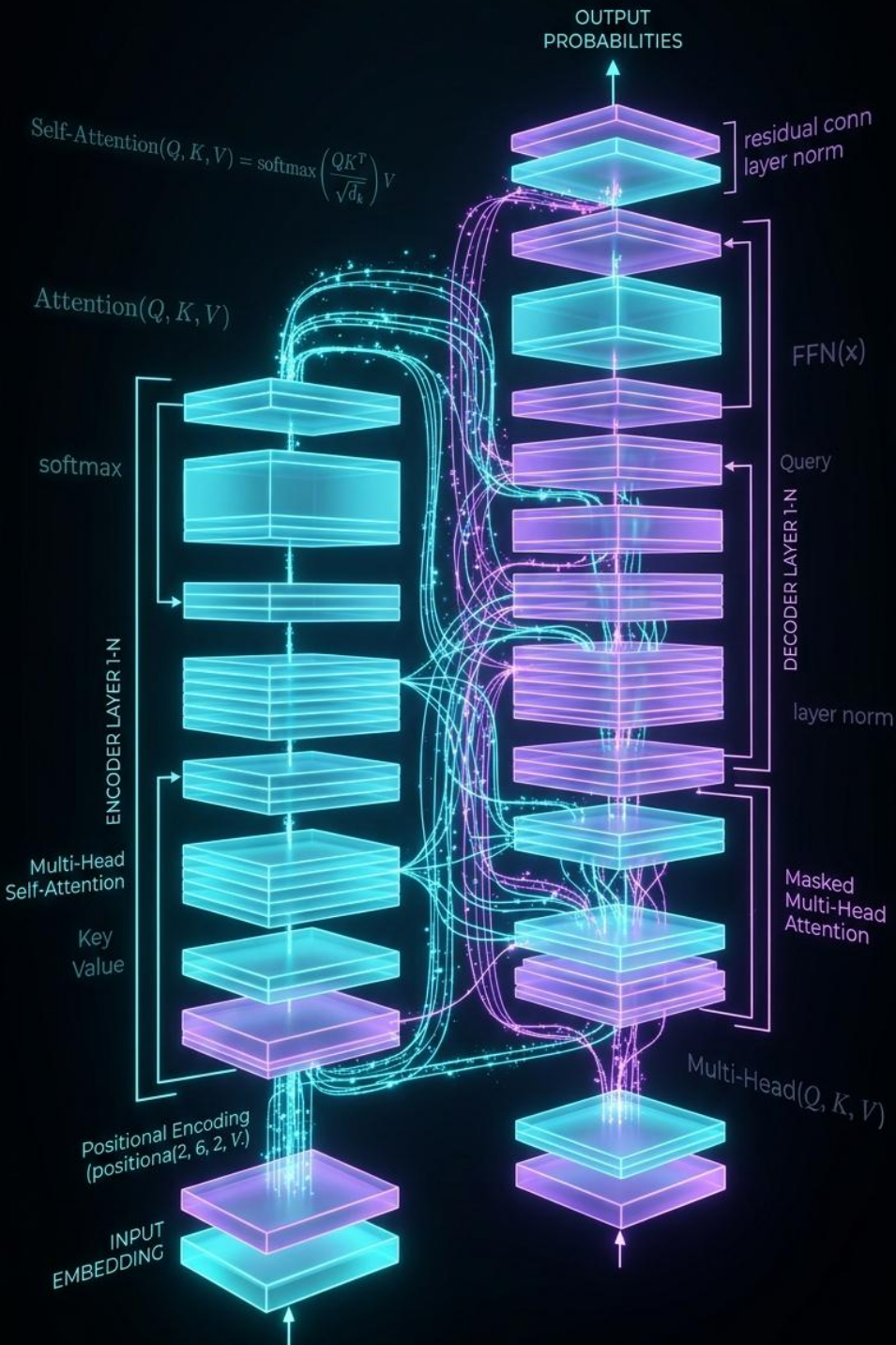


THE TRANSFORMER ARCHITECTURE: ATTENTION AND SELF-ATTENTION MECHANISMS



Transformers

A Complete Lecture on Transformers

Undergraduate Engineering — Neural Networks & Deep Learning

Attention Is All You Need (2017)

[Ashish Vaswani](#), [Noam Shazeer](#), [Niki Parmar](#), [Jakob Uszkoreit](#), [Llion Jones](#), [Aidan N. Gomez](#), [Lukasz Kaiser](#), [Illia Polosukhin](#)

In 2017, researchers at Google published the groundbreaking paper:
“Attention Is All You Need”

This paper introduced the **Transformer architecture**, which completely changed the field of Artificial Intelligence.

Before Transformers:

- RNNs and LSTMs processed words sequentially,
- training was slow,
- long-range dependencies were difficult to learn,
- and parallelization was limited.

The Transformer solved these problems using the **Self-Attention Mechanism**, allowing:

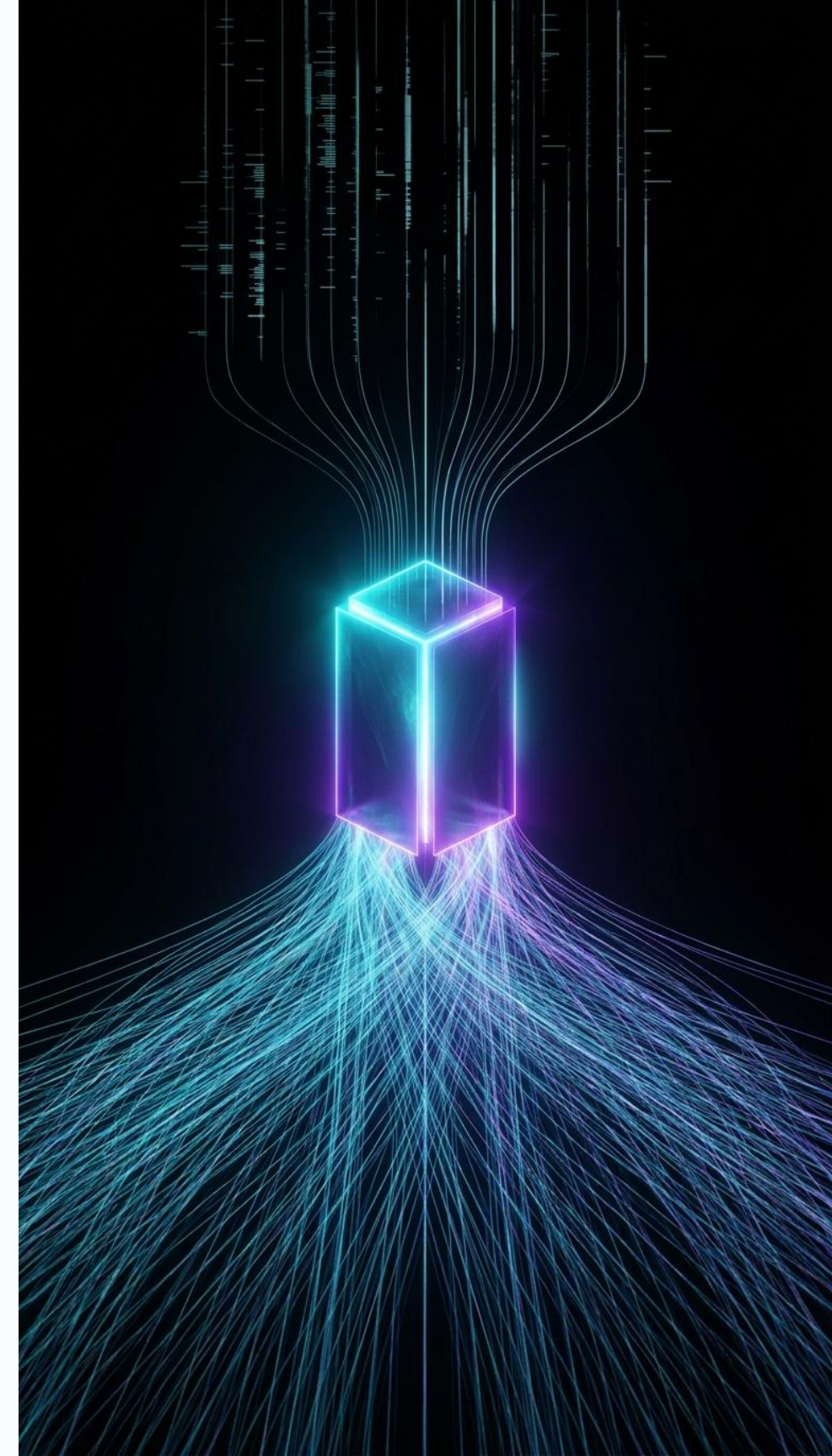
- parallel computation,
- better long-term dependency modeling,
- faster training,
- and significantly improved performance.

Today, Transformers are the foundation of:

- GPT models,
- ChatGPT,
- BERT,
- Vision Transformers (ViT),
- and many modern AI systems.

The central idea of the paper was revolutionary:

Instead of recurrence, use attention mechanisms to understand relationships between tokens.



Attention Layers-Modules

An improving mechanism for RNNs by applying an interface connecting the encoder and decoder

*Neural Machine Translation by Jointly Learning to Align and Translate
[Dzmitry Bahdanau](#), [Kyunghyun Cho](#), [Yoshua Bengio](#)

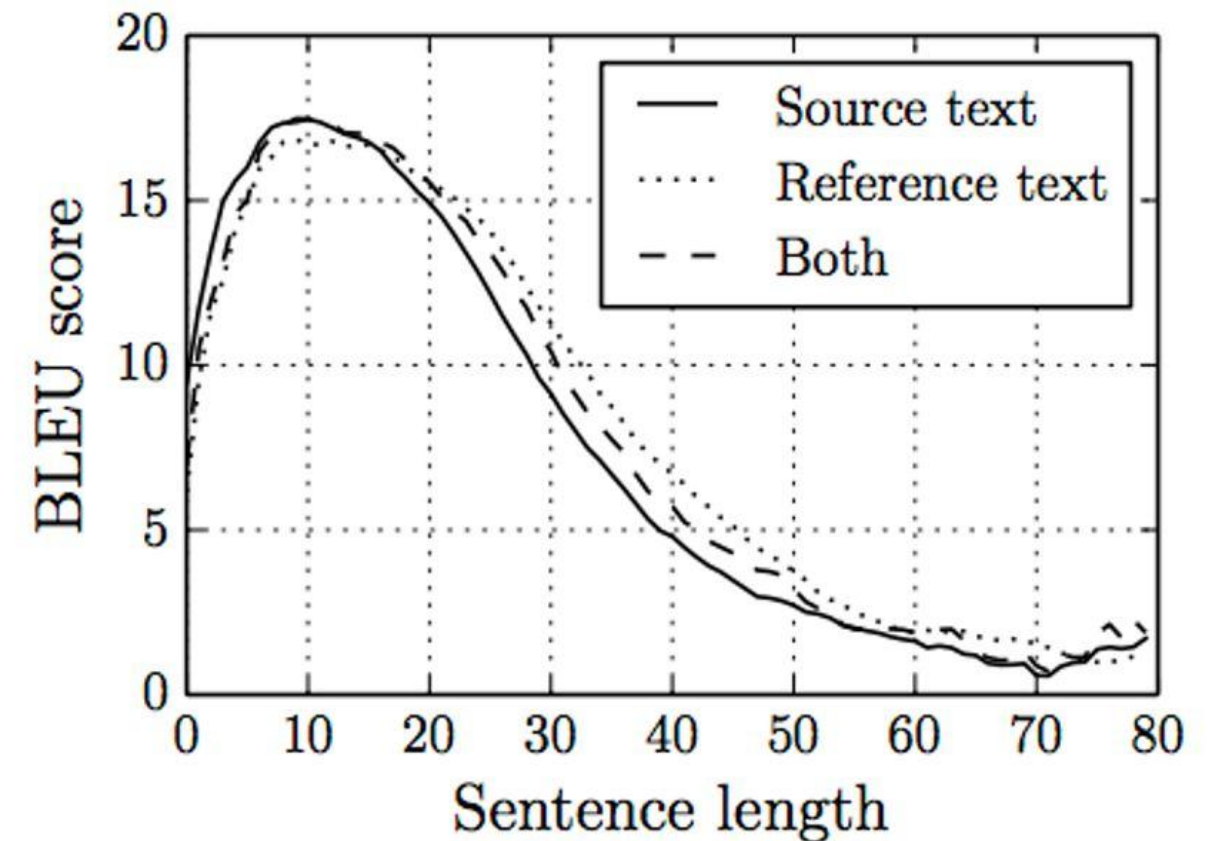
- In translation machines, LSTM/GRU as a seq2seq model can not predict successful translation for a sentence, when its length increases.

What is attention layer in Deep Learning?

- Every RNN/LSTM can be interpreted as an Encoder and Decoder.
 - Encoder encodes the input samples of a chronological signal to hidden state.
 - Decoder decodes the hidden state to the output.

Attention is an interface connecting the encoder and decoder that provides the decoder with information from every encoder hidden state.

With this framework, the model is able to selectively focus on valuable parts of the input sequence and hence, learn the association between them.



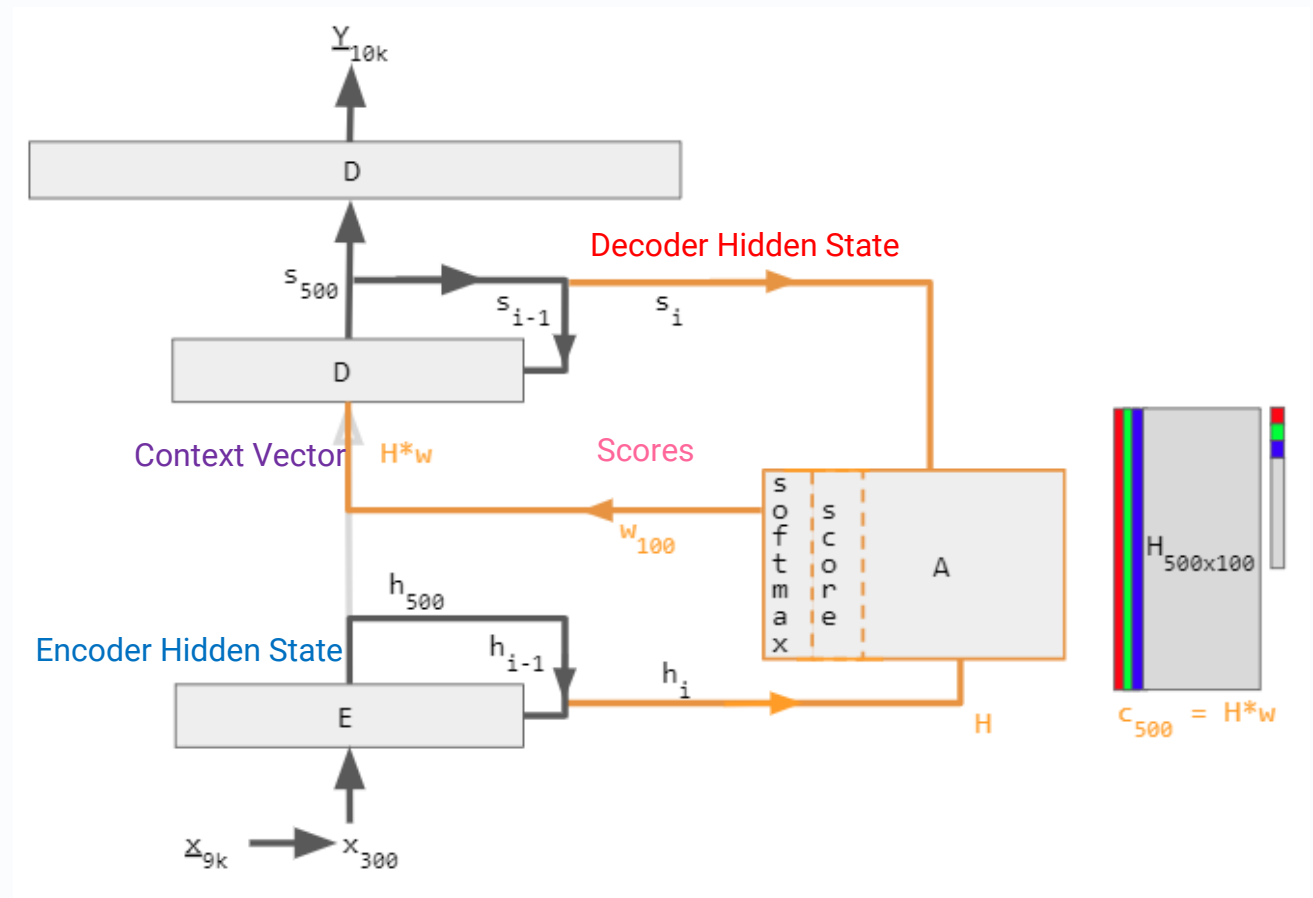
BLEU: Bilingual Evaluation Understudy

A language translation example

To build a machine that translates **English-to-French** (see the shown diagram), one starts with an **Encoder-Decoder** and **grafts an attention unit to it**. In the simplest case such as the shown example, **the attention unit is just lots of dot products of recurrent layer states and does not need training**. In practice, the attention unit consists of 3 fully connected neural network layers that needs to be trained. The 3 layers are called Query, Key, and Value.

Encoder-Decoder with attention. This diagram uses specific values to relieve an already cluttered notation alphabet soup. **The left part (in black) is the Encoder-Decoder, the middle part (in orange) is the attention unit, and the right part (in grey & colors) is the computed data**. Grey regions in H matrix and w vector are zero values.

Numerical subscripts are examples of vector sizes. Lettered subscripts i and $i-1$ indicate time step.

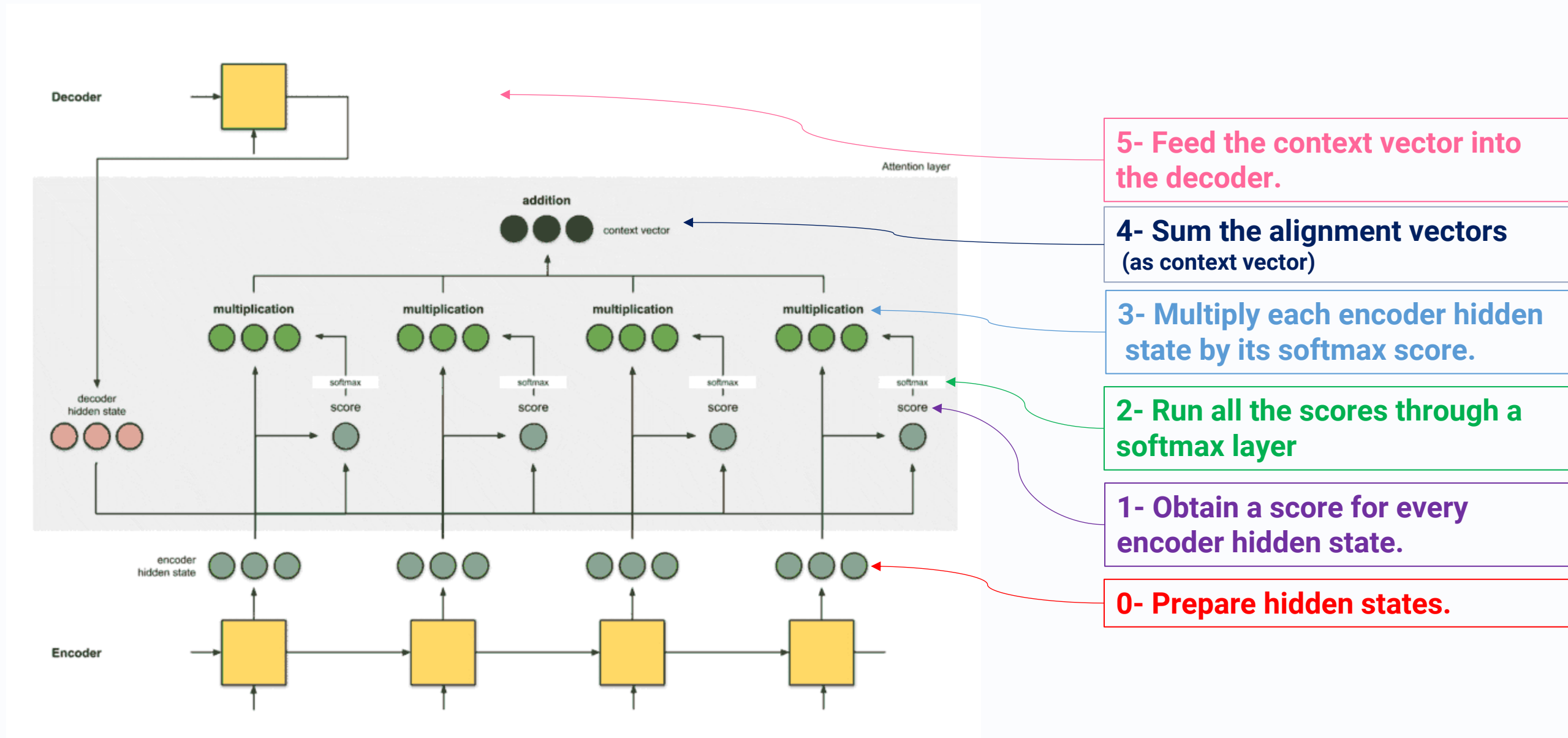


A language translation example

Label	Description
100	Max sentence length
300	Embedding size (word dimension)
500	Length of hidden vector
9k, 10k	Dictionary size of input & output languages respectively.
$\underline{x}, \underline{y}$	9k and 10k 1-hot dictionary vectors. $\underline{x} \rightarrow x$ implemented as a lookup table rather than vector multiplication. $\underline{y}$ is the 1-hot maximizer of the linear Decoder layer D; that is, it takes the argmax of D's linear layer output.
x	300-long word embedding vector. The vectors are usually pre-calculated from other projects such as GloVe or Word2Vec .
h	500-long encoder hidden vector. At each point in time, this vector summarizes all the preceding words before it. The final h can be viewed as a "sentence" vector, or a thought vector as Hinton calls it.
s	500-long decoder hidden state vector.
E	500 neuron RNN encoder. 500 outputs. Input count is 800–300 from source embedding + 500 from recurrent connections. The encoder feeds directly into the decoder only to initialize it, but not thereafter; hence, that direct connection is shown very faintly.
D	2-layer decoder. The recurrent layer has 500 neurons and the fully connected linear layer has 10k neurons (the size of the target vocabulary). ^[7] The linear layer alone has 5 million (500 * 10k) weights -- ~10 times more weights than the recurrent layer.
score	100-long alignment score
w	100-long vector attention weight. These are "soft" weights which changes during the forward pass, in contrast to "hard" neuronal weights that change during the learning phase.
A	Attention module — this can be a dot product of recurrent states, or the Query-Key-Value fully connected layers. The output is a 100-long vector w .
H	500x100. 100 hidden vectors h concatenated into a matrix
c	500-long context vector = $H * w$. c is a linear combination of h vectors weighted by w .

Unfolded attention layer

The below figure demonstrates an Encoder-Decoder architecture with an attention layer (All time steps).

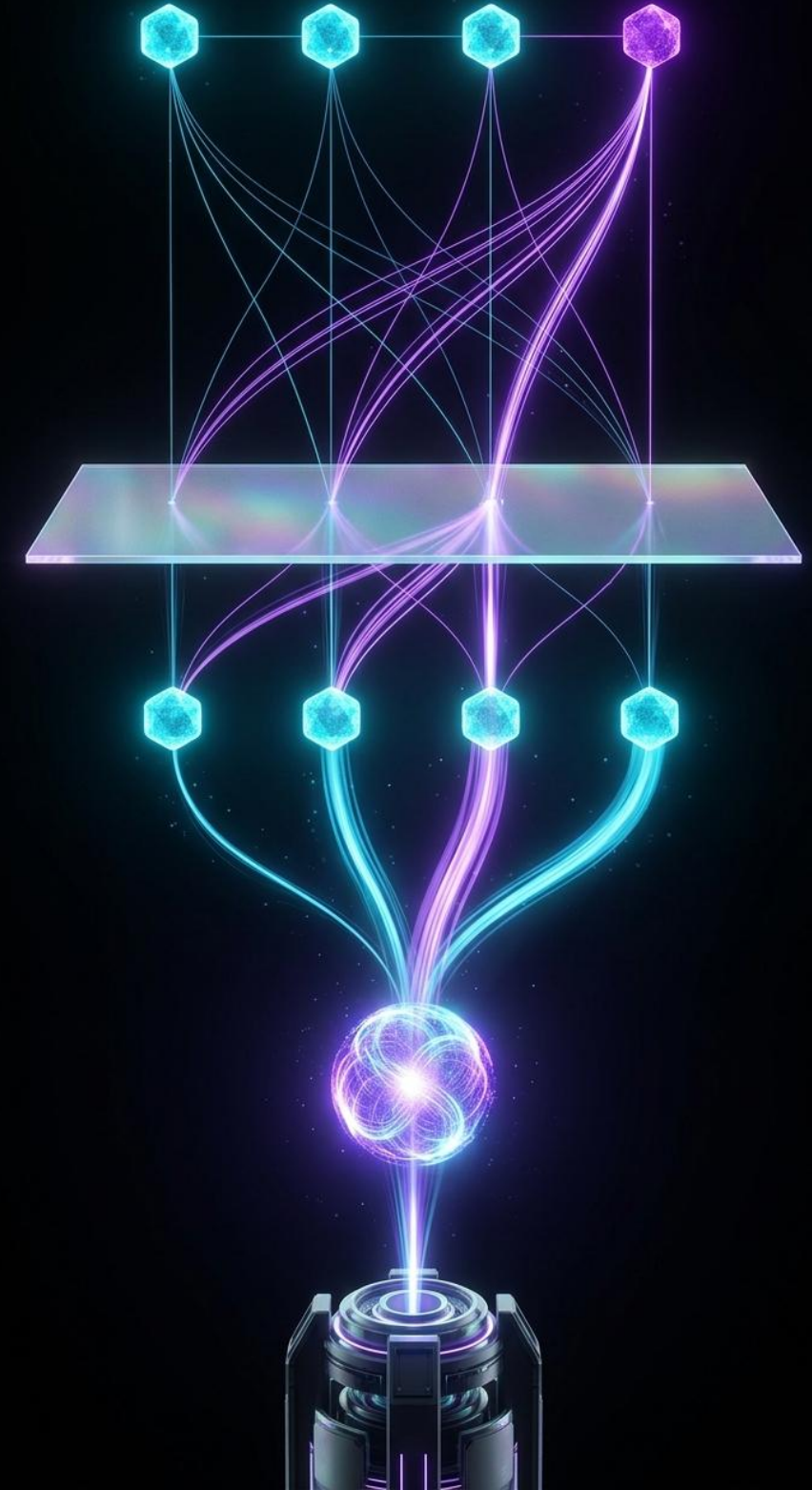


Unfolded attention layer

The implementations of an attention layer can be broken down into 4 steps.

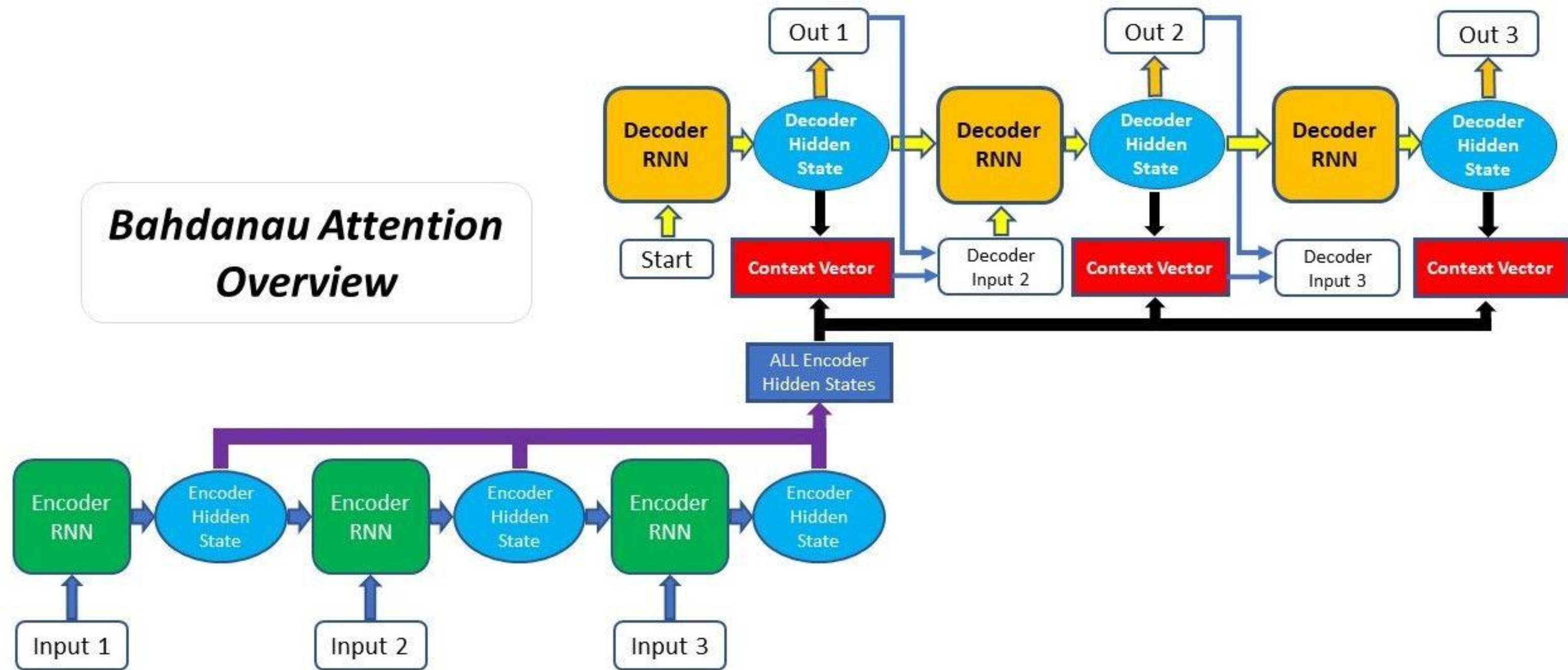
For each time step:

- **Step 0: Prepare hidden states.**
- First, prepare all the available encoder hidden states (green) and the first decoder hidden state (red). In our example, we have 4 encoder hidden states and the current decoder hidden state. (Note: the last consolidated encoder hidden state is fed as input to the first time step of the decoder. The output of this first time step of the decoder is called the first decoder hidden state.)
- **Step 1: Obtain a score for every encoder hidden state.**
- A score (scalar) is obtained by a score function (also known as alignment score function or alignment model). In this example, the score function is a dot product between the decoder and encoder hidden states.
- **Step 2: Run all the scores through a softmax layer.**
- We put the scores to a softmax layer so that the softmax scores (scalar) add up to 1. These softmax scores represent the attention distribution.
- **Step 3: Multiply each encoder hidden state by its softmax score.**
- By multiplying each encoder hidden state with its softmax score (scalar), we obtain the alignment vector or the annotation vector. This is exactly the mechanism where alignment takes place.
- **Step 4: Sum the alignment vectors.**
- The alignment vectors are summed up to produce the context vector. A context vector is an aggregated information of the alignment vectors from the previous step.
- **Step 5: Feed the context vector into the decoder.**



Bahdanau attention layer

Bahdanau Attention Overview



*Neural Machine Translation by Jointly Learning to Align and Translate (2014-2016)

[Dzmitry Bahdanau](#), [Kyunghyun Cho](#), [Yoshua Bengio](#)

Attention variants

1. Encoder-decoder dot product
2. Encoder-decoder QKV (Query-Key-Value)
3. Encoder-only dot product
4. Encoder-only QKV
5. PyTorch Tutorial



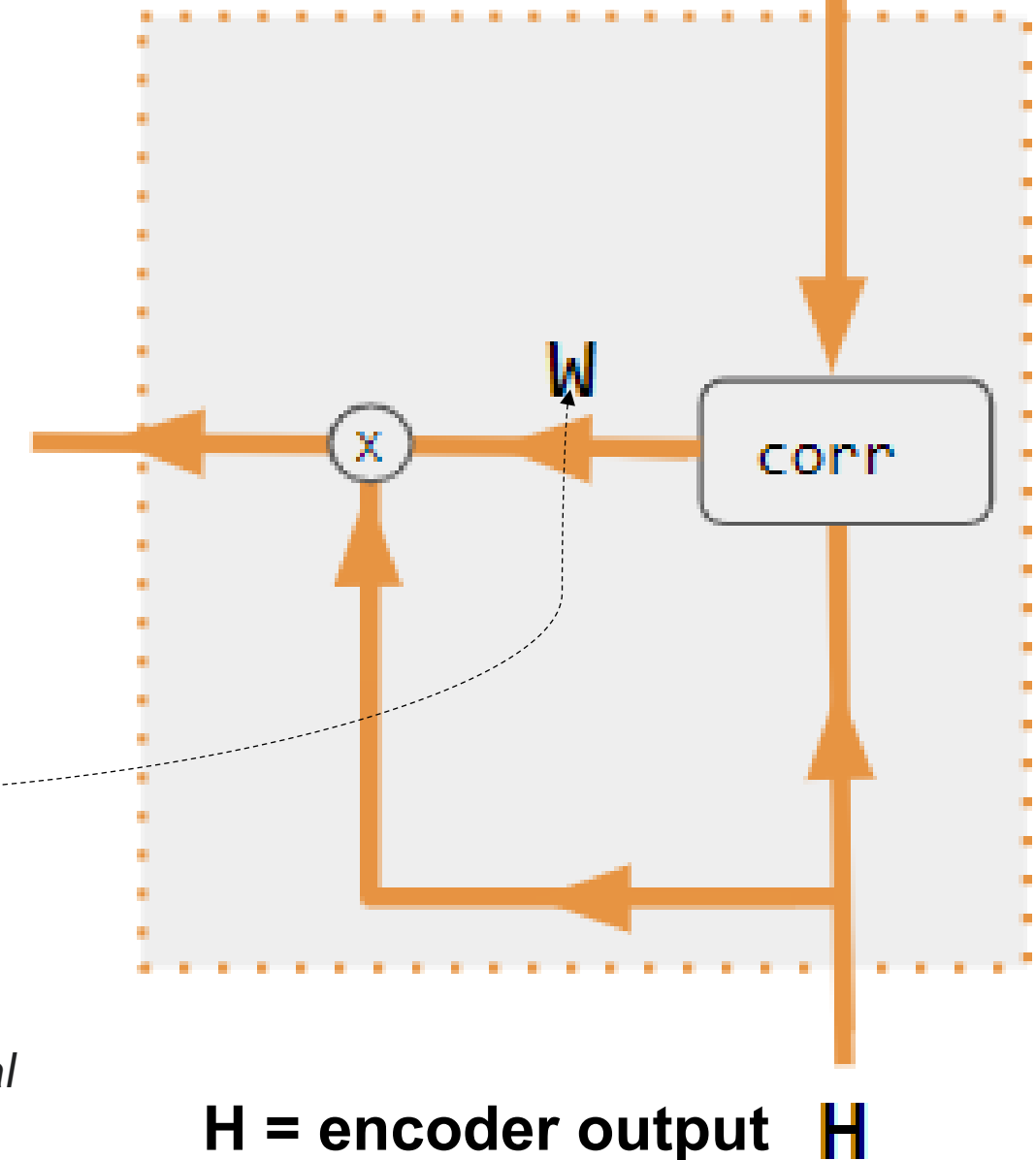
1. Encoder-decoder dot product

Both Encoder & Decoder are needed to calculate Attention.

c = context vector = $H \cdot w$ $W \times H$

w = attention weight = softmax(score)

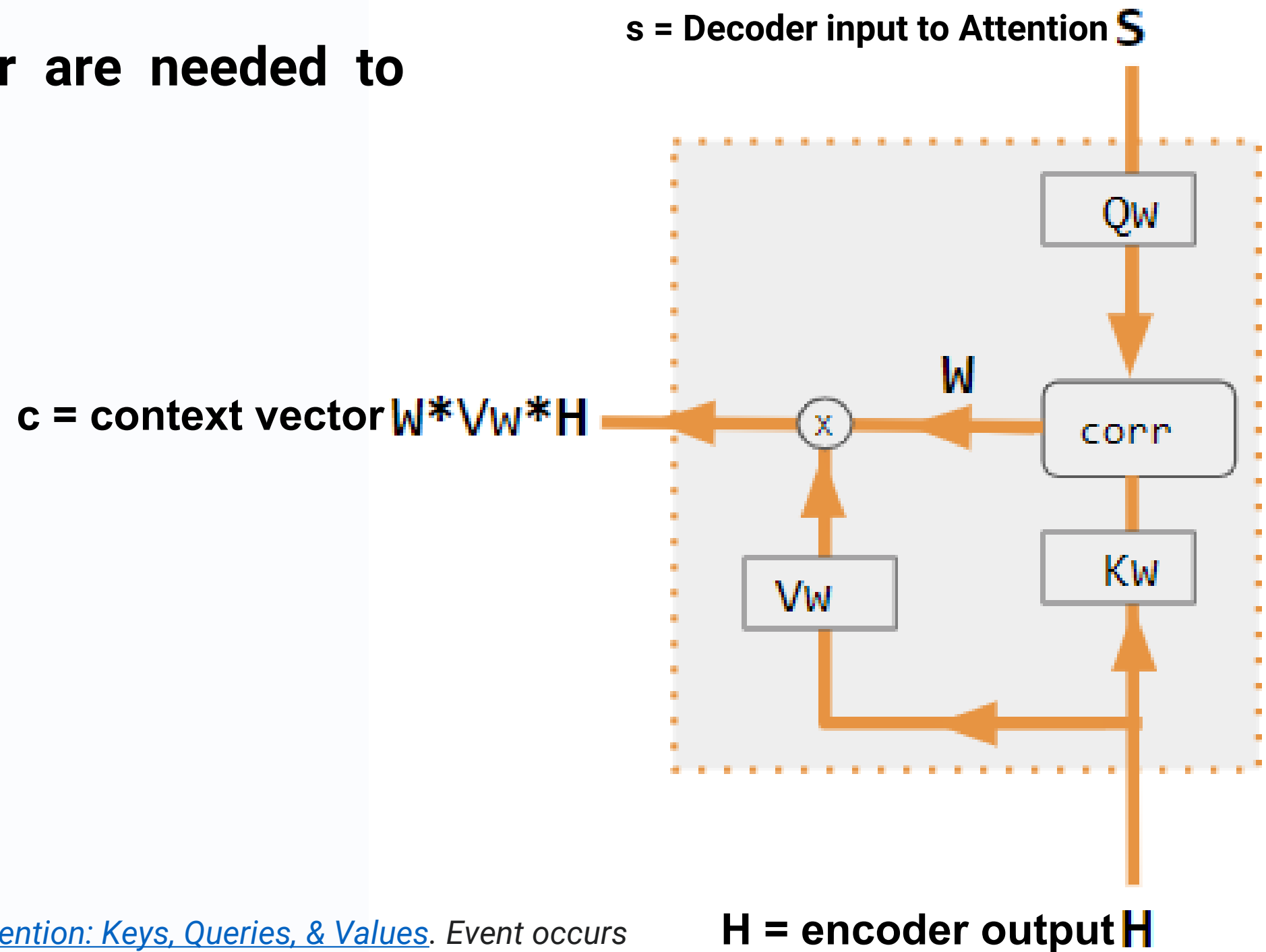
s = Decoder input to Attention
= Decoder hidden state



1. Luong, Minh-Thang (2015-09-20). "Effective Approaches to Attention-based Neural Machine Translation". [arXiv:1508.04025v5](https://arxiv.org/abs/1508.04025v5) [cs.CL].

2. Encoder-decoder QKV

Both Encoder & Decoder are needed to calculate Attention.

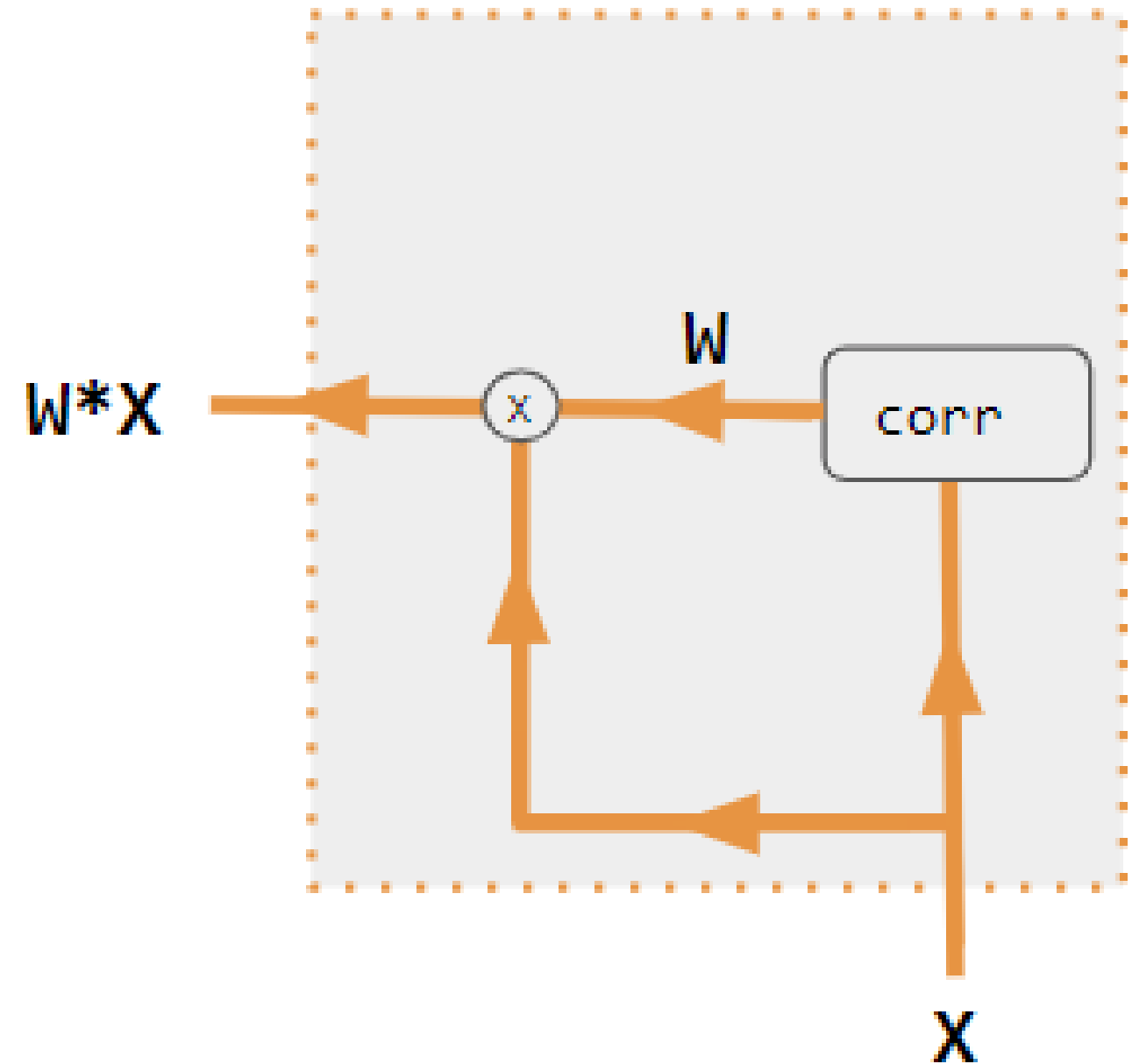


1. Neil Rhodes (2021). [CS 152 NN-27: Attention: Keys, Queries, & Values](#). Event occurs at 06:30. Retrieved 2021-12-22.

3. Encoder-only dot product

Decoder is NOT used to calculate Attention. With only 1 input into corr, **W** is an **auto-correlation** of dot products.

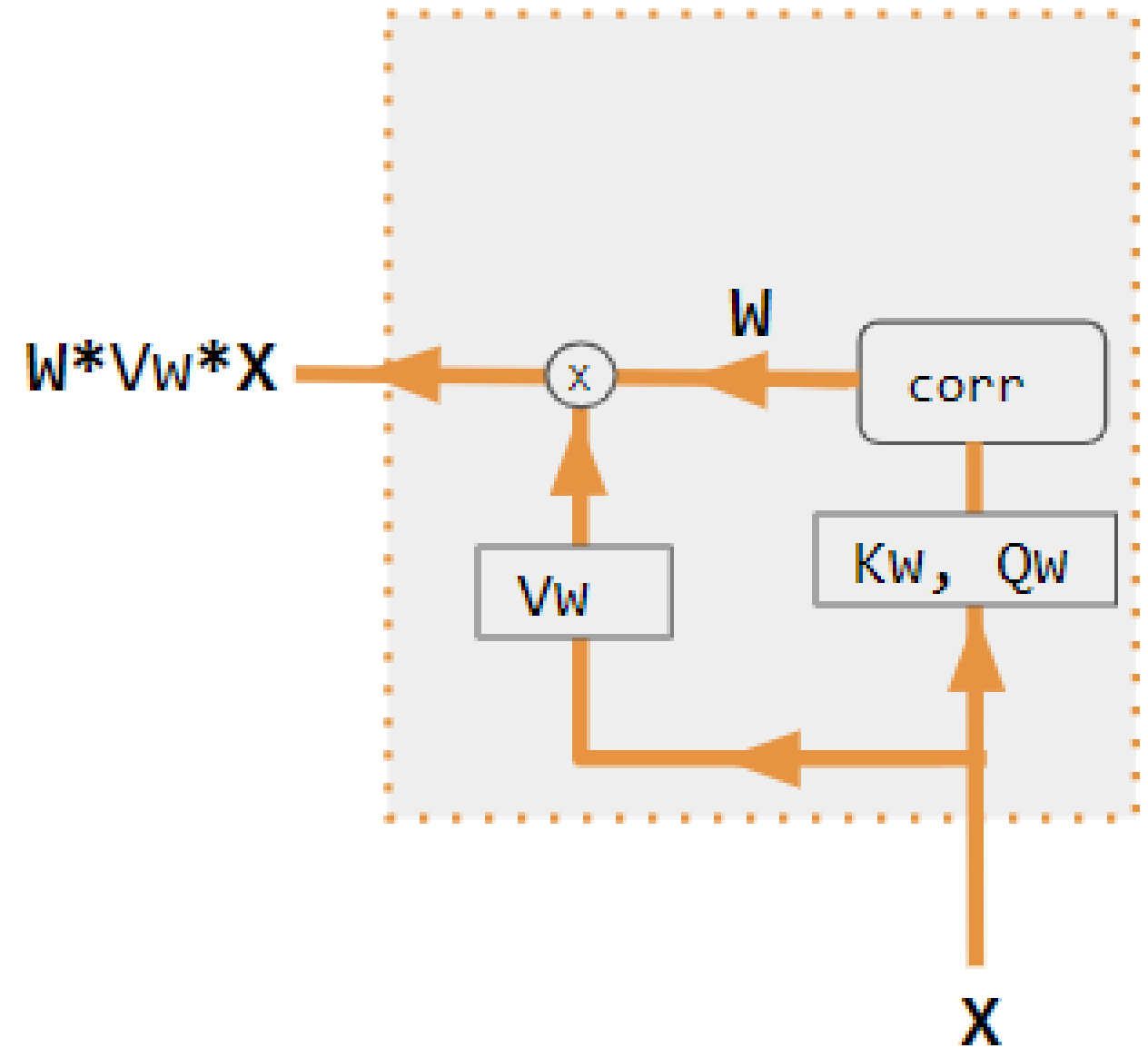
$$W_{ij} = x_i * x_j \text{ [10]}$$



1. Alfredo Canziani & Yann Lecun (2021). [NYU Deep Learning course, Spring 2020](#). Event occurs at 05:30. Retrieved 2021-12-22.

4. Encoder-only QKV

Decoder is NOT used to calculate Attention. [\[11\]](#)



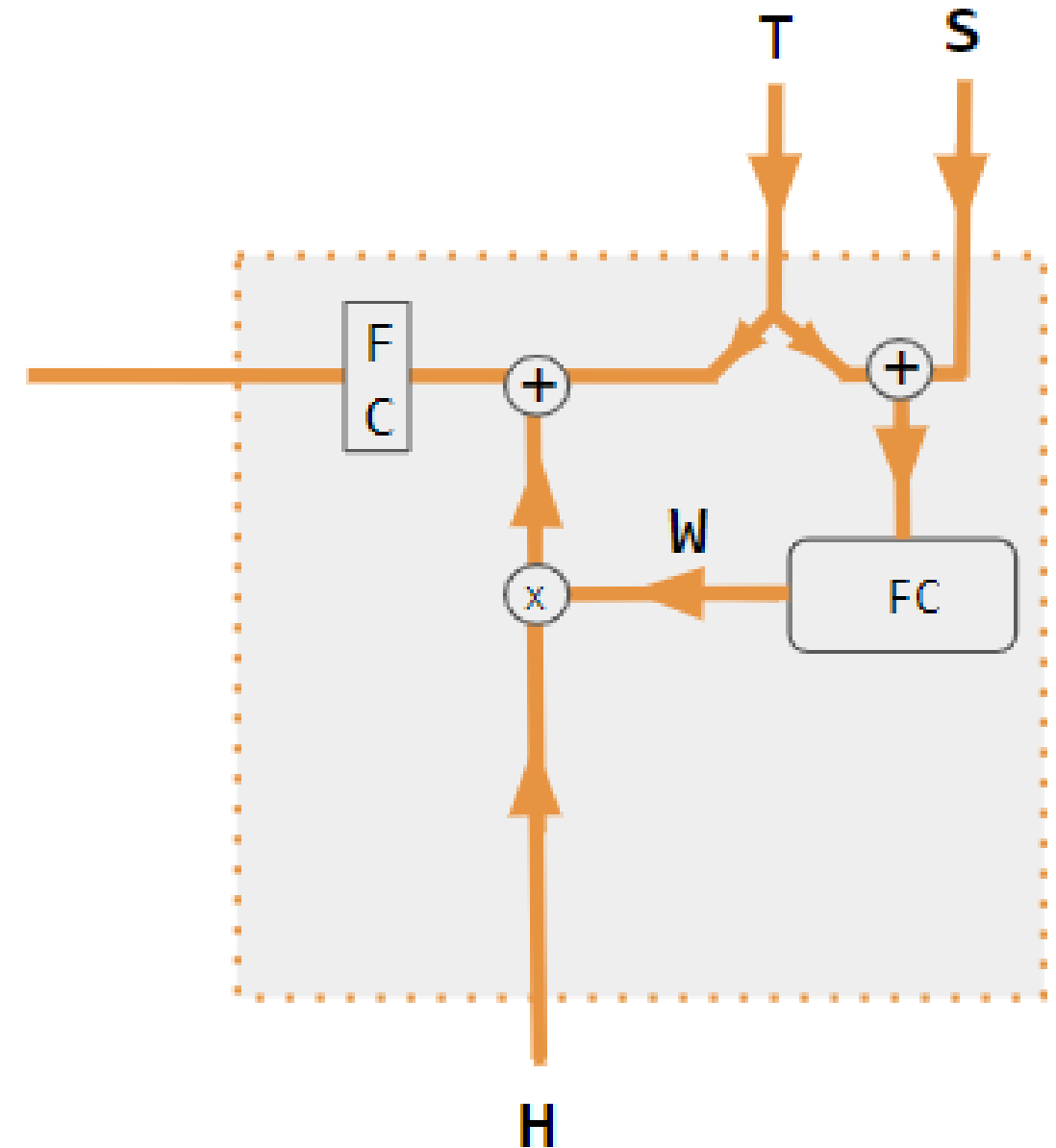
1. Alfredo Canziani & Yann Lecun (2021). [NYU Deep Learning course, Spring 2020](#). Event occurs at 05:30. Retrieved 2021-12-22.

5. PyTorch Tutorial (attention model based on the model available on the PyTorch tutorial)

A FC layer is used to calculate Attention instead of dot product correlation.

S = decoder hidden state, T = target word embedding

H = encoder hidden state, X = input word embedding



1. Robertson, Sean. ["NLP From Scratch: Translation With a Sequence To Sequence Network and Attention"](https://pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html). pytorch.org. Retrieved 2021-12-22.

5. PyTorch Tutorial

Key Idea

- Instead of dot-product (Luong) or QKV self-attention, this variant uses a **feed-forward (FC) layer** to compute attention scores.
- Implements **Bahdanau attention** (also called *additive attention*).
- Allows the decoder to focus on different parts of the encoder's output at each decoding step.

Attention Mechanism Formulation

Given:

- Decoder hidden state h_t (query)
- All encoder hidden states $\bar{h}_s$ (keys/values)

The attention score e_{ts} is computed as:

$$e_{ts} = v_a^T \tanh(W_a h_t + U_a \bar{h}_s)$$

where v_a, W_a, U_a are **trainable weight matrices** (FC layers).

Then:

$$\alpha_{ts} = \frac{\exp(e_{ts})}{\sum_{k=1}^T \exp(e_{tk})}$$
$$\text{context}_t = \sum_{s=1}^T \alpha_{ts} \bar{h}_s$$

5. PyTorch Tutorial

PyTorch Implementation – Attention Module

```
class BahdanauAttention(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.Wa = nn.Linear(hidden_size, hidden_size)
        self.Ua = nn.Linear(hidden_size, hidden_size)
        self.Va = nn.Linear(hidden_size, 1)

    def forward(self, query, keys):
        # query: decoder hidden state, keys: all encoder outputs
        scores = self.Va(torch.tanh(self.Wa(query) + self.Ua(keys)))
        scores = scores.squeeze(2).unsqueeze(1)
        weights = F.softmax(scores, dim=-1)
        context = torch.bmm(weights, keys) # weighted sum
        return context, weights
```

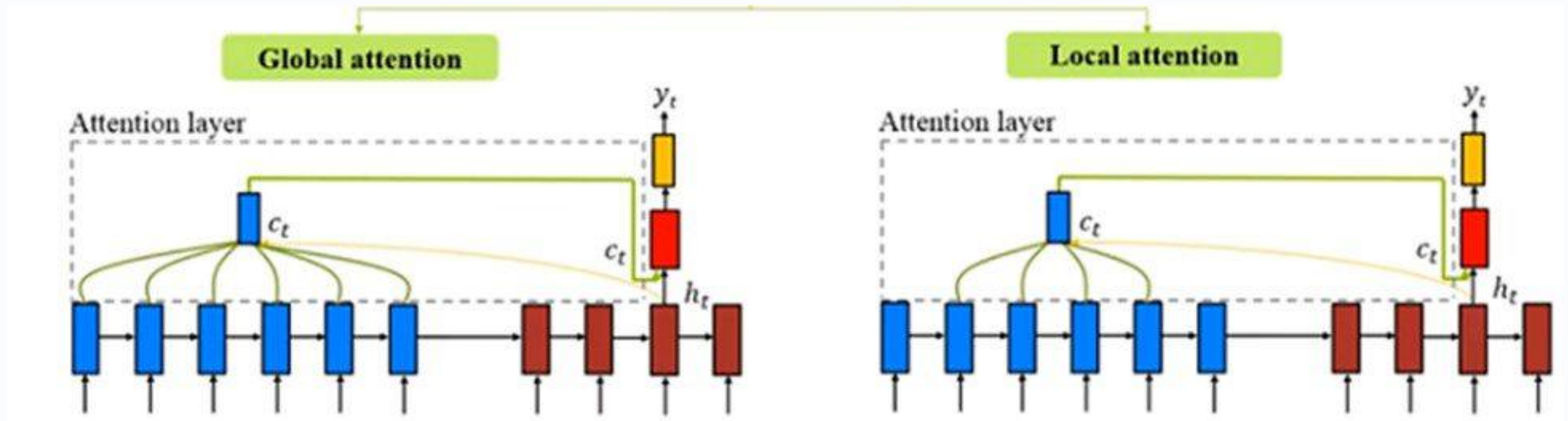
Decoder with Attention

```
class BahdanauAttention(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.Wa = nn.Linear(hidden_size, hidden_size)
        self.Ua = nn.Linear(hidden_size, hidden_size)
        self.Va = nn.Linear(hidden_size, 1)

    def forward(self, query, keys):
        # query: decoder hidden state, keys: all encoder outputs
        scores = self.Va(torch.tanh(self.Wa(query) + self.Ua(keys)))
        scores = scores.squeeze(2).unsqueeze(1)
        weights = F.softmax(scores, dim=-1)
        context = torch.bmm(weights, keys) # weighted sum
        return context, weights
```

Types of attention

Depending on how many source states contribute while deriving the attention vector (α), there can be three types of attention mechanisms:



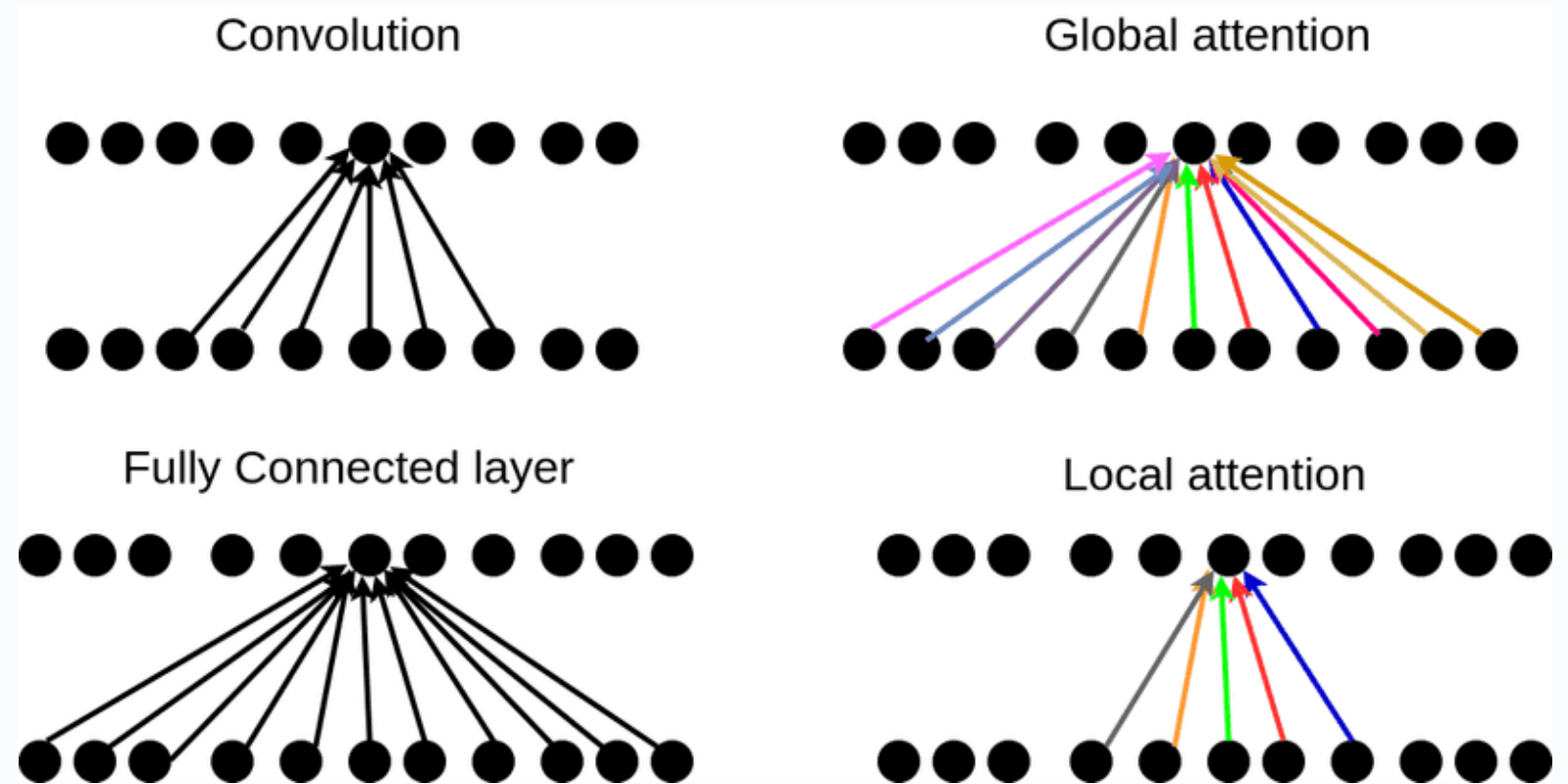
- **Global Attention:** When attention is placed on all source states. In global attention, we require as many weights as the source sentence length.
- **Local Attention:** When attention is placed on a few source states.
- **Hard Attention:** When attention is placed on only one source state.

You can check out this [Kaggle notebook](#) or [GitHub repo](#) to implement NMT with the attention mechanism using TensorFlow.

Types of attention

A comparison of different attention mechanisms within neural network architectures:

- **Convolution** restricts the receptive field, focusing on a local neighborhood of input elements to produce an output element.
- **Fully Connected Layer** connects every input element to every output element, allowing for a global view but with a high parameter count.
- **Global Attention** allows a model to consider the entire input sequence when computing attention scores for each target, making it useful when all input parts contribute to the output.
- **Local Attention** focuses only on a small window of context around specific input elements, similar to convolution but typically with learned weights.



Transformers

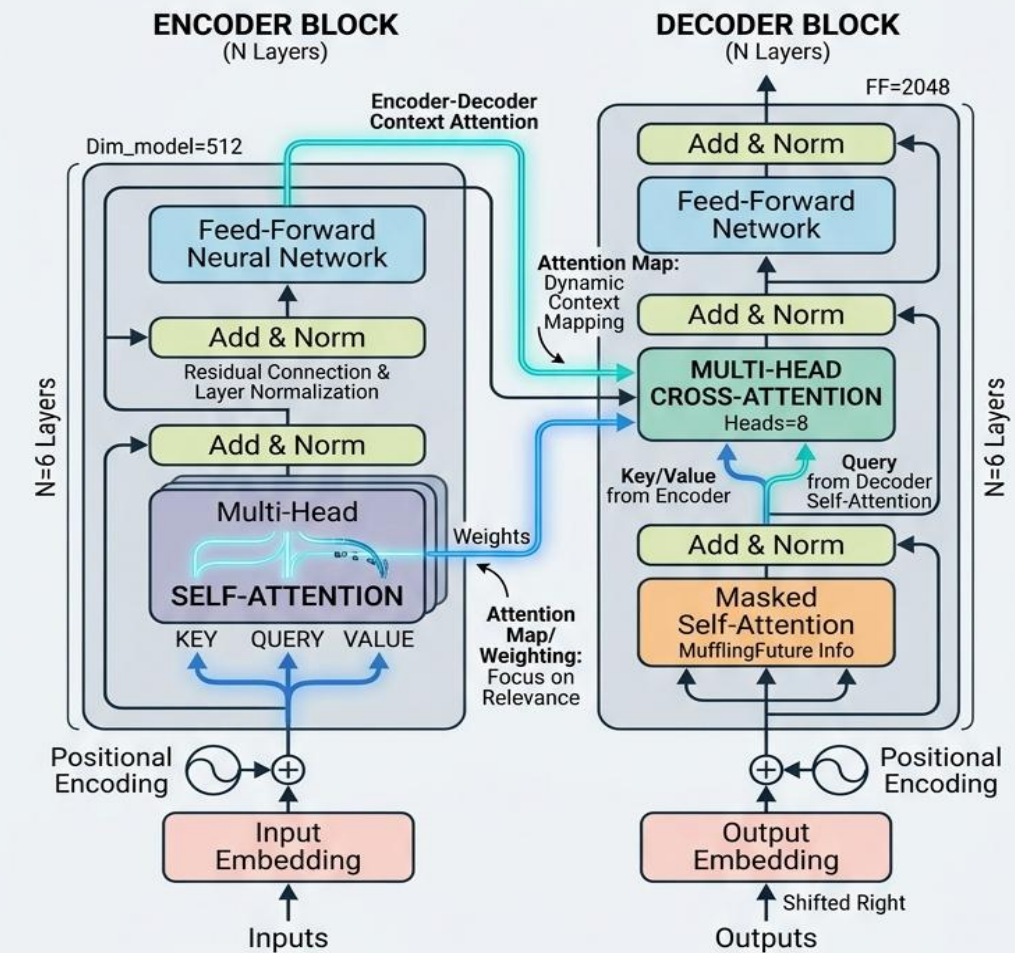
A transformer is a deep learning model that generates significant weights to each part of the input data through using mechanisms of self-attention and cross-attention. It is used primarily in the fields of natural language processing (NLP), computer vision (CV), and speech processing.

* The vision transformer models are marked in red.

TRANSFORMER DEEP LEARNING MODEL

A Comprehensive Technical Architectural Overview

Transformer Architecture & Mechanisms



APPLICATIONS:
NLP
(Natural Language Processing)

Translation,
Summarization,
Sentiment Analysis

Source/Target
Sentences



APPLICATIONS:
CV
(Computer Vision)



Vision Transformers (ViT),
Image Classification,
Object Detection



APPLICATIONS:
**SPEECH
PROCESSING**



Automatic Speech
Text-to-Speech,
Audio Classification

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

■ Natural Language Processing (NLP) ■ Computer Vision ■ Multimodal & Generative AI ■ Reasoning, Agents & Future AI



■ Natural Language Processing (NLP) ■ Computer Vision ■ Multimodal & Generative AI ■ Reasoning, Agents & Future AI

What is Tokenization in Transformers?

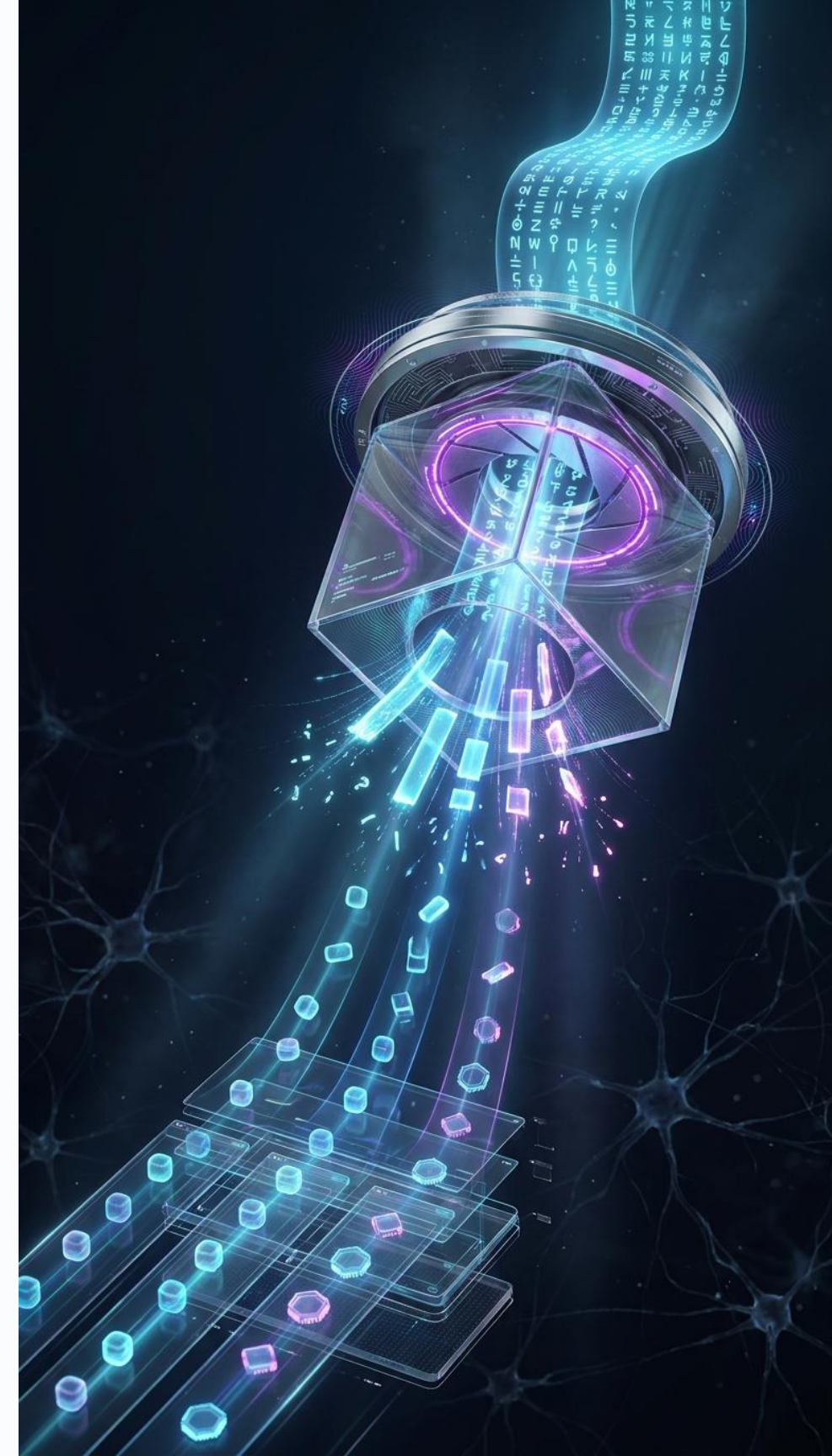
Tokenization is the first step in every Natural Language Processing (NLP) system and Transformer-based model. Computers cannot directly understand raw text such as sentences or paragraphs. Therefore, text must first be converted into smaller units called **tokens**.

A token may represent:

- a complete word,
- part of a word,
- a single character,
- or even punctuation symbols.

For example, consider the sentence:

“Transformers are changing AI.”



What is Tokenization in Transformers?

After tokenization, the sentence may become:

Original Text	Tokens
Transformers are changing AI.	["Transformers", "are", "changing", "AI", "."]

The tokenizer acts like a bridge between human language and machine learning models. Its job is to transform readable text into numerical representations that neural networks can process.

Modern Transformer architectures such as BERT, GPT, and Vision-Language Models strongly depend on efficient tokenization methods because the quality of tokenization directly affects:

- model accuracy,
- vocabulary size,
- memory usage,
- and computational efficiency.



Types of Tokenization Methods

Different Transformer models use different tokenization strategies depending on the application and vocabulary size.

The three most common tokenization methods are:

1. Word-Level Tokenization

In this method, each word becomes a separate token.

Example:

Sentence	Tokens
"Deep learning is powerful"	["Deep", "learning", "is", "powerful"]

Advantages:

- Simple and easy to understand.

Disadvantages:

- Very large vocabulary,
- Cannot handle unknown words effectively.



Types of Tokenization Methods

2. Character-Level Tokenization

Each individual character becomes a token.

Example:

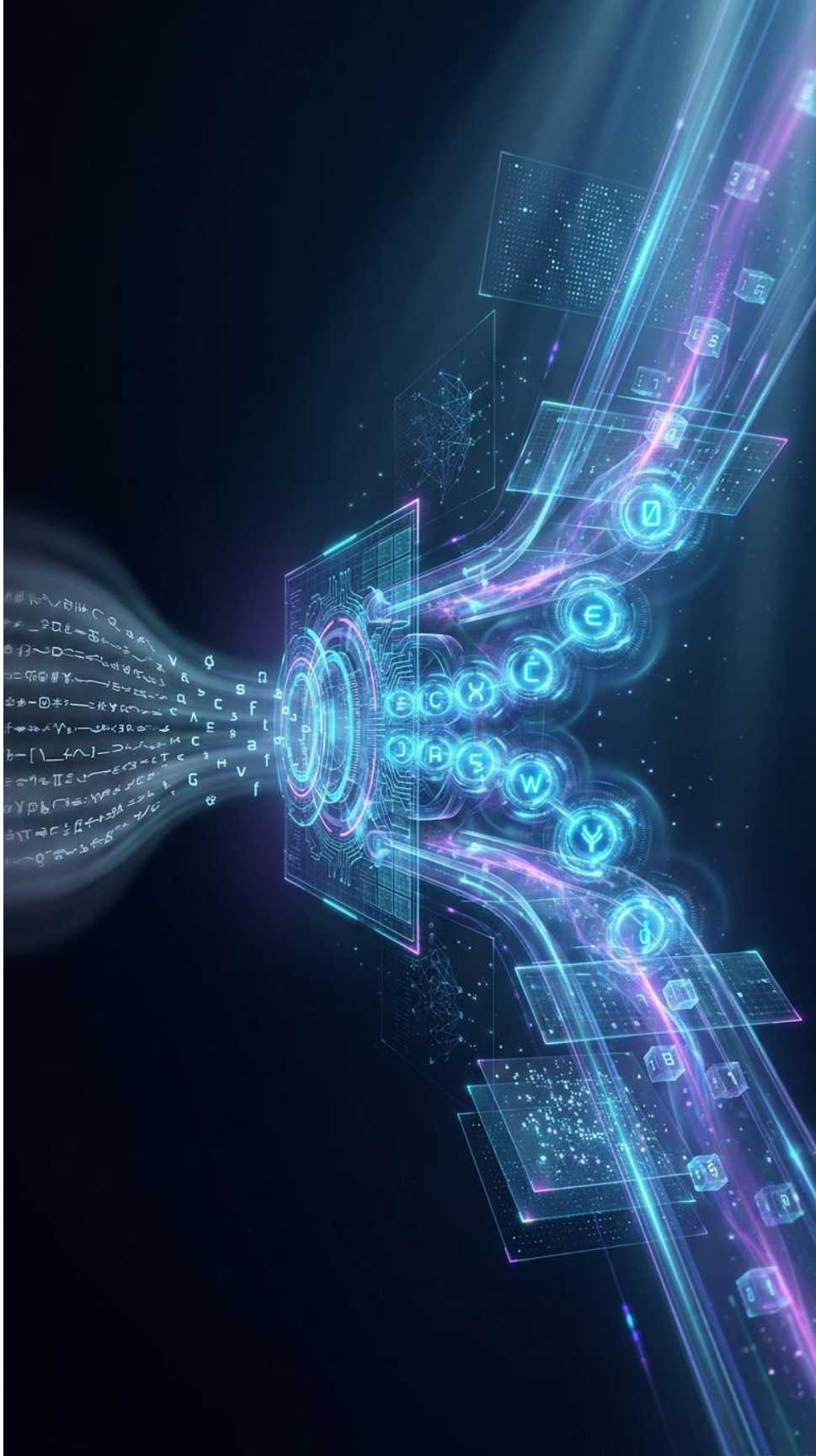
Sentence	Tokens
"AI"	["A", "I"]

Advantages:

- No unknown words,
- Small vocabulary.

Disadvantages:

- Longer sequences,
- Higher computational cost.



Types of Tokenization Methods

3. Subword Tokenization (Most Important)

Modern Transformers mainly use subword tokenization techniques such as:

- Byte Pair Encoding (BPE),
- WordPiece,
- SentencePiece.

Example:

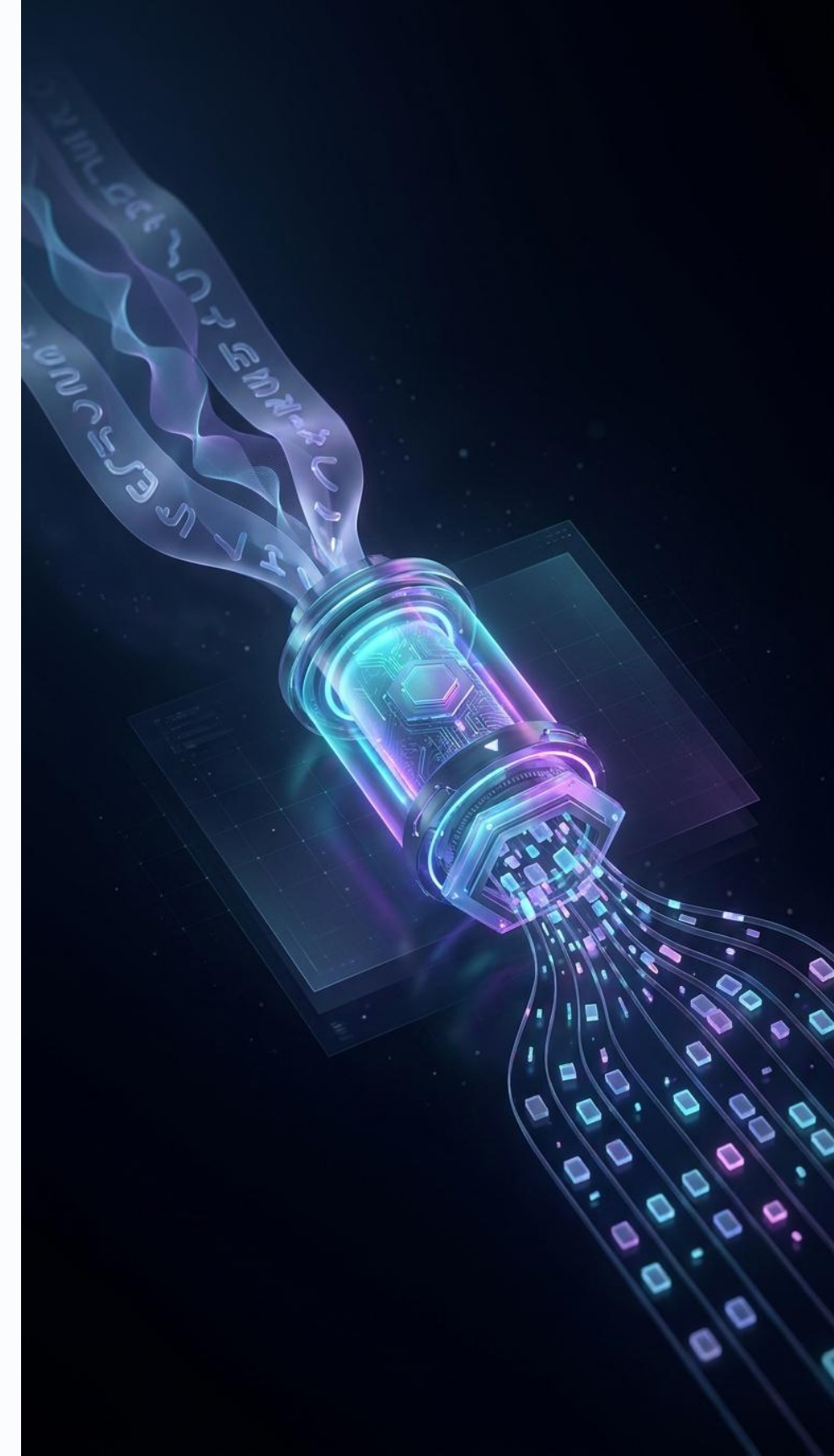
Sentence	Tokens
"unhappiness"	["un", "happy", "ness"]

Subword tokenization is powerful because it:

- reduces vocabulary size,
- handles rare words,
- preserves semantic meaning,
- improves generalization.

This is the standard approach used in models such as:

- GPT,
- BERT,
- T5,
- LLaMA.



Converting Tokens into Numerical IDs

After tokenization, Transformer models still cannot process text directly because neural networks only work with numbers. Therefore, each token must be converted into a unique numerical identifier called a **Token ID**.

This process uses a predefined vocabulary table called a **Tokenizer Vocabulary**.

Example sentence:

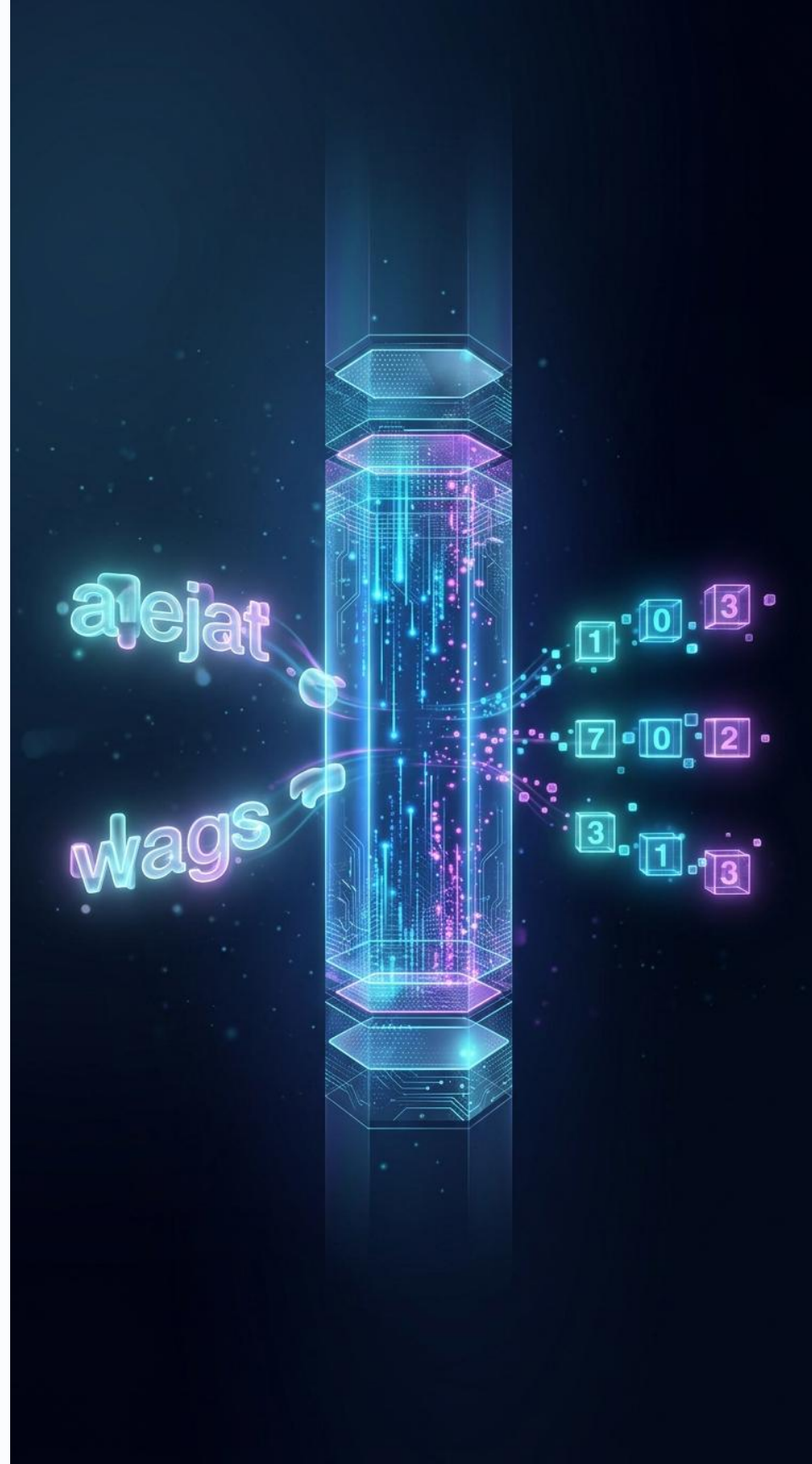
“Thinking Machines”

After tokenization:

[“Thinking”, “Machines”]

The tokenizer then searches inside its vocabulary table:

Token	Token ID
Thinking	2451
Machines	7312



Converting Tokens into Numerical IDs

The final output becomes:

[2451, 7312]

These numerical IDs are then passed into the embedding layer of the Transformer.

Important Concept: Vocabulary Lookup

The tokenizer maintains a large lookup table that maps:

- tokens → integer IDs

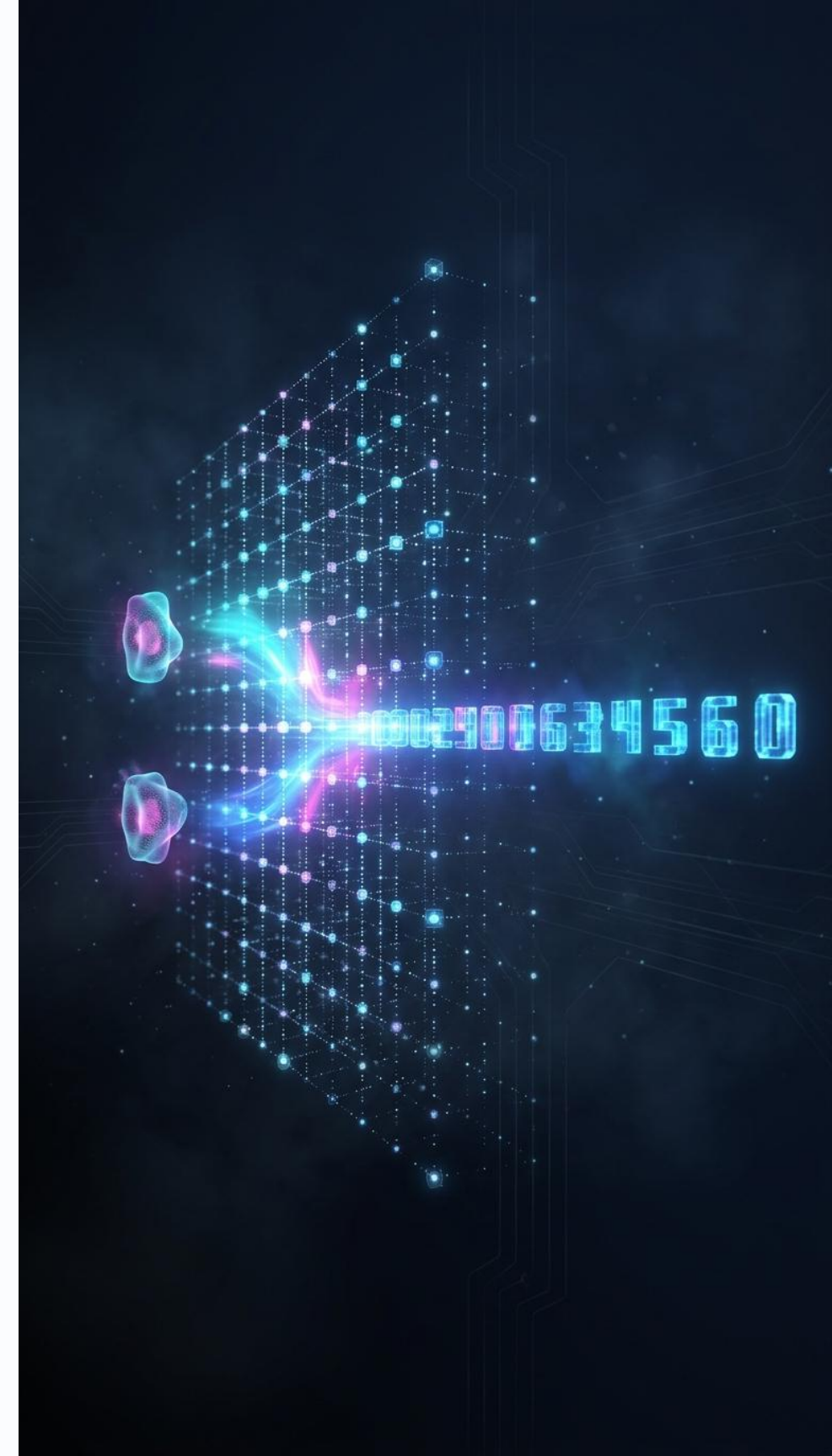
This lookup table is fixed after training and allows the model to efficiently convert text into machine-readable form.

For large language models:

- vocabulary size may exceed 50,000 tokens,
- some advanced models use vocabularies larger than 100,000 tokens.

Token IDs are the foundation of:

- embeddings,
- positional encoding,
- self-attention,
- and all Transformer computations.



Input Embedding: Converting Token IDs into Meaningful Vectors

After tokenization, the input sentence is converted into Token IDs.

However, these numerical IDs still do not contain any semantic meaning.

For example:

Token	Token ID
Thinking	2451
Machines	7312

The numbers 2451 and 7312 are simply identifiers and do not represent the meaning of the words.

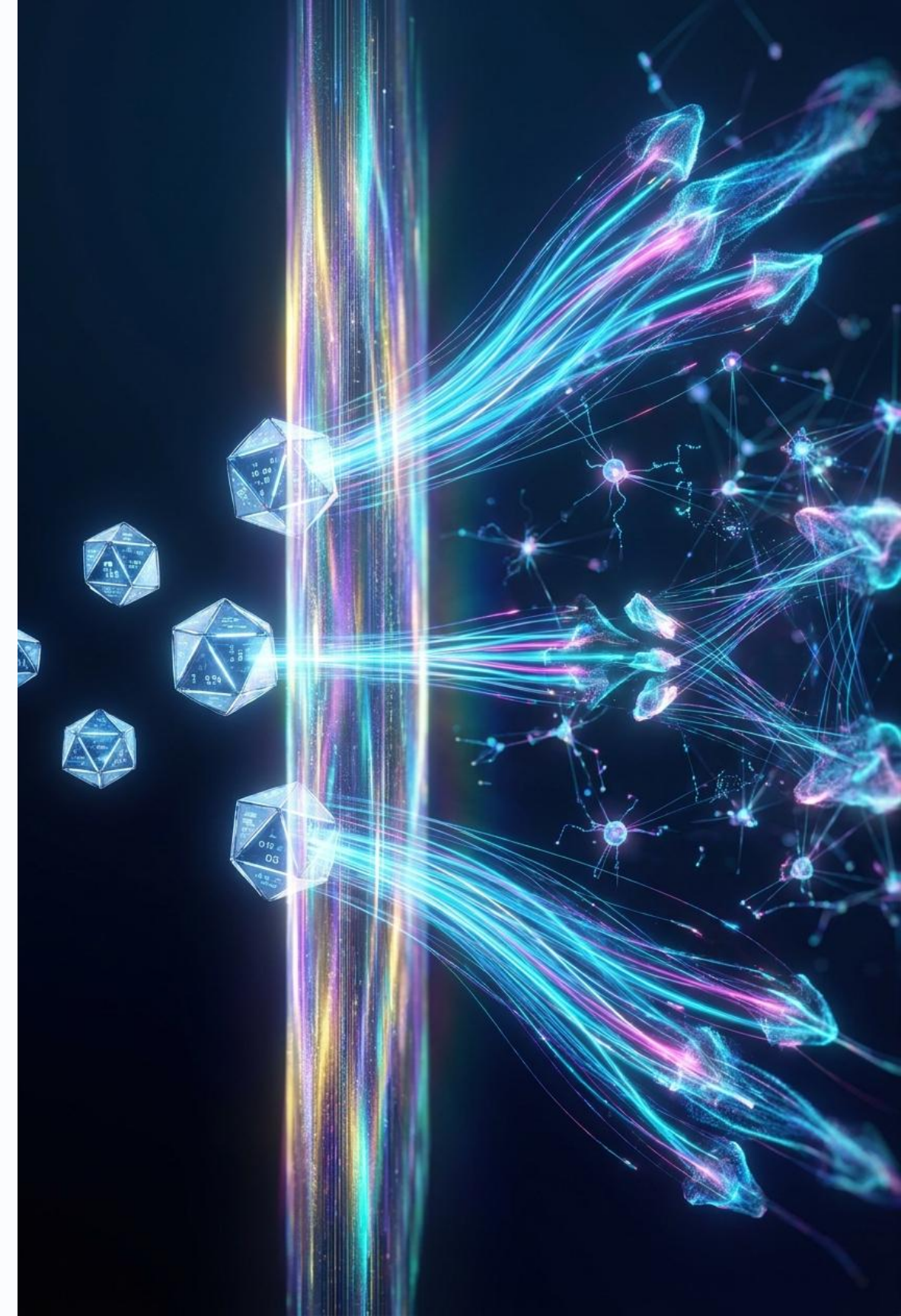
To solve this problem, Transformers use an **Embedding Layer**.

The embedding layer maps each token ID into a dense numerical vector called an **Embedding Vector**.

These vectors are learned during training and contain semantic information about words.

Example:

Words with similar meanings usually produce similar embedding vectors in high-dimensional space.



Input Embedding: Converting Token IDs into Meaningful Vectors

For example:

Token	Embedding Vector
Thinking	[0.21, -0.44, 0.73, 0.18]
Machines	[0.55, 0.11, -0.29, 0.67]

- “car” and “vehicle” may have similar embeddings,
- while “car” and “banana” will be very different.

The embedding layer is essentially a large trainable lookup table:

$$\text{Embedding} = \text{Vocabulary Size} \times \text{Embedding Dimension}$$

For example:

- Vocabulary Size = 50,000
- Embedding Dimension = 512

Then the embedding matrix size becomes:

$$50000 \times 512$$

Each row corresponds to one token in the vocabulary.

The output embeddings are then passed to the positional encoding stage before entering the Transformer encoder.



Positional Encoding: Teaching Order to Transformers

Unlike Recurrent Neural Networks (RNNs), Transformers process all tokens simultaneously in parallel.

Because of this parallel processing, Transformers do not naturally understand the order of words in a sentence.

For example:

- “***Dog bites man***”
- “***Man bites dog***”

contain the same words but completely different meanings.

Therefore, positional information must be added to the embeddings so the model can understand sequence order.

This is achieved using **Positional Encoding (PE)**.

The positional encoding vector is added directly to the input embedding vector:

$$Input = Embedding + Positional\ Encoding$$

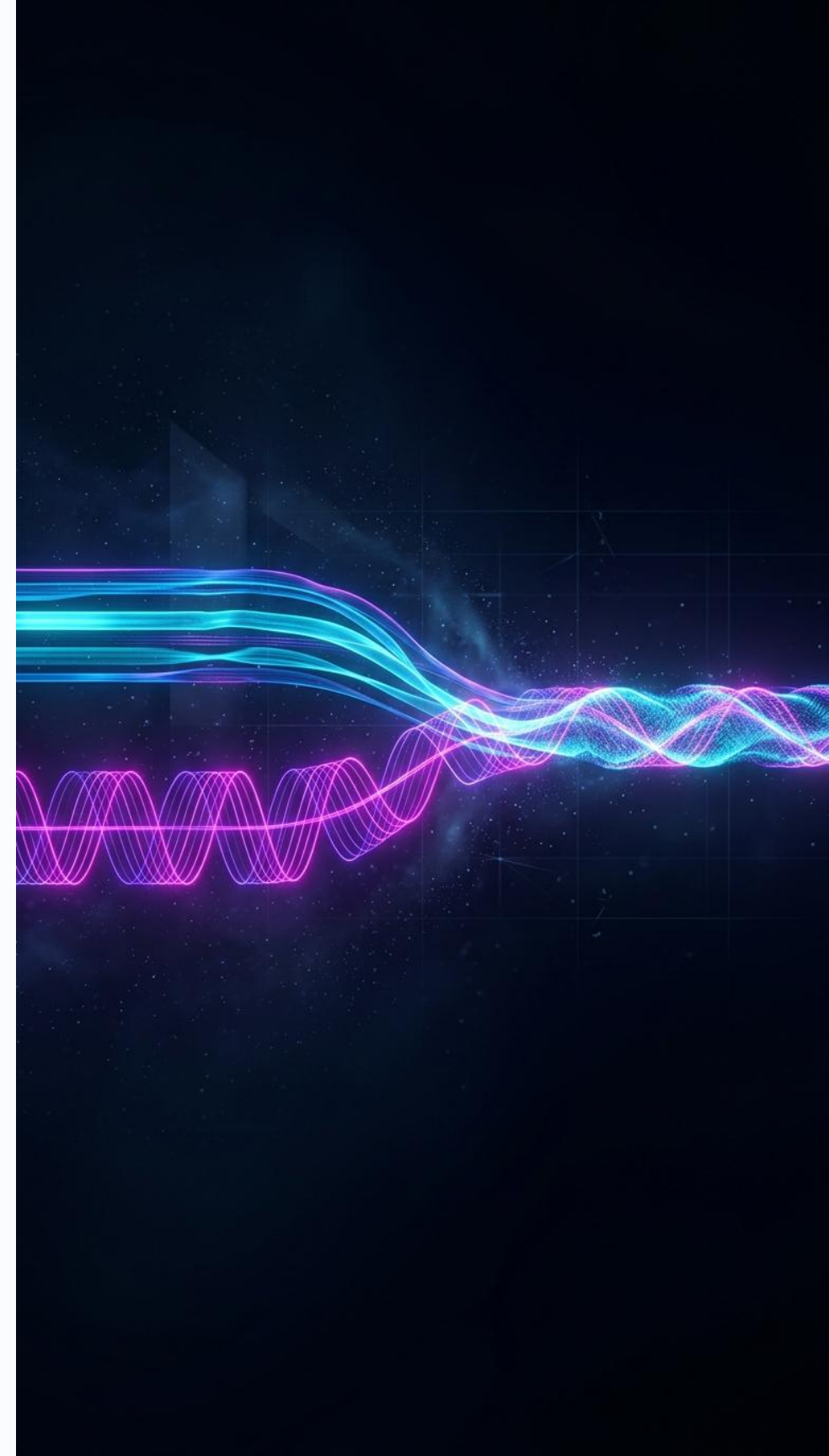
The original Transformer paper uses sinusoidal functions to generate positional encodings.

Even dimensions use sine functions:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Odd dimensions use cosine functions:

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$



Positional Encoding: Teaching Order to Transformers

Where:

- pos = token position,
- i = embedding dimension index,
- d_{model} =embedding size.

These sinusoidal patterns allow the model to:

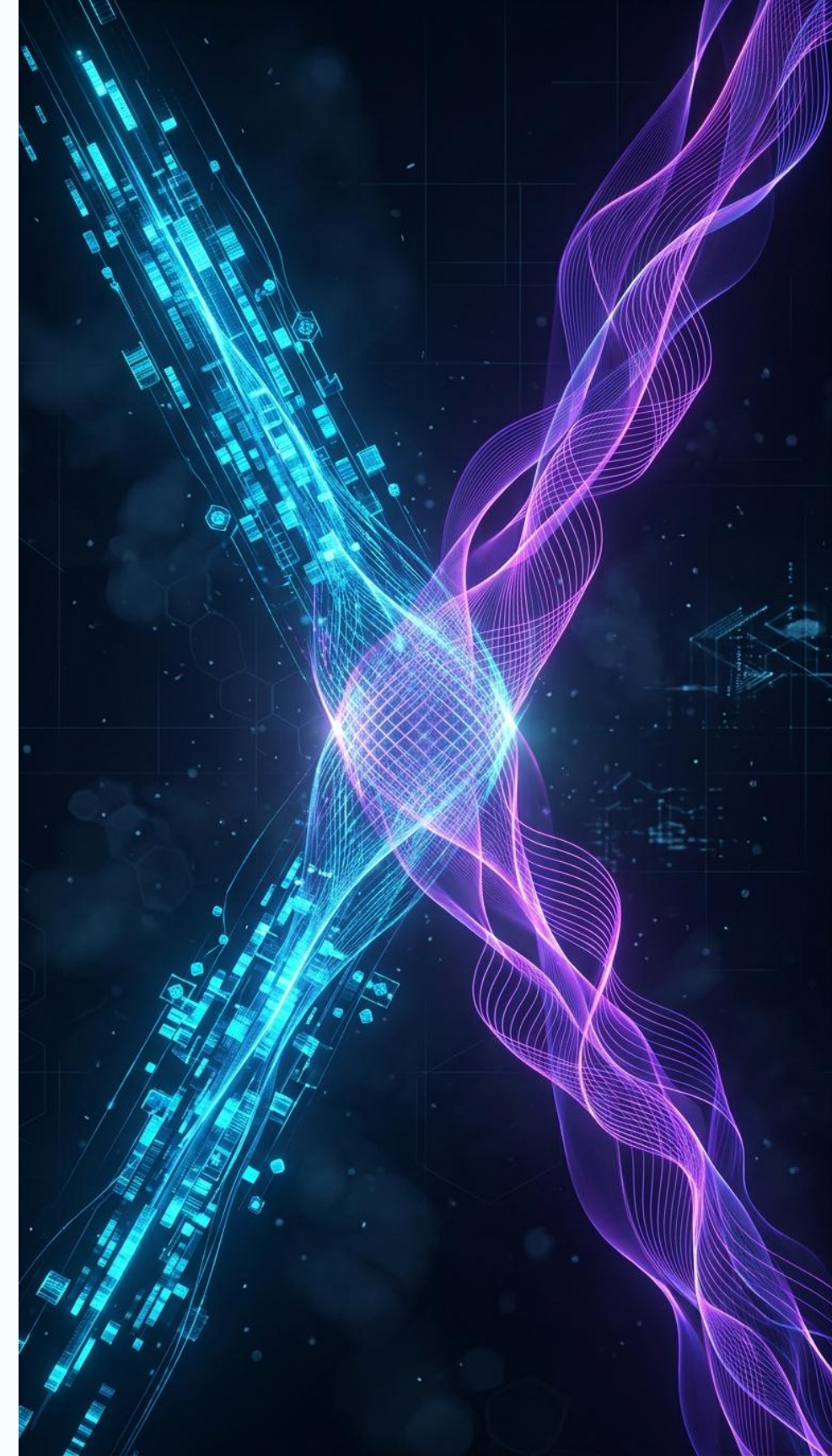
- learn relative distances between words,
- detect short-range and long-range dependencies,
- generalize to longer sequences.

Example:

Position	Encoding Example
0	[0, 1, 0, 1]
1	[0.84, 0.54, 0.01, 0.99]
2	[0.91, -0.42, 0.02, 0.98]

The final vector entering the Transformer is:

$$Final\ Input = Word\ Embedding + Positional\ Encoding$$



Query, Key, and Value in Self-Attention

The most important operation inside Transformers is the **Self-Attention Mechanism**.

Self-attention allows each word to determine which other words in the sentence are important.

To achieve this, every input embedding is transformed into three vectors:

- Query (Q)
- Key (K)
- Value (V)

These vectors are generated using learned weight matrices:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

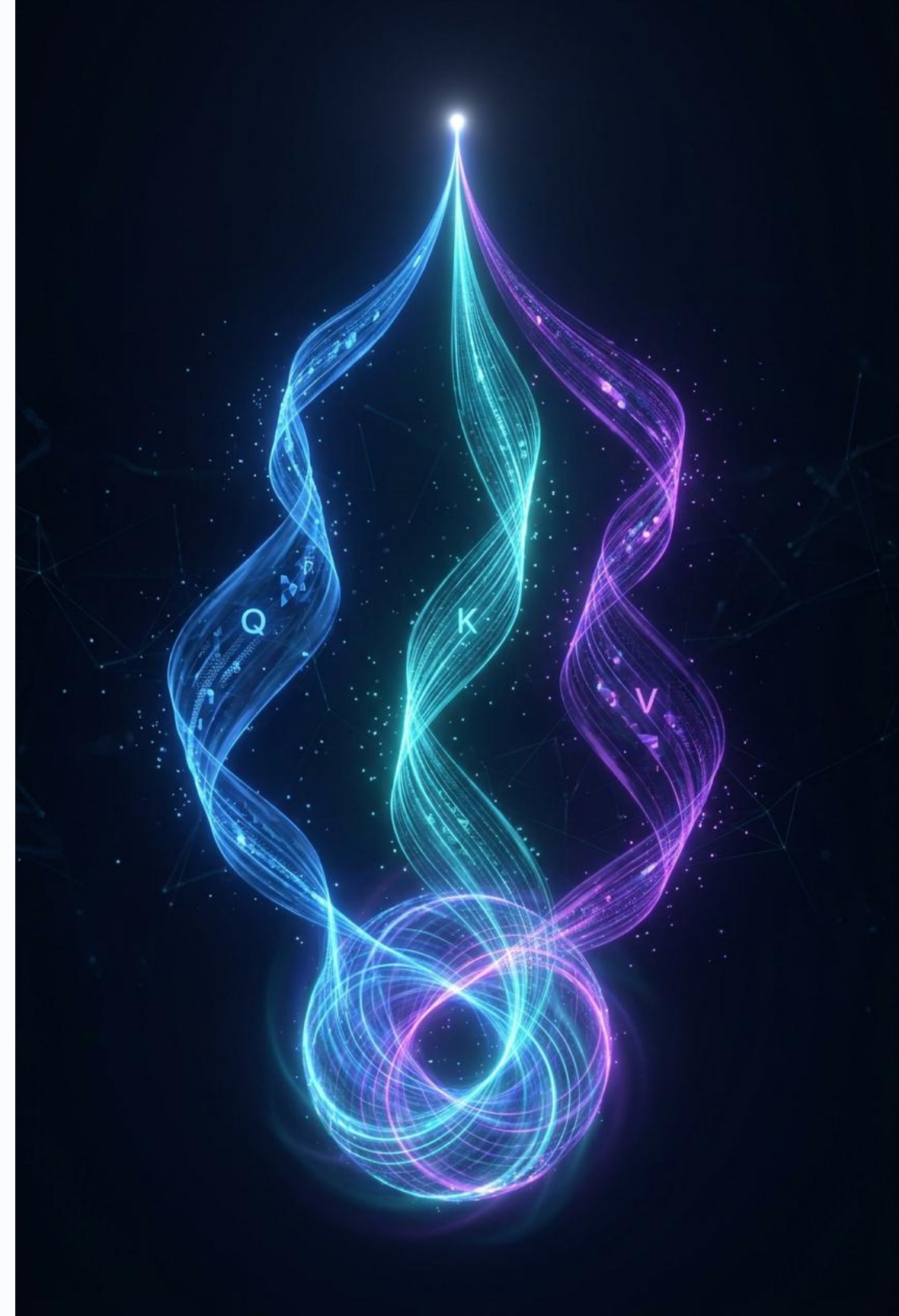
Where:

- X = input embedding matrix,
- W_Q, W_K, W_V = trainable weight matrices.

Numerical Example

Suppose we have the sentence:

"Thinking Machines"



Query, Key, and Value in Self-Attention

Input embeddings:

Token	Embedding
Thinking	[1, 0, 1]
Machines	[0, 1, 1]

Assume:

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

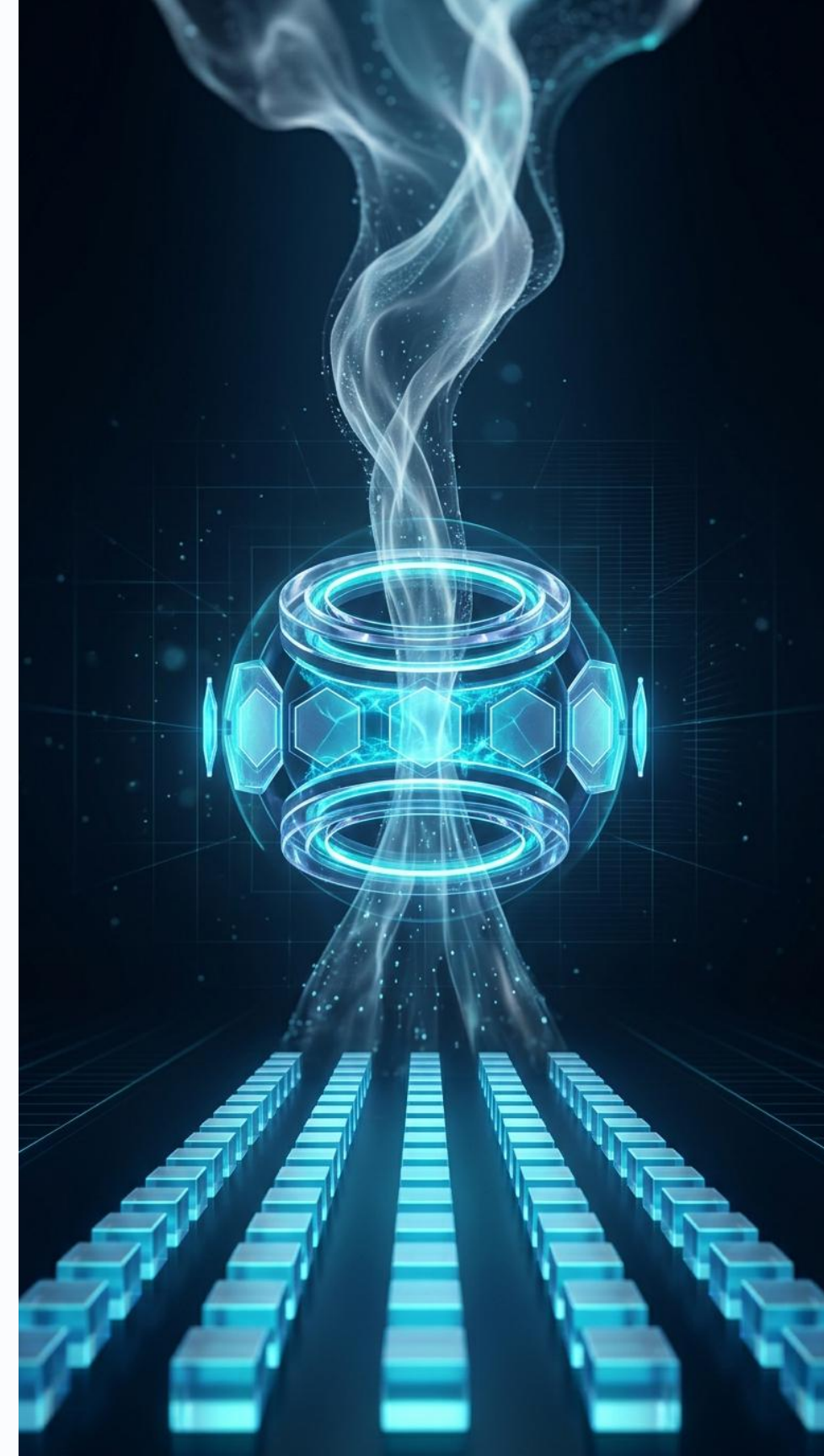
Then:

For “Thinking”:

$$Q_1 = [1, 0, 1]W_Q = [2, 1]$$

Similarly, assume:

$$W_K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$



Query, Key, and Value in Self-Attention

Then:

$$K_1 = [1, 0, 1]W_K = [2, 1]$$

For “Machines”:

$$K_2 = [0, 1, 1]W_K = [1, 1]$$

Attention Score Calculation

The attention score is computed using the dot product between Query and Key vectors.

Attention between “Thinking” and itself:

$$Q_1 \cdot K_1 = (2)(2) + (1)(1) = 5$$

Attention between “Thinking” and “Machines”:

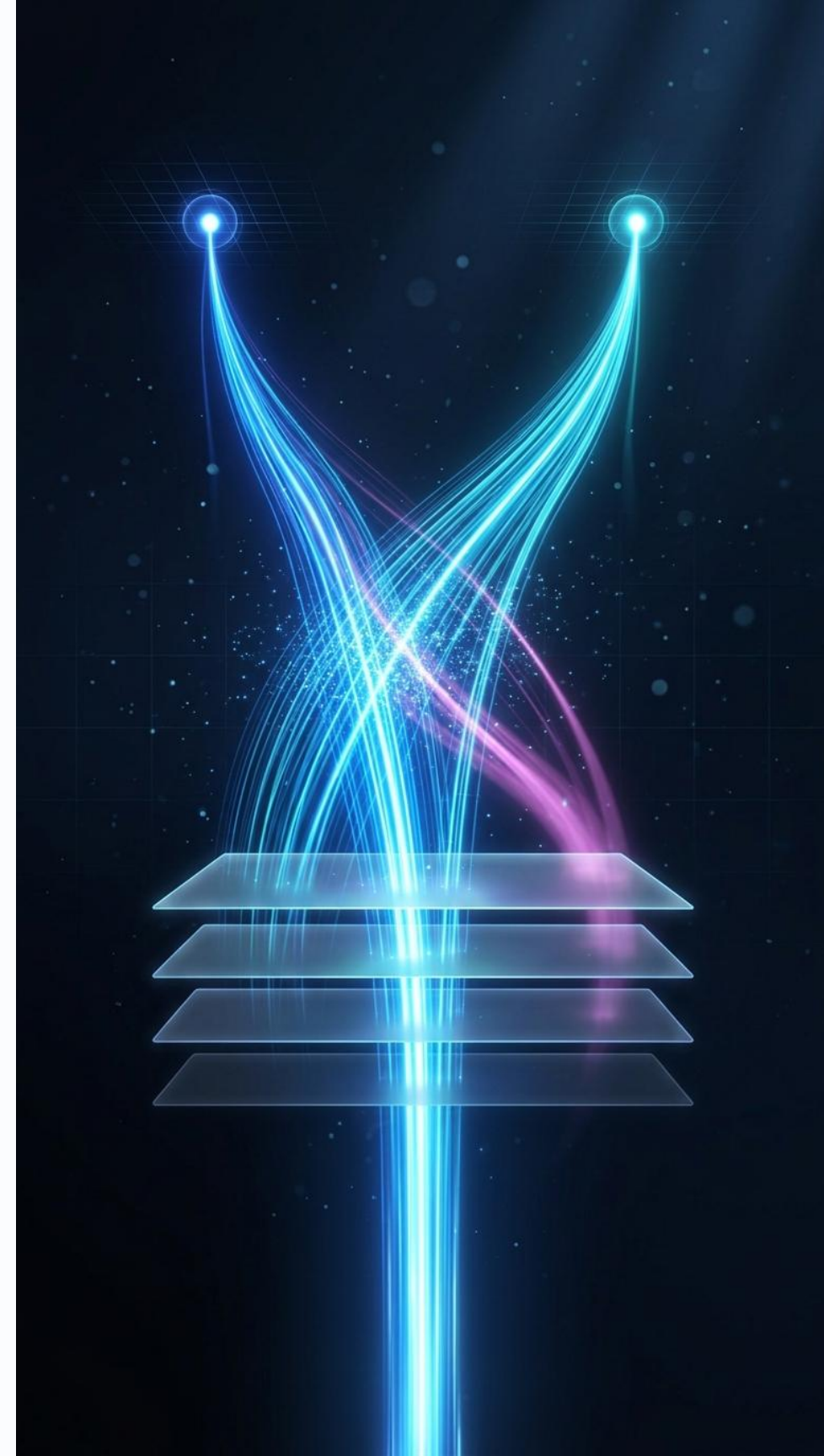
$$Q_1 \cdot K_2 = (2)(1) + (1)(1) = 3$$

A larger score means a stronger relationship between tokens.

Finally:

- scores are normalized using Softmax,
- weighted values are combined,
- and the final attention output is produced.

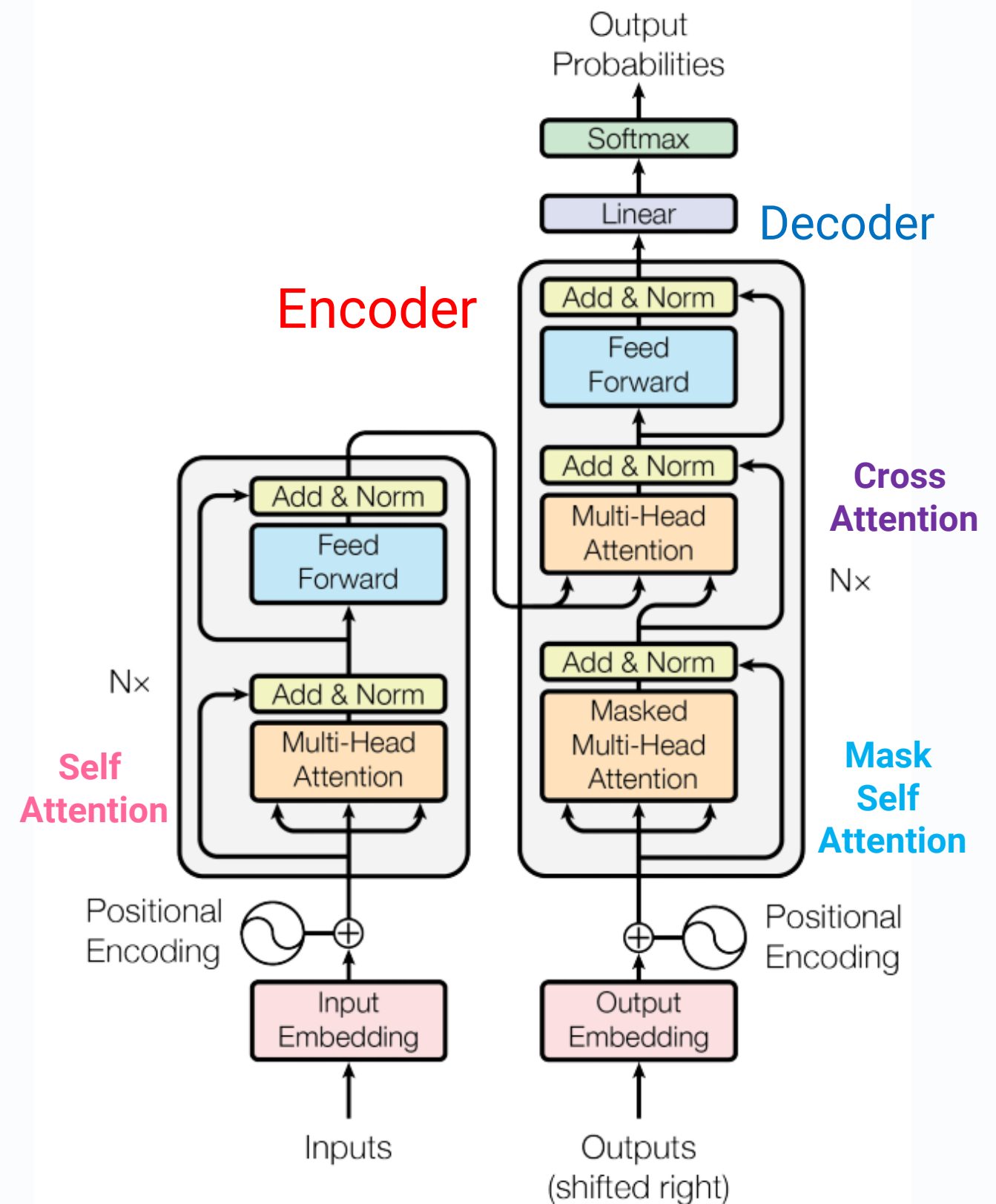
This mechanism allows Transformers to dynamically focus on the most relevant words in a sentence.



Vanilla Transformer

*Vaswani... 2017 (Attention is all you need)

- A sequence-to-sequence model and consists of an encoder and a decoder, each of which is a stack of “N” identical blocks each *encoder block* is mainly composed of a multi-head self-attention module and a position-wise feed-forward network (FFN). In this work, the encoder is composed of a stack of N=6 identical layers.
- For building a deeper model, a residual connection is employed around each module, followed by Layer Normalization module.
- Compared to the encoder blocks, **decoder** blocks additionally insert **cross-attention** modules between the multi-head self-attention modules and the position-wise FFNs. In this work, the decoder is also composed of a stack of N=6 identical layers.
- Furthermore, the self-attention modules in the decoder are adapted to prevent each position from attending to subsequent positions.



Vanilla Transformer

The key modules of the vanilla Transformer

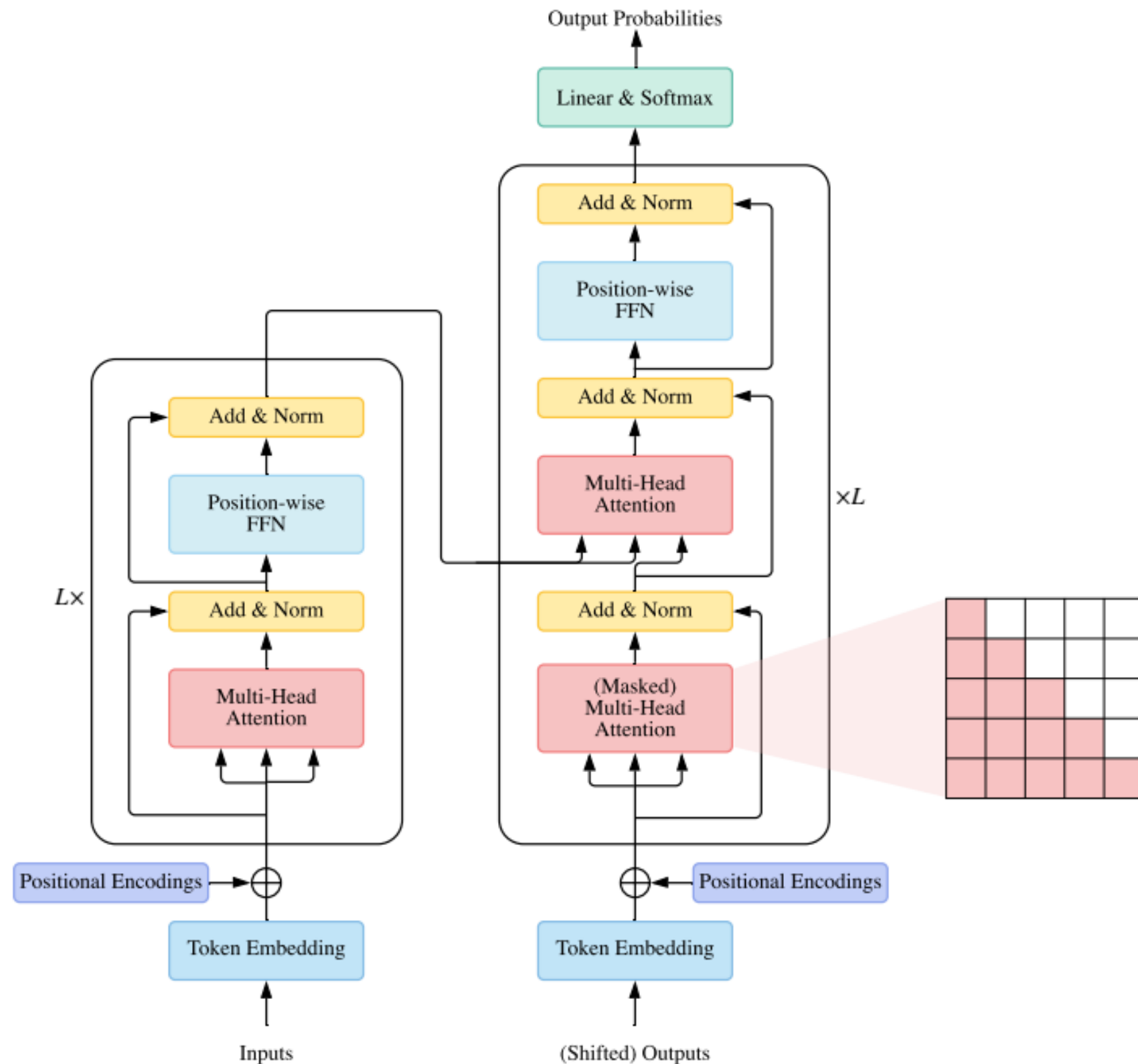
1. Attention Modules

- Single Attention
- Multi Head-Attention
 - Self-attention
 - Masked Self-attention
 - Cross-attention

2. Position-wise FFN (Feed Forward Network)

3. Residual Connection and Normalization.

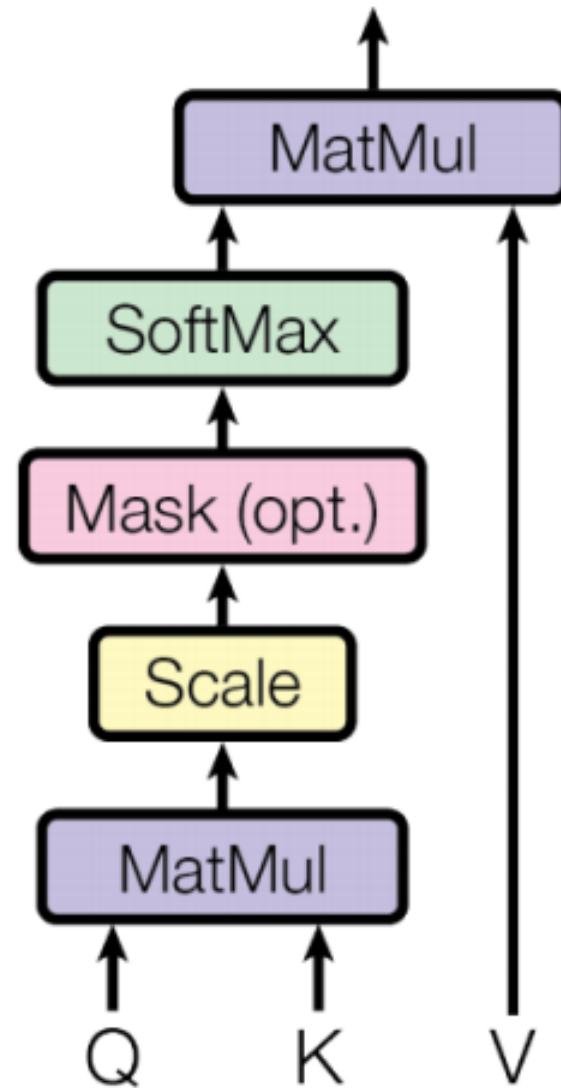
4. Position Encodings.



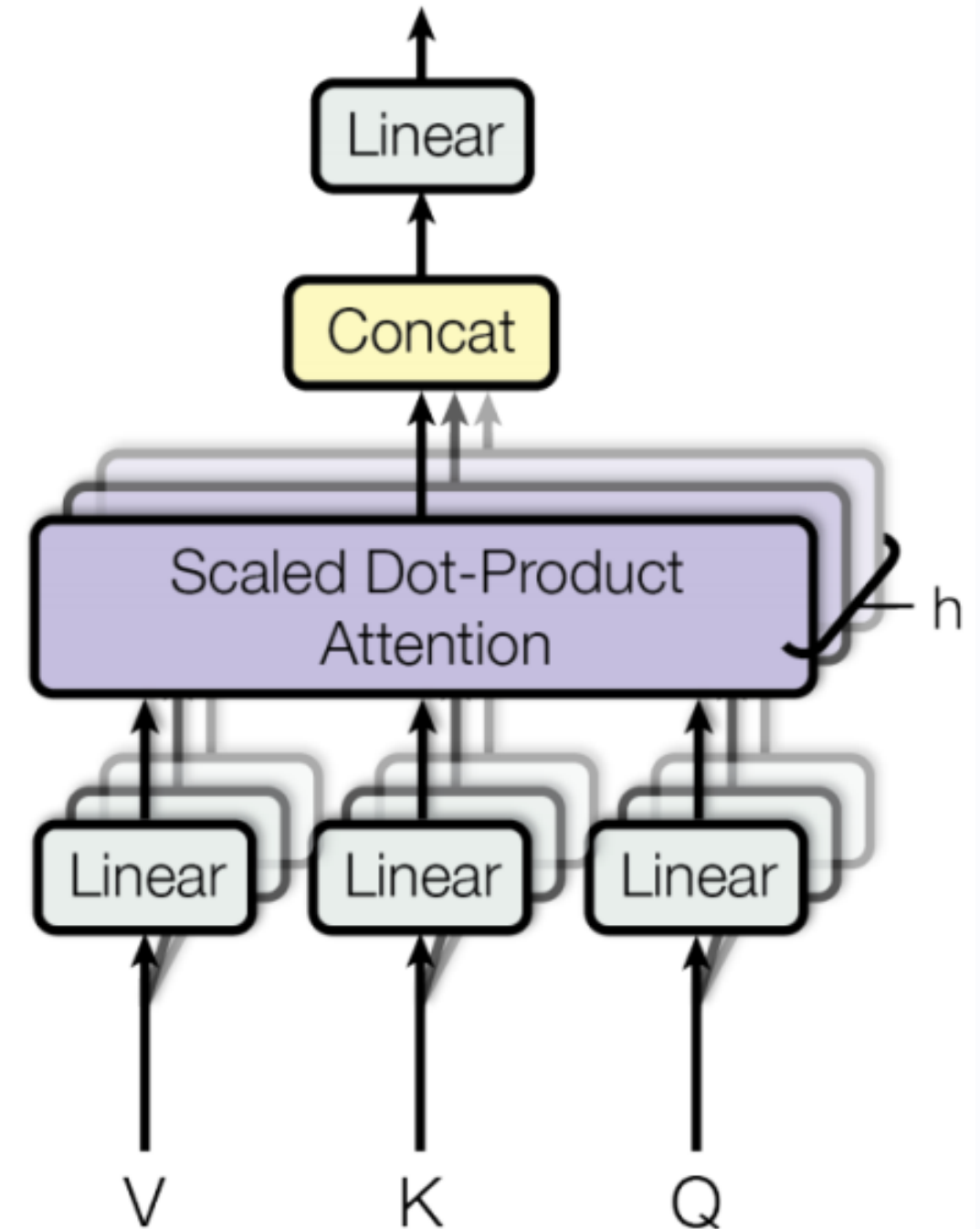
Overview of vanilla Transformer architecture

Vanilla Transformer

In this work we employ $h=8$ parallel attention layers, or heads



Single-attention function



Multi-head attention function.

1. Attention Modules

- **Single attention function**

Transformer adopts attention mechanism with Query-Key-Value (QKV) model. Given the packed matrix representations of queries, keys, and values, the scaled dot-product attention used by Transformer is given by

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{D_k}}\right), Q \in R^{N \times D_k}, K \in R^{M \times D_k}, V \in R^{M \times D_v}$$

N : length of Query, M : length of Keys (or values), D_k : Dimension of query and key and D_v : Dimension of values

- Softmax is applied in a row-wise manner.
- The dot-products of queries and keys are divided by $\sqrt{D_k}$ to alleviate gradient vanishing problem of the softmax function.
- **Multi head attention function**

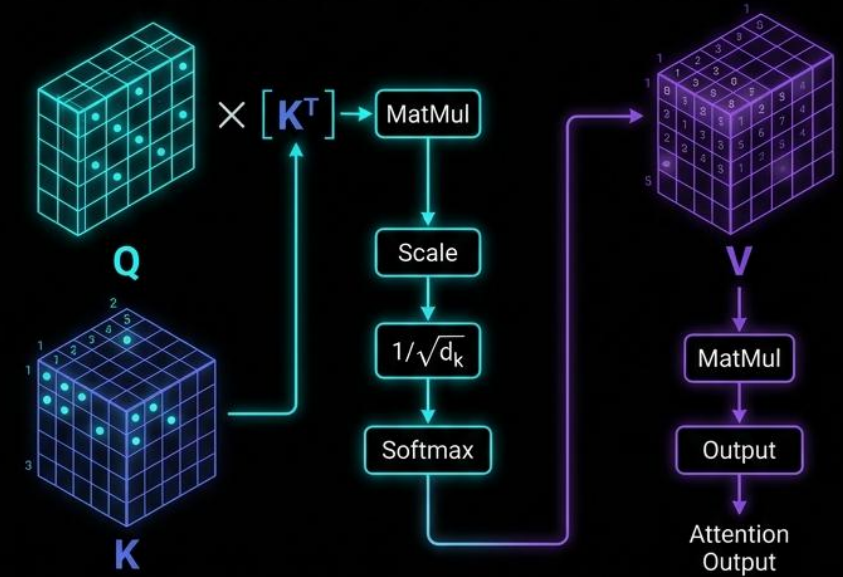
Instead of simply applying a single attention function, Transformer uses multi-head attention, where the original queries, keys and values are with H different sets of learned projections.

$$\text{MultiHeadAttn}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O$$
$$\text{head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right), \quad i = 1, 2, \dots, H$$

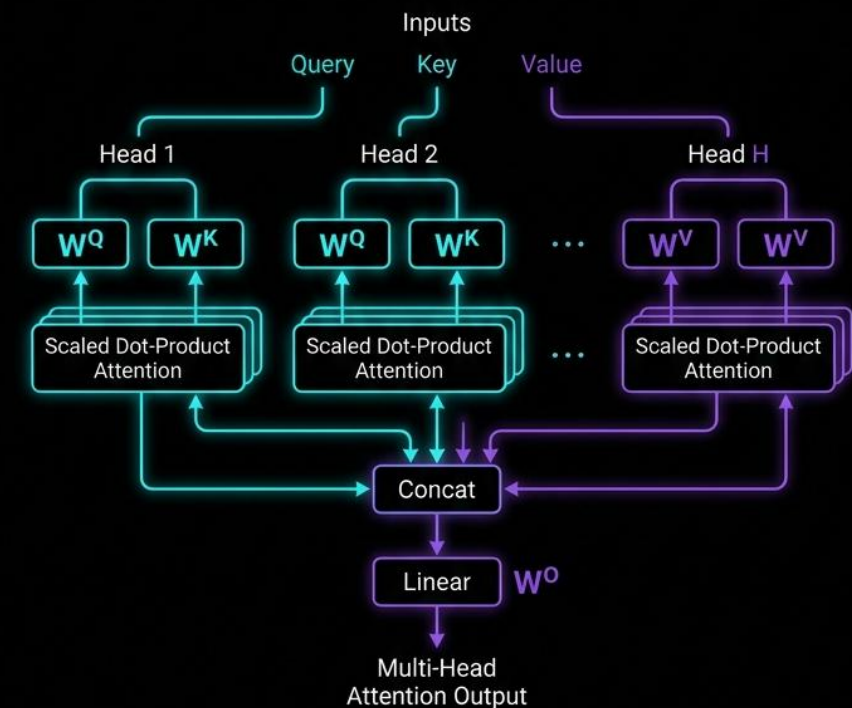
The model then concatenates all the outputs.

TRANSFORMER MULTI-HEAD ATTENTION MECHANISM

SCALED DOT-PRODUCT ATTENTION



MULTI-HEAD ATTENTION



Three types of attention

(in terms of the source of queries and key-value pairs)

- **Self-attention**

In Transformer encoder, we set $Q = K = V = X$, where X is the outputs of the previous layer.

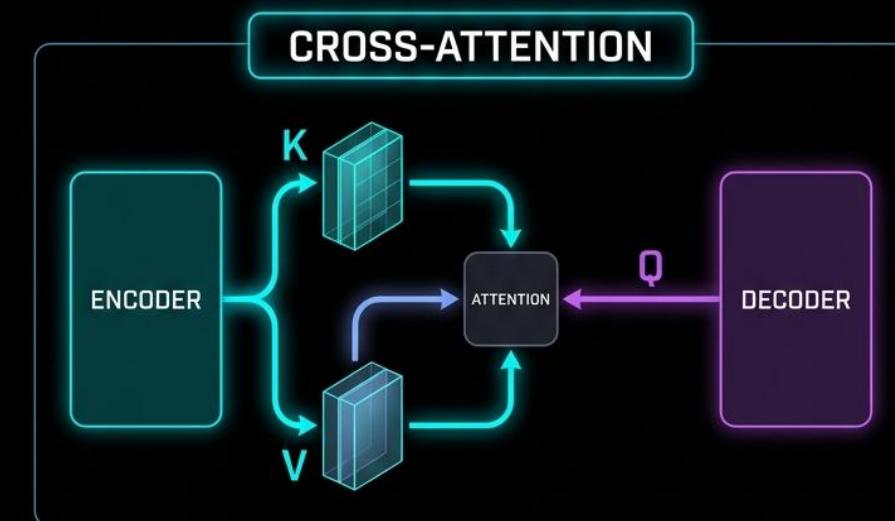
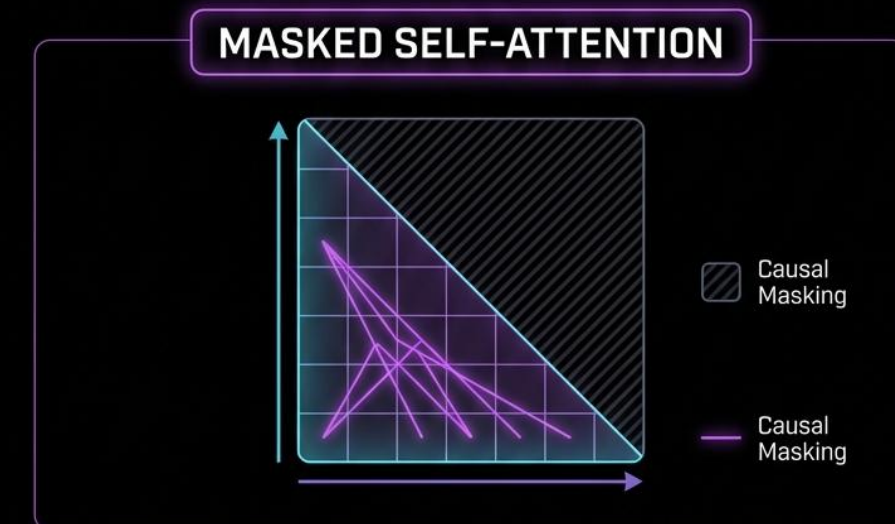
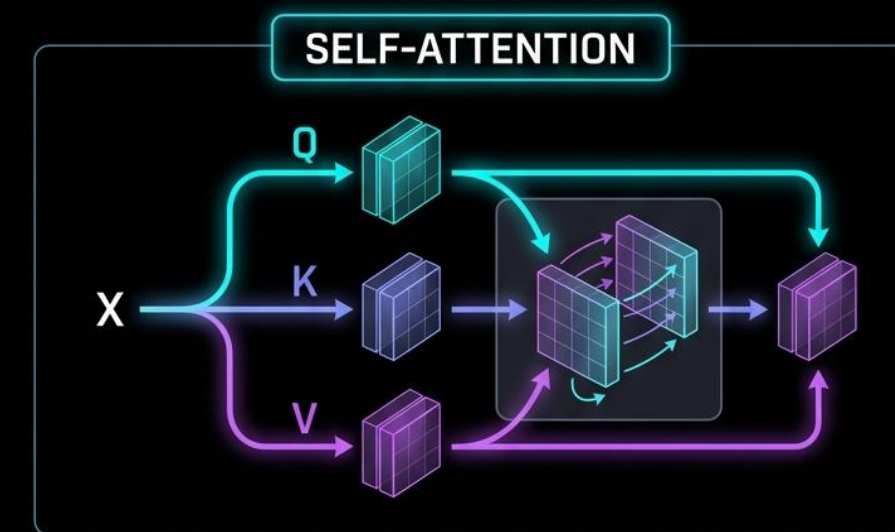
- **Masked Self-attention**

In the Transformer decoder, the self-attention is restricted such that queries at each position can only attend to all key-value pairs up to and including that position.

To enable parallel training, this is typically done by applying a mask function to the un-normalized attention matrix $\hat{A} = \exp(\frac{QK^T}{\sqrt{D_k}})$ where the illegal positions are masked out by setting $\hat{A}_{ij} = -\infty$ if $i < j$. This kind of self-attention is often referred to as autoregressive or causal attention.

- **Cross-attention**

The queries are projected from the outputs of the previous (decoder) layer, whereas the keys and values are projected using the outputs of the encoder.



2. Position-wise FFN.

The position-wise FFN is a fully connected feed-forward module that operates separately and identically on each position

$$\text{FFN}(H') = \text{ReLU}(H'W^1 + b^1)W^2 + b^2$$

where H' is the outputs of previous layer, and $W^1 \in R^{D_m \times D_f}$, $W^2 \in R^{D_f \times D_m}$, $b^1 \in R^{D_f}$, $b^2 \in R^{D_m}$ are trainable parameters. Typically, the intermediate dimension D_f of the FFN is set to be larger than D_m .

3. Residual Connection and Normalization

In order to build a deep model, Transformer employs a residual connection around each module, followed by Layer Normalization. For instance, each Transformer encoder block may be written as

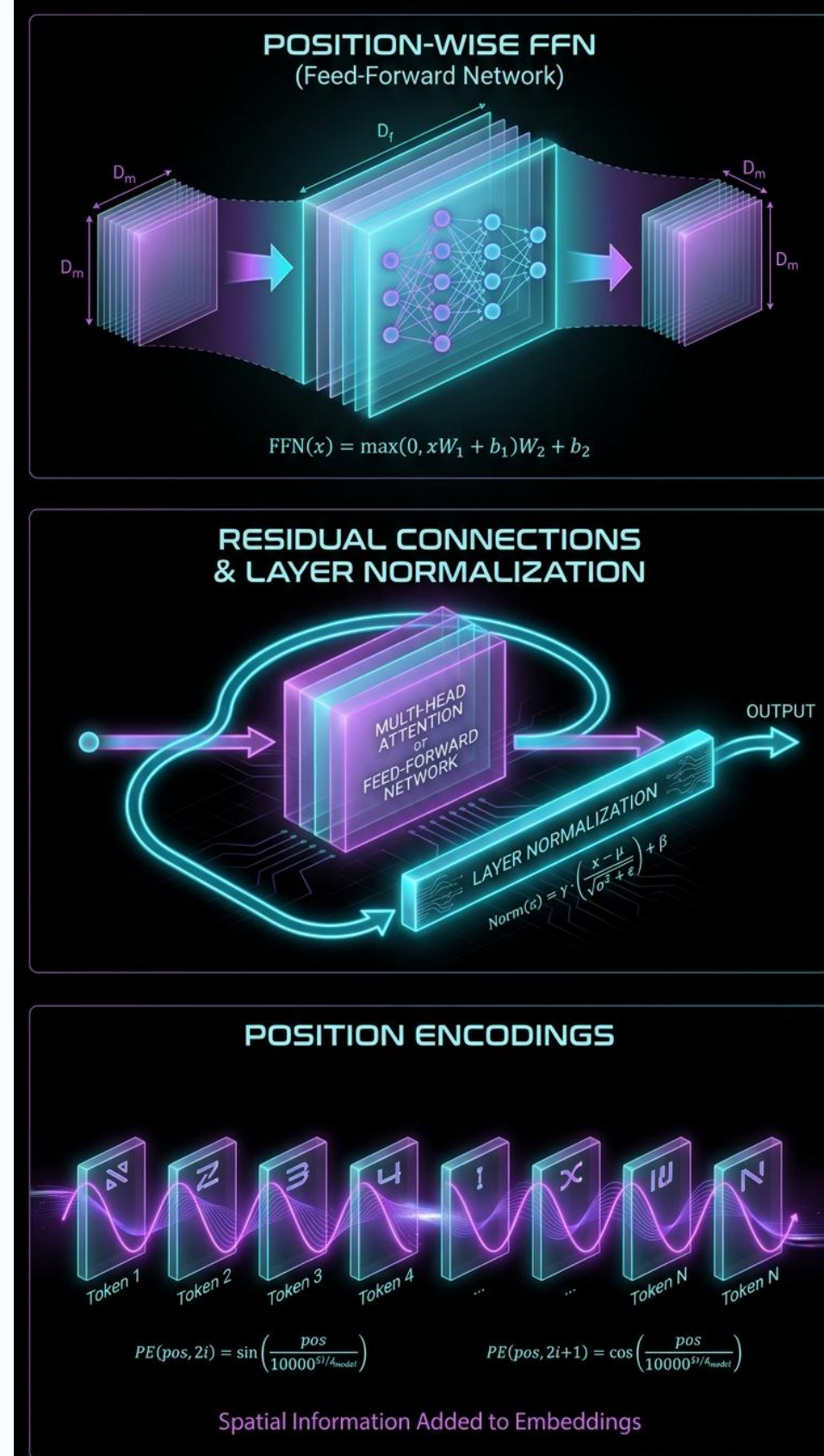
$$H' = \text{LayerNorm}(\text{SelfAttention}(X) + X)$$

$$H = \text{LayerNorm}(\text{FFN}(H') + H')$$

where $\text{SelfAttention}(\cdot)$ denotes self attention module and $\text{LayerNorm}(\cdot)$ denotes the layer normalization operation.

4. Position Encodings.

Since Transformer doesn't introduce recurrence or convolution, it is ignorant of positional information (especially for the encoder). Thus, additional positional representation is needed to model the ordering of tokens.



Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position **separately and identically**.

This consists of two linear transformations with a ReLU activation in between.

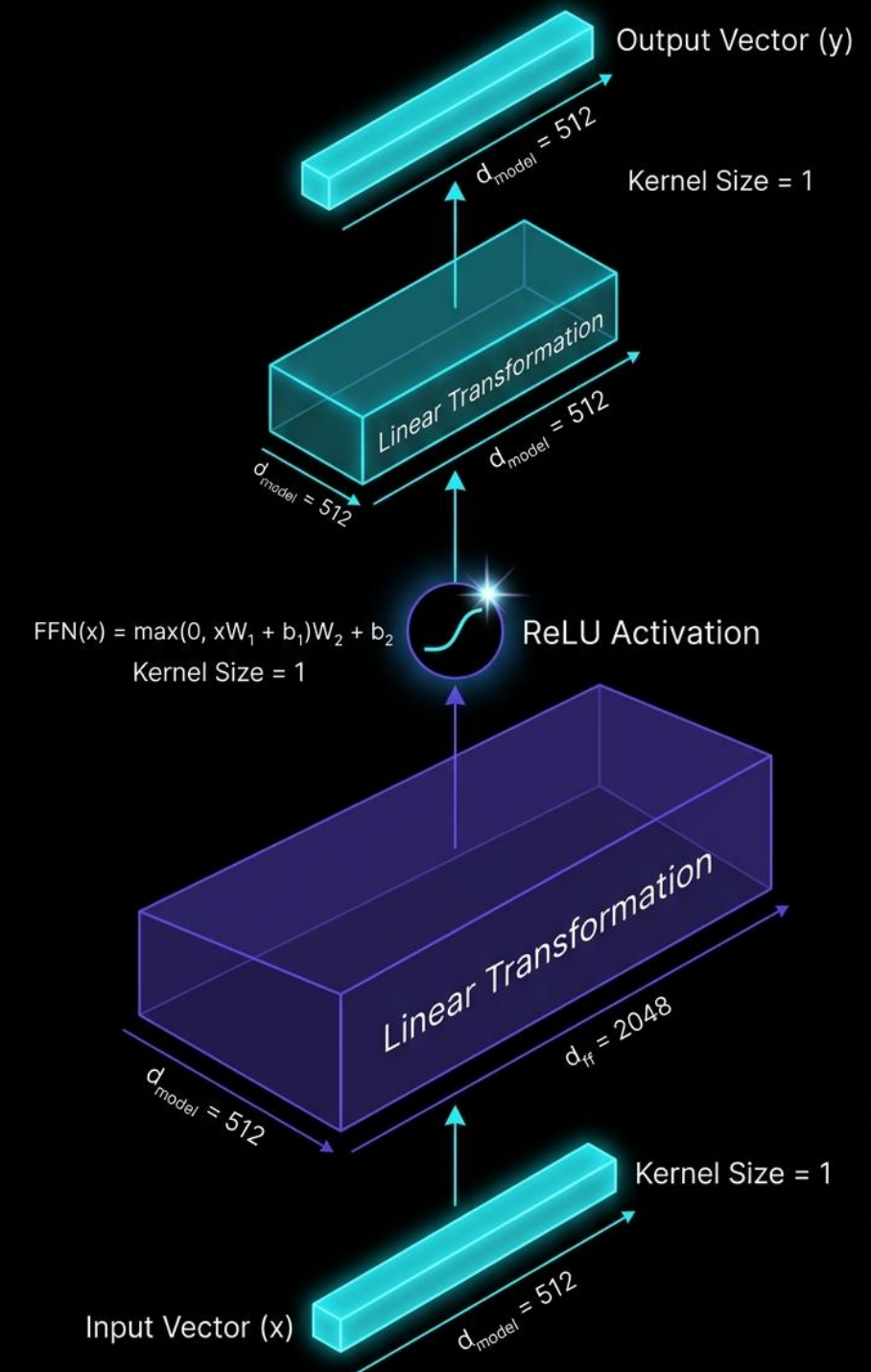
$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer.

Another way of describing this is as two convolutions with kernel size 1.

The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{\text{ff}} = 2048$.

Position-wise Feed-Forward Network (FFN)



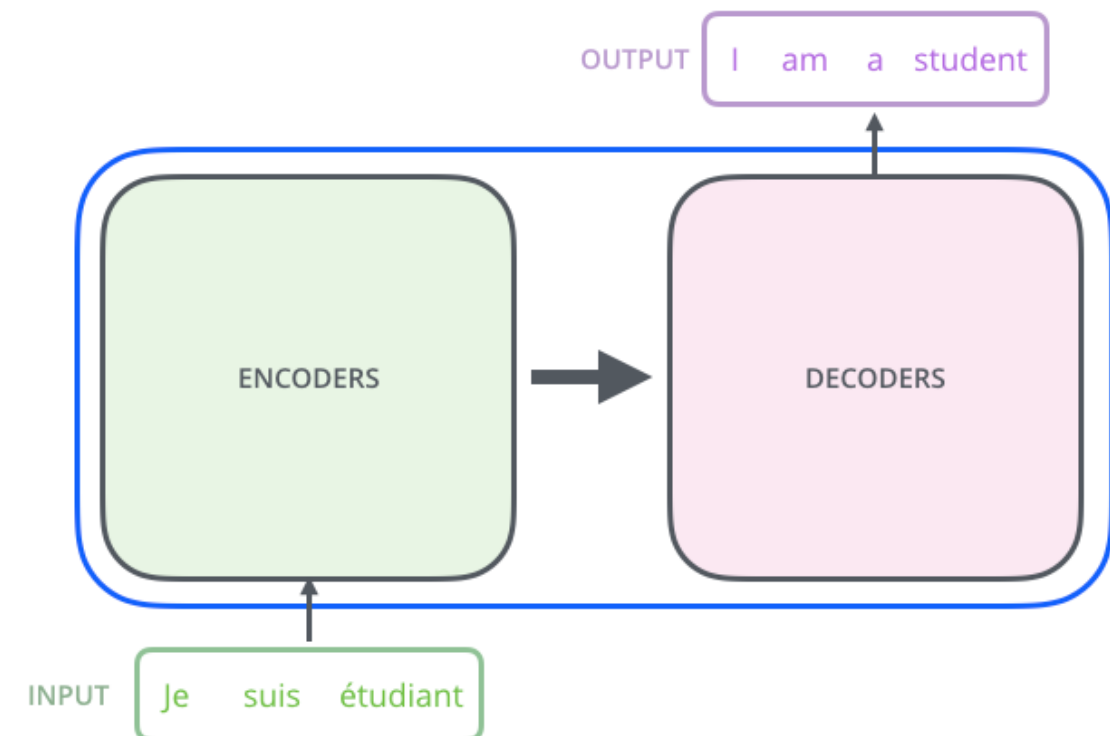
Position-wise Feed-Forward Networks



[Jay Alammam](#)

Visualizing machine learning one concept at a time.

[@JayAlammam](#) on Twitter. [YouTube Channel](#)



Encoder and Decoder with six blocks

The original Transformer architecture consists of **six stacked encoder blocks** and **six stacked decoder blocks**.

Each encoder block contains:

- Multi-Head Self-Attention
- Feed-Forward Network (FFN)
- Residual Connections and Layer Normalization

The decoder contains similar components but additionally includes:

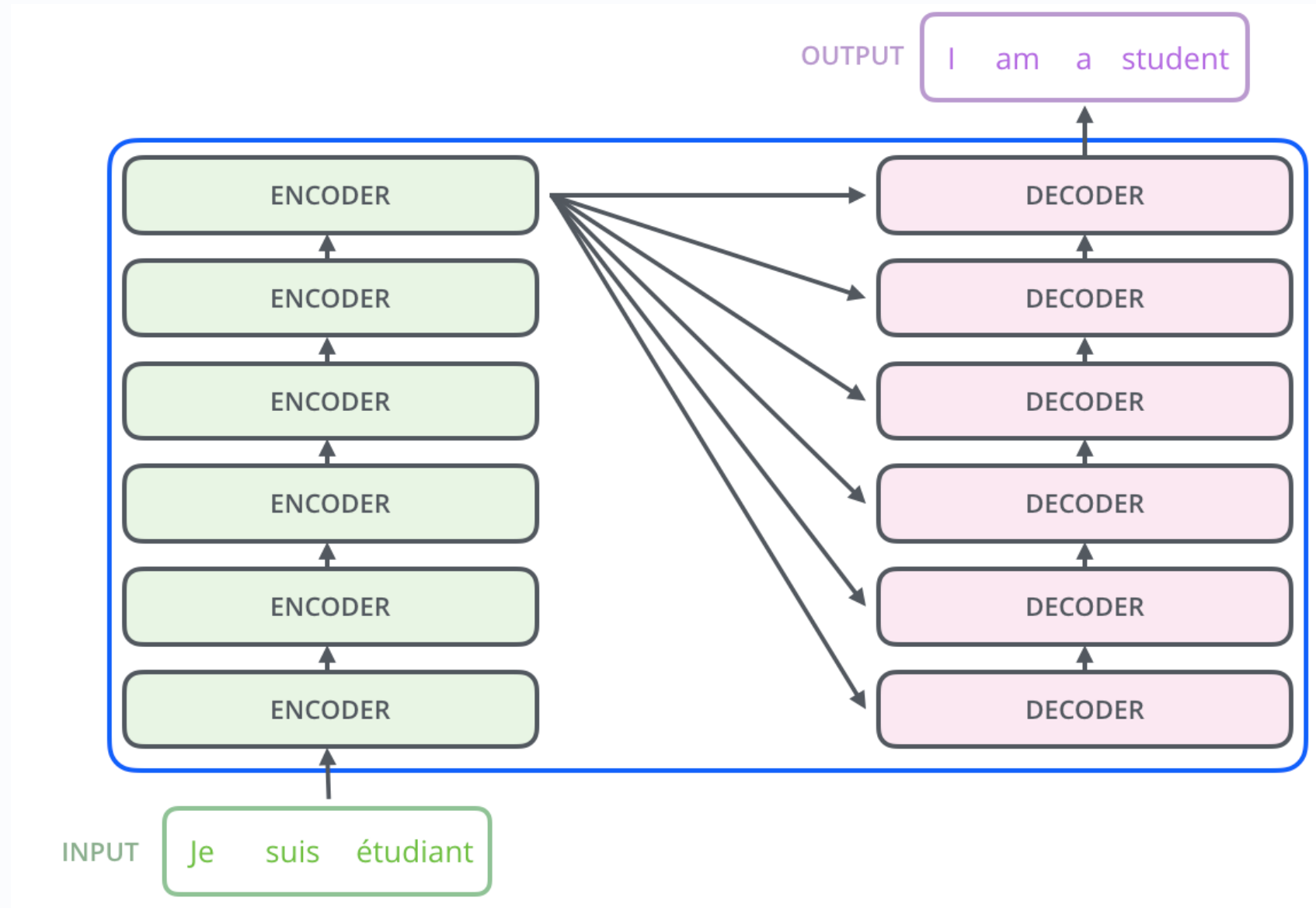
- Masked Self-Attention
- Encoder–Decoder (Cross) Attention

Stacking multiple layers allows the model to progressively learn:

- low-level syntactic relationships,
- semantic context,
- long-range dependencies,
- and high-level representations.

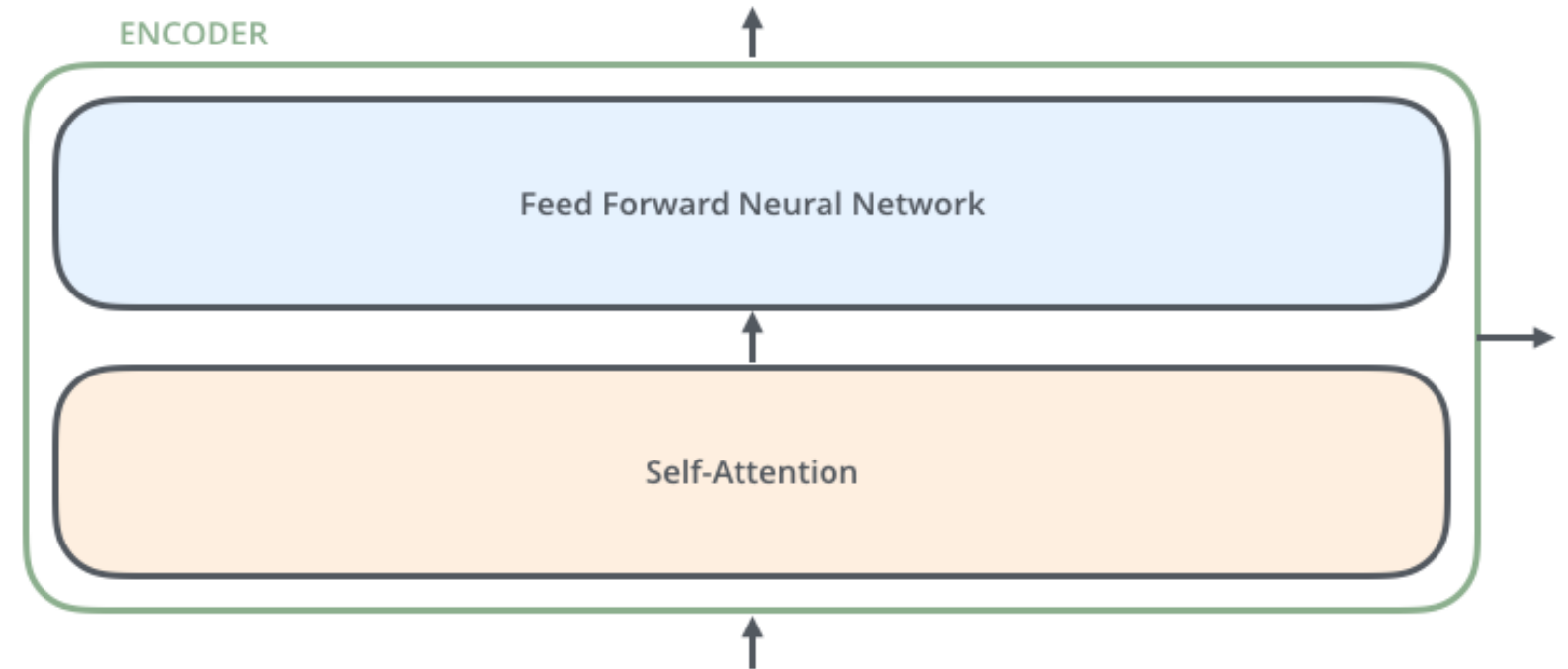
The encoder transforms the input sequence into contextual embeddings, while the decoder uses these representations to generate the output sequence token by token.

In the original Transformer proposed in 2017, **N = 6 layers** provided an effective balance between model capacity and computational efficiency.



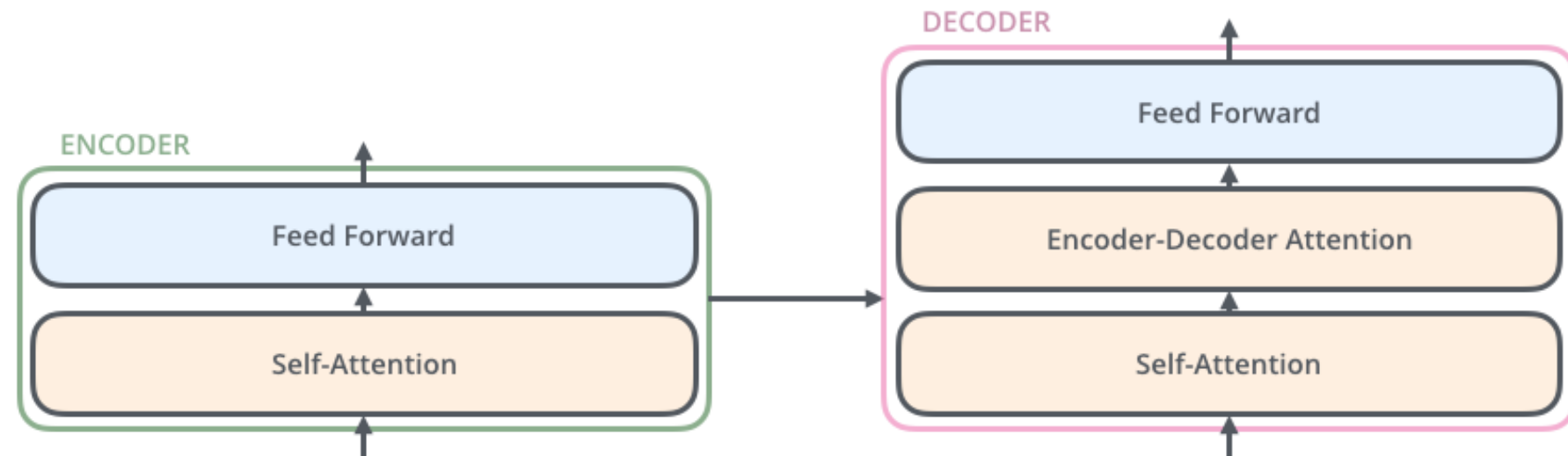
Encoder Block

- The Encoder Block converts the input sequence into context-aware representations.
- **Self-Attention** lets each token attend to all other tokens in the sequence. Using Q, K, V and $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$, it captures long-range dependencies.
- **Multi-Head Attention** learns multiple interaction patterns in parallel and then combines them.
- The output of attention is passed to a **Feed-Forward Network** to apply nonlinear feature transformations.
- In practice, **residual connections + layer normalization** (often with dropout) improve stability and generalization. The block outputs a same-length sequence with richer embeddings for subsequent layers.



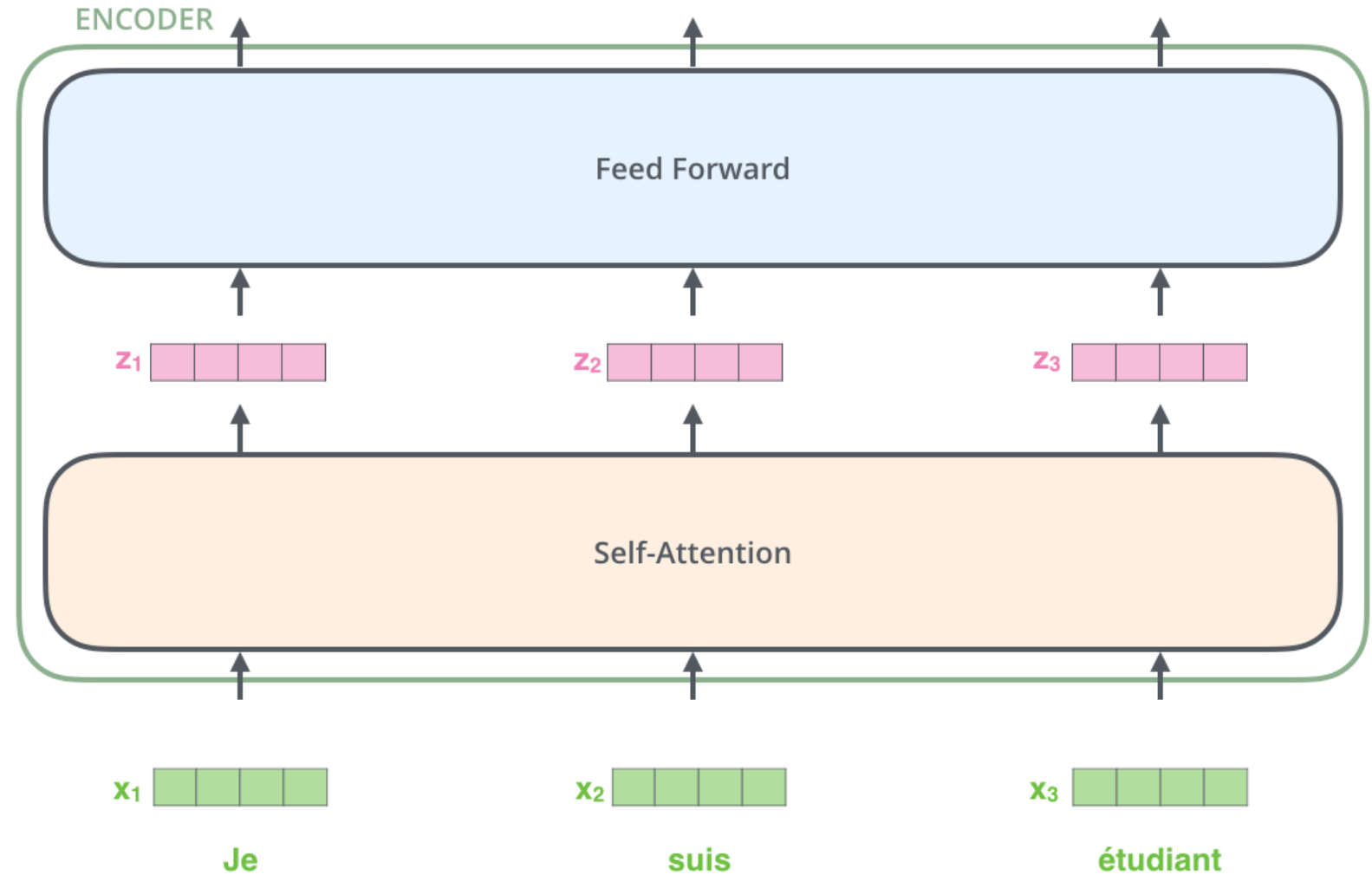
Encoder and Decoder with one block

- This diagram shows a Transformer with one **encoder block** and one **decoder block**.
- The **encoder** applies self-attention + a feed-forward network to produce contextual “memory” for the input sequence.
- The **decoder self-attention** models dependencies among already-generated target tokens.
- During generation it is typically **masked** to prevent attending to future tokens.
- **Encoder–decoder (cross) attention** lets the decoder attend to the encoder outputs to condition on the source sequence.
- A **feed-forward network** refines features after attention in both encoder and decoder.
- Stacking more blocks increases capacity; the final decoder output is mapped to token probabilities.



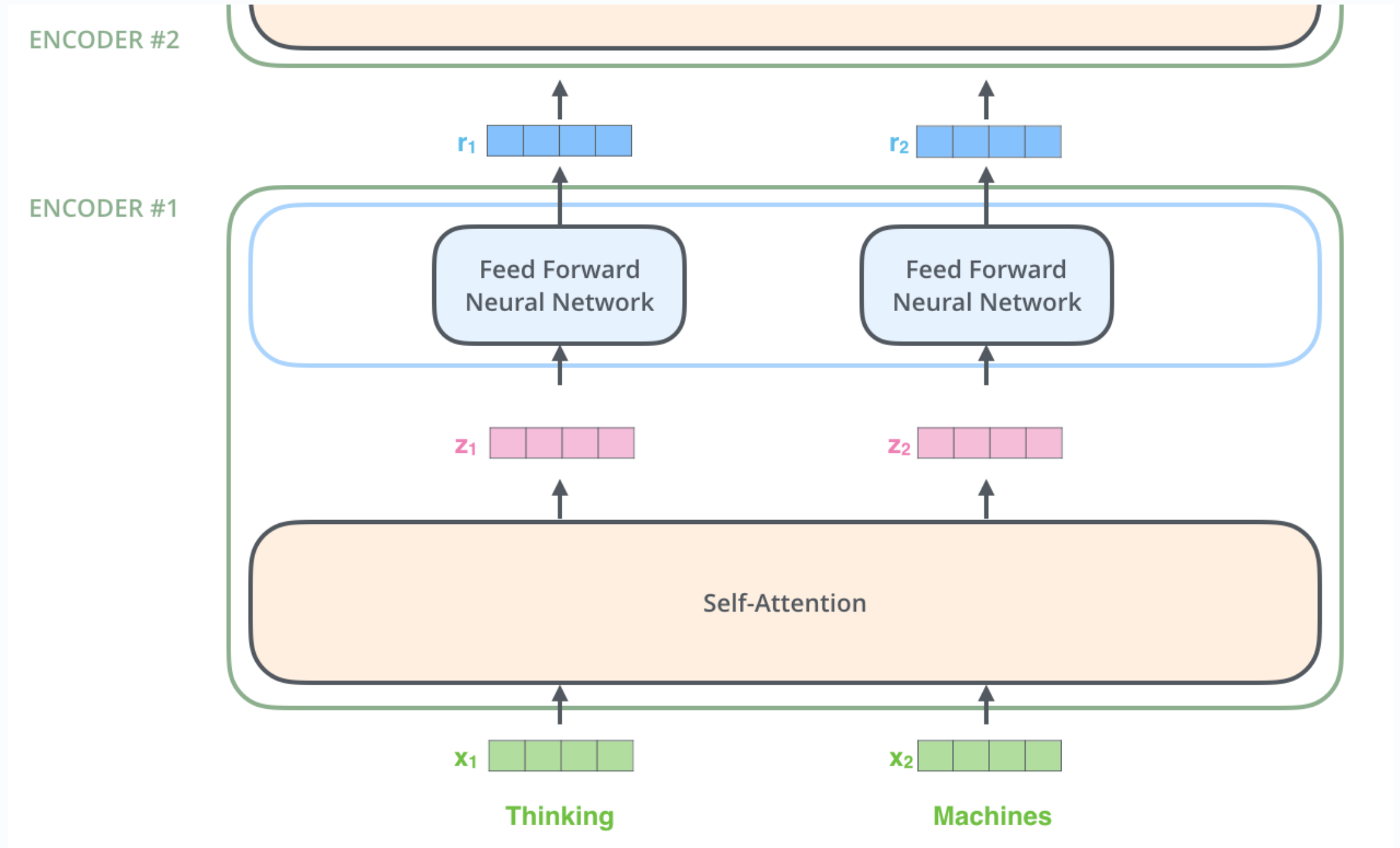
Bringing The Tensors Into The Picture

- Each input word is mapped to an embedding vector $x_i \in \mathbb{R}^{d_{model}}$ (often with positional encoding added).
- The sequence forms a matrix $X \in \mathbb{R}^{n \times d_{model}}$, where n is the number of tokens.
- Self-attention projects X to $Q = XW_Q$, $K = XW_K$, $V = XW_V$.
- Attention mixes information across tokens: $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$.
- The output is a contextual representation per token, denoted z_i (stacked as $Z \in \mathbb{R}^{n \times d_{model}}$). A position-wise feed-forward network transforms each z_i independently to increase expressiveness.
- The encoder outputs a new sequence of vectors that can be passed to the next layer or to a decoder via cross-attention.



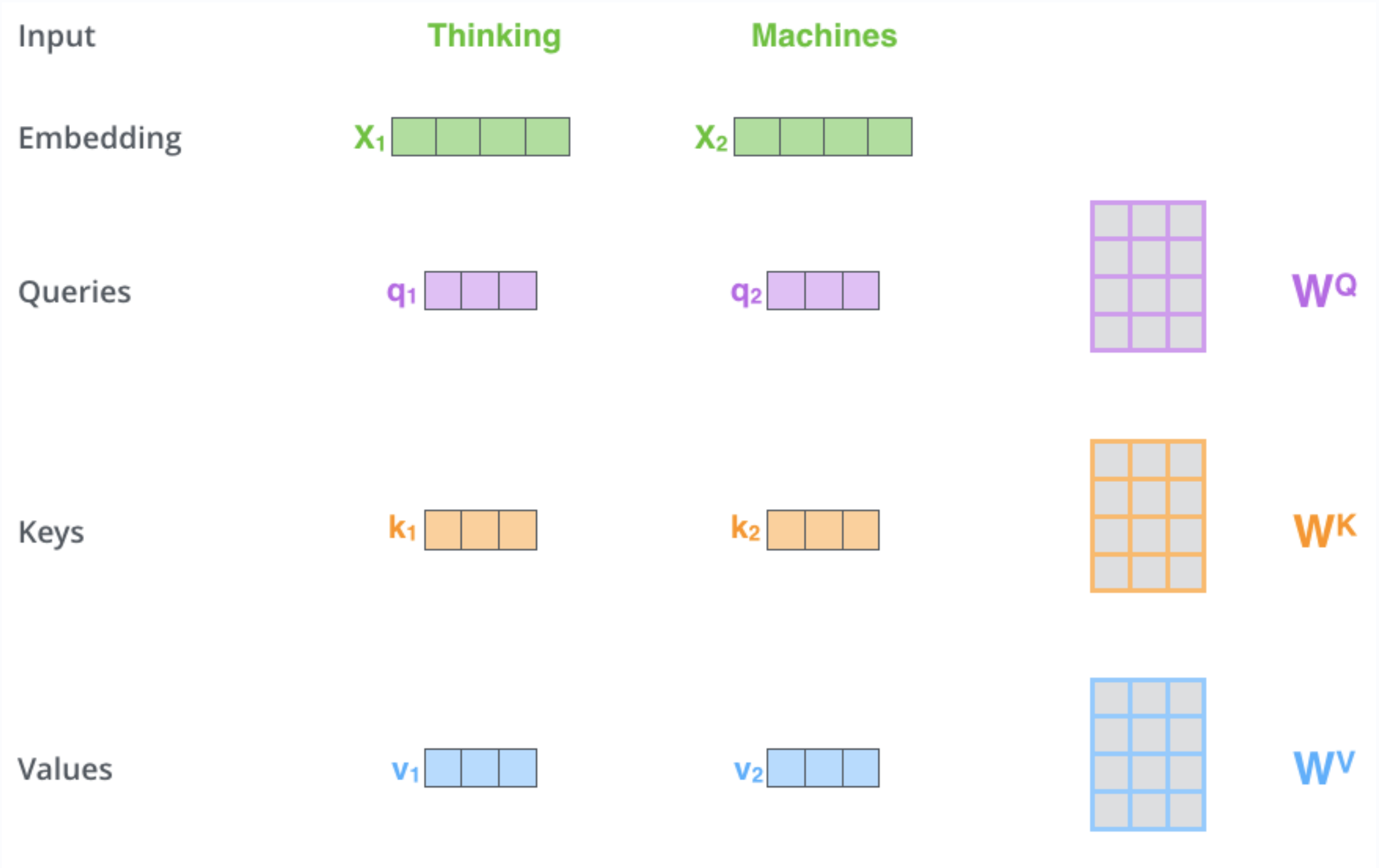
Now We're Encoding!

The word at each position passes through a self-attention process. Then, they each pass through a feed-forward neural network -- the exact same network with each vector flowing through it separately.



Self-Attention in Detail

Multiplying x_1 by the W_Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.



Self-Attention in Detail

Step 1 – Input Embeddings

Suppose the sentence is:

“Thinking Machines”

After the embedding layer, each token is converted into a dense vector representation.

Example embeddings:

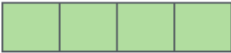
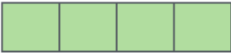
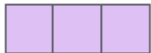
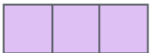
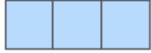
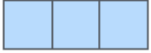
$$x_1 = [2, 3, 5, 4]$$

$$x_2 = [1, 4, 2, 6]$$

Where:

- x_1 represents the word “Thinking”
- x_2 represents the word “Machines”

These embeddings contain semantic information learned during training.

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self-Attention in Detail

Step 2 – Generate Query, Key, and Value Vectors

Each embedding is transformed into:

- a Query vector,
- a Key vector,
- and a Value vector.

Using learned weight matrices:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

Assume the Transformer generates:

For “Thinking”

$$q_1 = [8, 4, 6]$$

$$k_1 = [6, 4, 8]$$

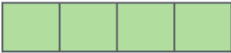
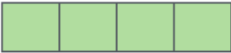
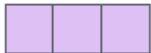
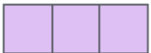
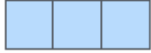
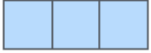
$$v_1 = [5, 3, 7]$$

For “Machines”

$$q_2 = [5, 6, 4]$$

$$k_2 = [4, 4, 8]$$

$$v_2 = [2, 8, 5]$$

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self-Attention in Detail

Step 3 – Compute Attention Scores

The attention score is computed using the dot product between Query and Key vectors.

Attention between “Thinking” and itself:

$$q_1 \cdot k_1 = (8)(6) + (4)(4) + (6)(8) = 48 + 16 + 48 = 112$$

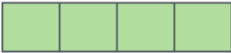
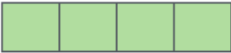
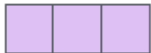
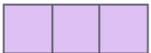
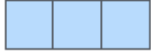
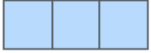
Attention between “Thinking” and “Machines”:

$$q_1 \cdot k_2 = (8)(4) + (4)(4) + (6)(8) = 32 + 16 + 48 = 96$$

Since:

$$112 > 96$$

the word “Thinking” attends more strongly to itself.

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self-Attention in Detail

Step 4 – Scale the Scores

Large dot-product values can make the Softmax function unstable.

Therefore, the scores are scaled by:

$$\sqrt{d_k}$$

where:

- d_k is the Key vector dimension.

Here:

$$d_k = 64$$

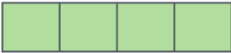
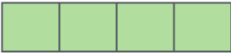
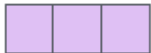
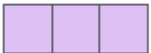
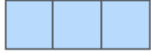
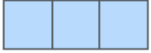
thus:

$$\sqrt{64} = 8$$

Scaled scores become:

$$\frac{112}{8} = 14$$

$$\frac{96}{8} = 12$$

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self-Attention in Detail

Step 5 – Apply Softmax

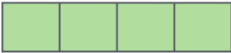
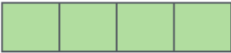
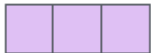
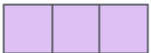
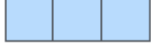
The Softmax function converts scores into probabilities.

$$\text{Softmax}(14,12) = [0.88, 0.12]$$

Interpretation:

- “Thinking” gives 88% attention to itself,
- and 12% attention to “Machines”.

These attention weights are then used to combine the Value vectors and produce the final attention output.

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self-Attention in Detail

Step 1 – Compute Attention Scores

The Transformer first calculates similarity scores between Query and Key vectors using the dot product operation.

For the token “Thinking”:

$$q_1 \cdot k_1 = 112$$

$$q_1 \cdot k_2 = 96$$

These scores indicate how strongly the word “Thinking” should attend to:

- itself,
- and the word “Machines”.

A larger score means a stronger semantic relationship.

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax

X

Value

Sum

Thinking

x_1

q_1

k_1

v_1

$$q_1 \cdot k_1 = 112$$

14

0.88

v_1

z_1

Machines

x_2

q_2

k_2

v_2

$$q_1 \cdot k_2 = 96$$

12

0.12

v_2

z_2

Self-Attention in Detail

Step 2 – Scale the Scores

To stabilize training and prevent extremely large values, the scores are divided by:

$$\sqrt{d_k}$$

where:

- d_k is the dimension of the Key vectors.

Assume:

$$d_k = 64$$

then:

$$\sqrt{64} = 8$$

Scaled scores become:

$$\frac{112}{8} = 14$$

$$\frac{96}{8} = 12$$

Scaling prevents the Softmax function from becoming overly sensitive to large values.

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax

X

Value

Sum

Thinking

x_1

q_1

k_1

v_1

$$q_1 \cdot k_1 = 112$$

14

0.88

v_1

z_1

Machines

x_2

q_2

k_2

v_2

$$q_1 \cdot k_2 = 96$$

12

0.12

v_2

z_2

Self-Attention in Detail

Step 3 – Apply Softmax

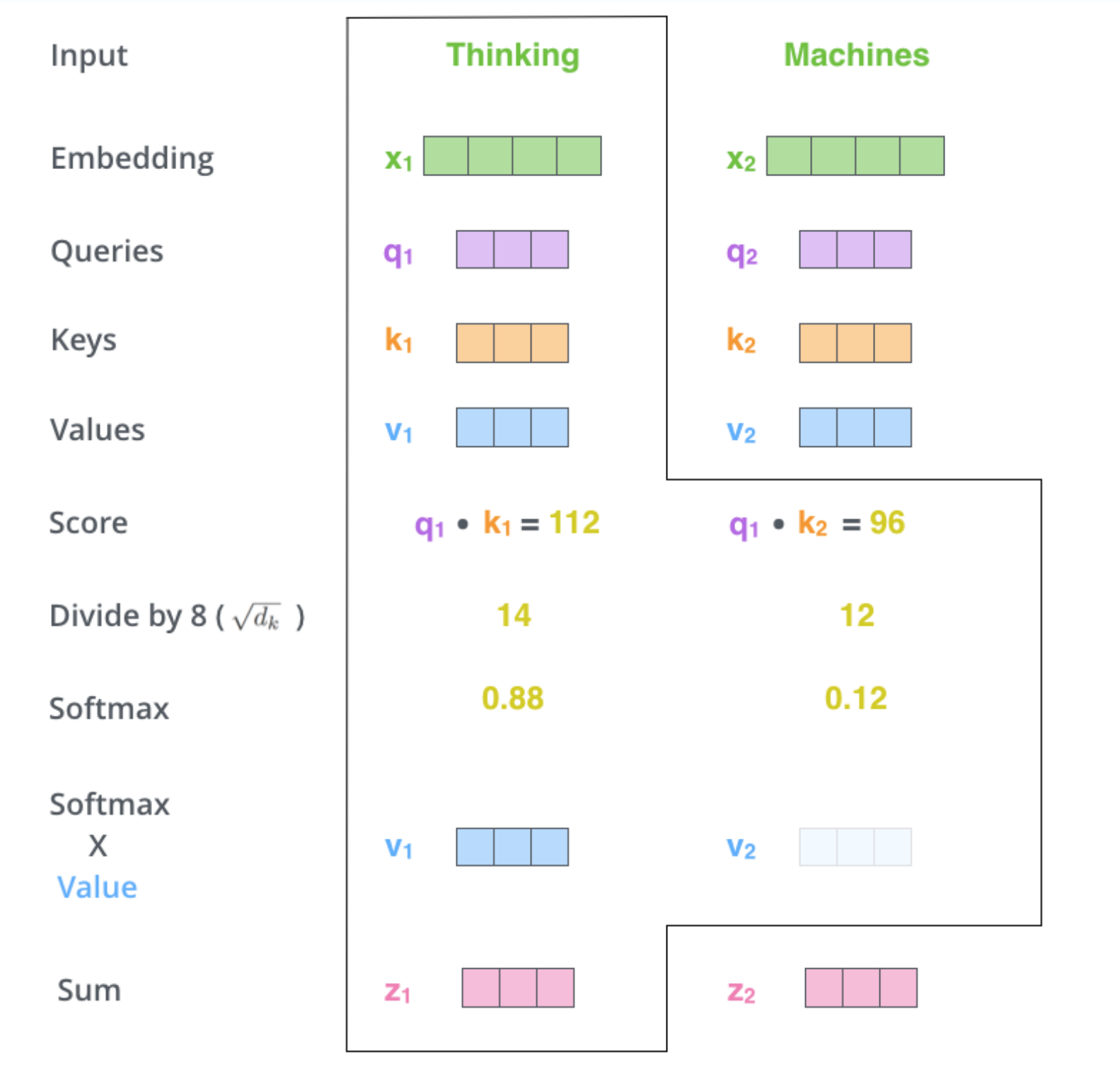
The Softmax function converts the scaled scores into probabilities:

$$\text{Softmax}(14, 12) = [0.88, 0.12]$$

Interpretation:

- 88% attention is assigned to “Thinking”
- 12% attention is assigned to “Machines”

These probabilities are called **Attention Weights**.



Self-Attention in Detail

Step 4 – Weight the Value Vectors

The attention weights are multiplied by the corresponding Value vectors.

Assume:

$$v_1 = [5, 3, 7]$$

$$v_2 = [2, 8, 5]$$

Weighted Value vectors become:

For v_1 :

$$0.88 \times v_1 = [4.40, 2.64, 6.16]$$

For v_2 :

$$0.12 \times v_2 = [0.24, 0.96, 0.60]$$

The larger attention weight causes v_1 to contribute more strongly to the final output.

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax

X

Value

Sum

Thinking

x_1

q_1

k_1

v_1

$$q_1 \cdot k_1 = 112$$

14

0.88

v_1

z_1

Machines

x_2

q_2

k_2

v_2

$$q_1 \cdot k_2 = 96$$

12

0.12

v_2

z_2

Self-Attention in Detail

Step 5 – Sum the Weighted Values

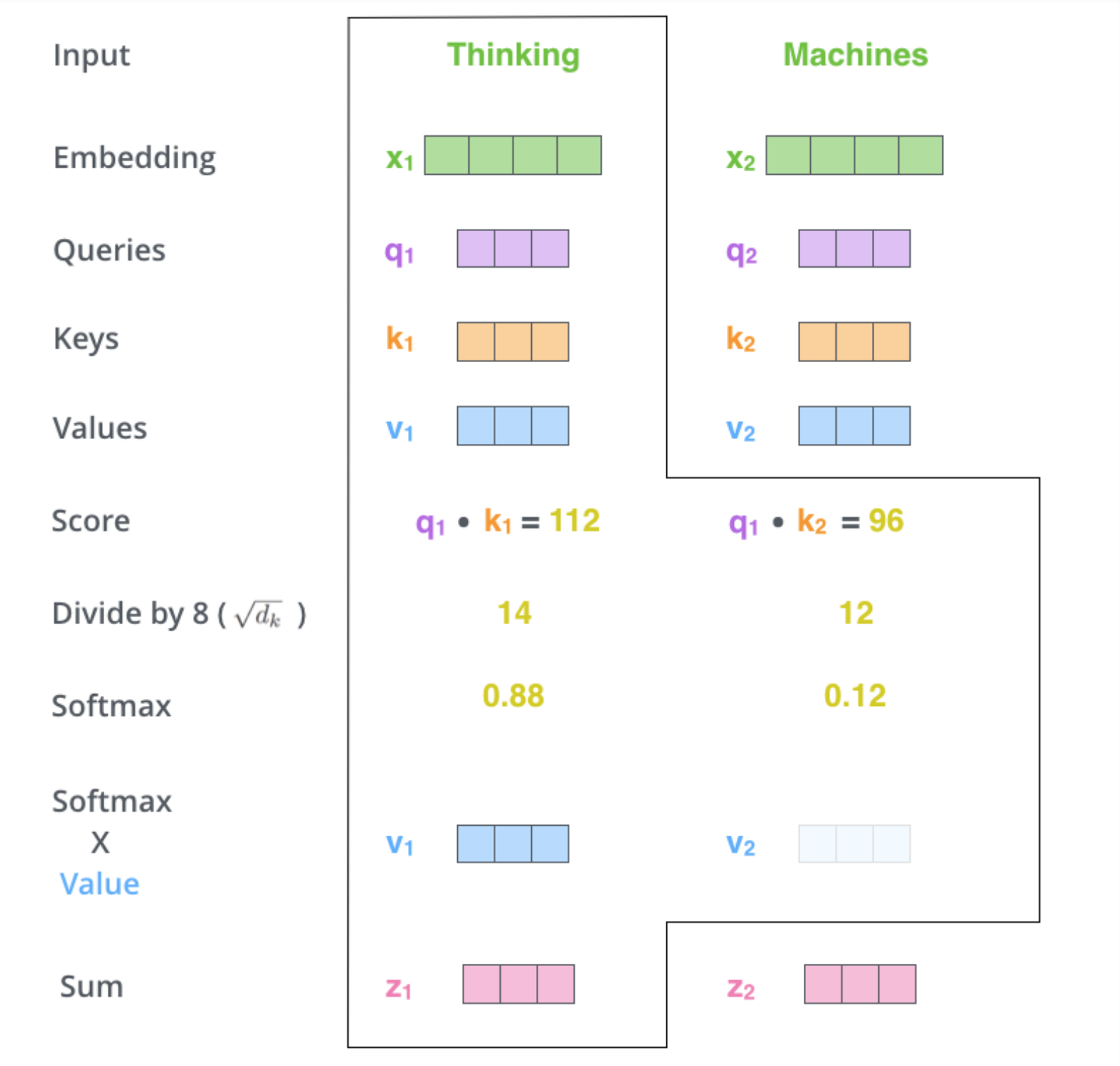
The final attention output is obtained by summing the weighted Value vectors.

$$\begin{aligned} z_1 &= [4.40, 2.64, 6.16] + [0.24, 0.96, 0.60] \\ &= [4.64, 3.60, 6.76] \end{aligned}$$

This final vector:

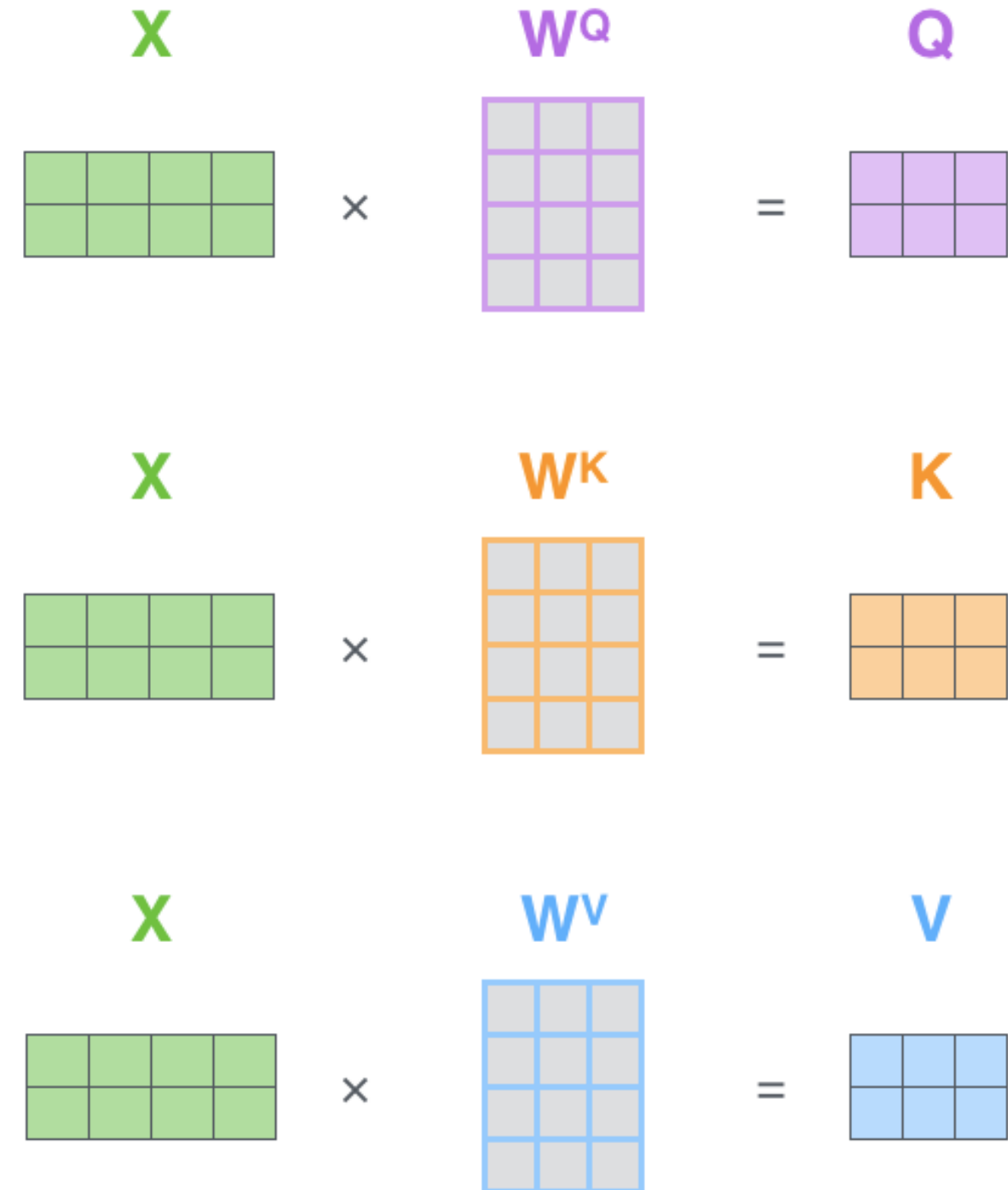
$$z_1 = [4.64, 3.60, 6.76]$$

contains contextual information from all relevant tokens in the sentence.



Matrix Calculation of Self-Attention

Every row in the X matrix corresponds to a word in the input sentence. We again see the difference in size of the embedding vector (512, or 4 boxes in the figure), and the q/k/v vectors (64, or 3 boxes in the figure)



The self-attention calculation in matrix form

Matrix Form of Self-Attention

Instead of computing attention separately for each token, Transformers perform all attention operations using matrix multiplication.

The input embedding matrix $\mathbf{X}$ is projected into three matrices:

- Query matrix ($\mathbf{Q}$)
- Key matrix ($\mathbf{K}$)
- Value matrix ($\mathbf{V}$)

The attention scores between all tokens are computed simultaneously as:

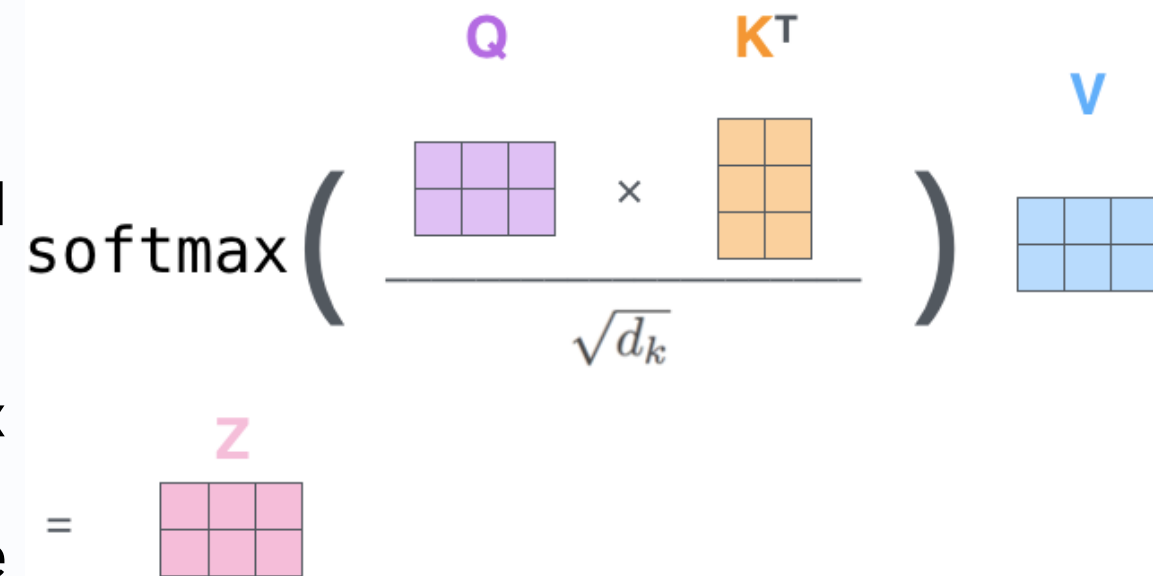
$$\mathbf{Q}\mathbf{K}^T$$

The scores are then scaled and normalized using the Softmax function to obtain attention weights.

Finally, the attention output is computed by multiplying the attention weights with the value matrix:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

This matrix formulation enables highly efficient parallel computation on GPUs and is one of the key reasons Transformers train much faster than RNN-based models.



The diagram illustrates the self-attention calculation in matrix form. It shows the Query matrix ($\mathbf{Q}$, purple 2x3 grid), Key matrix ($\mathbf{K}^T$, orange 3x2 grid), and Value matrix ($\mathbf{V}$, blue 2x3 grid). The formula is: $\text{softmax}\left(\frac{\mathbf{Q} \times \mathbf{K}^T}{\sqrt{d_k}}\right) \times \mathbf{V}$. The result is the attention weights matrix ($\mathbf{Z}$, pink 2x3 grid).

The Beast With Many Heads

Multi-Head Attention

Instead of learning a single attention pattern, Transformers use multiple attention heads operating in parallel.

Each head learns its own:

- Query projection
- Key projection
- Value projection

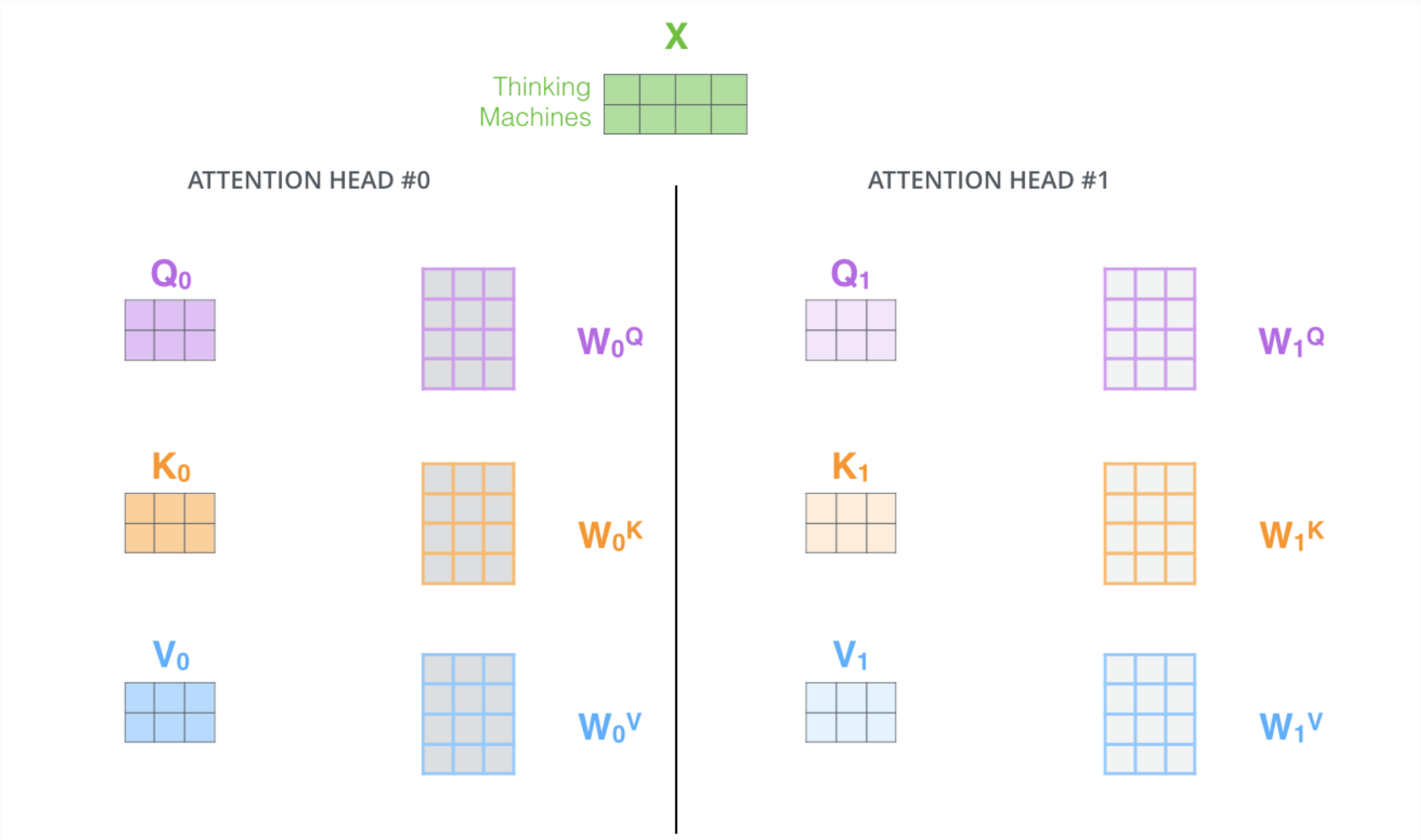
As a result, different heads can focus on different relationships within the sequence.

For example:

- one head may learn grammatical dependencies,
- another may focus on long-range context,
- another may capture semantic relationships.

The outputs of all heads are concatenated and projected into a single representation.

This mechanism significantly increases the expressive power of the Transformer.



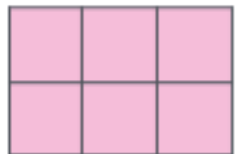
The Beast With Many Heads



Calculating attention separately in
eight different attention heads

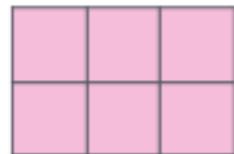
ATTENTION
HEAD #0

Z_0



ATTENTION
HEAD #1

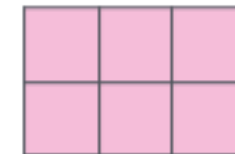
Z_1



...

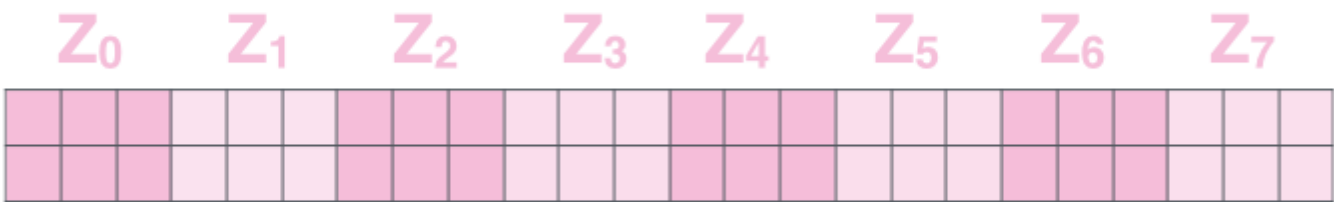
ATTENTION
HEAD #7

Z_7

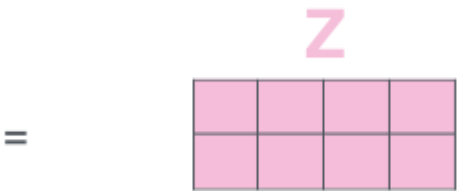


The Beast With Many Heads

1) Concatenate all the attention heads

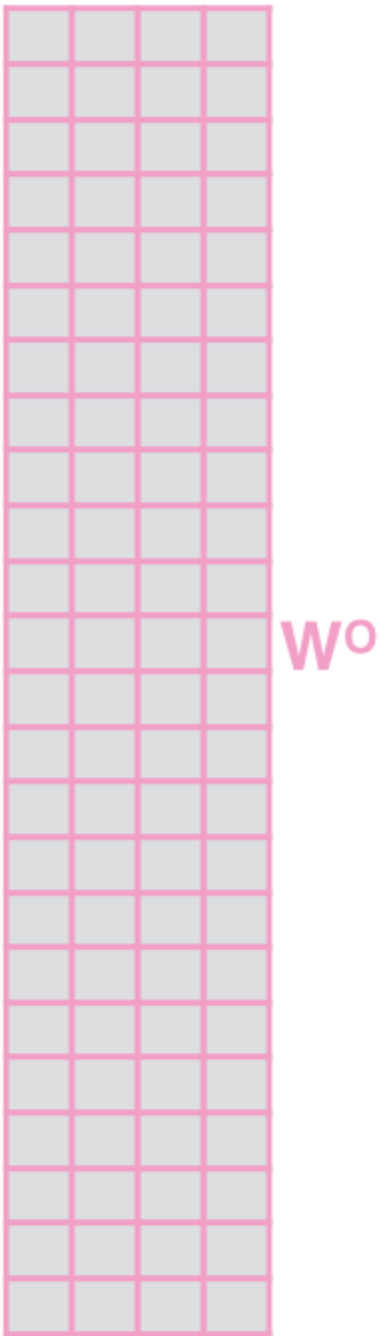


3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$

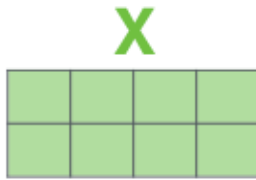


The Beast With Many Heads

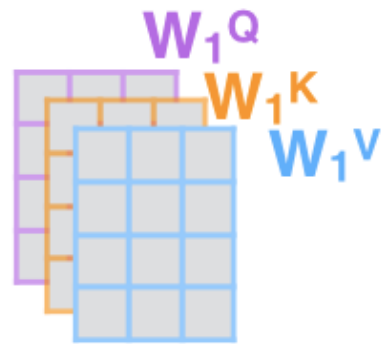
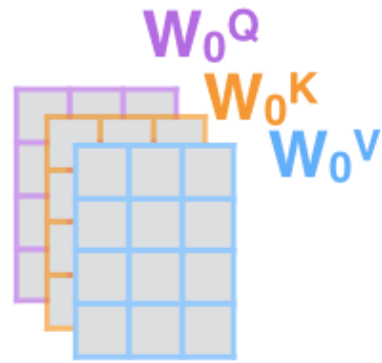
1) This is our input sentence*

Thinking
Machines

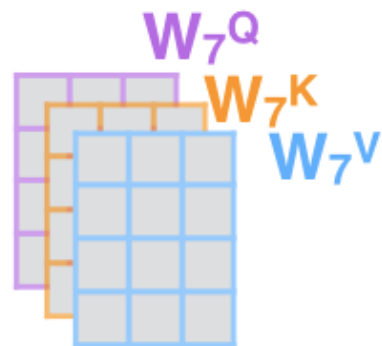
2) We embed each word*



3) Split into 8 heads.
We multiply X or R with weight matrices



...



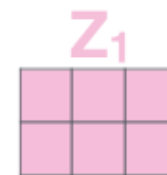
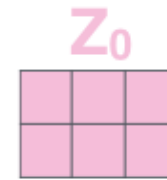
4) Calculate attention using the resulting $Q/K/V$ matrices



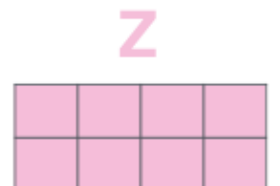
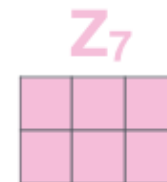
...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



...

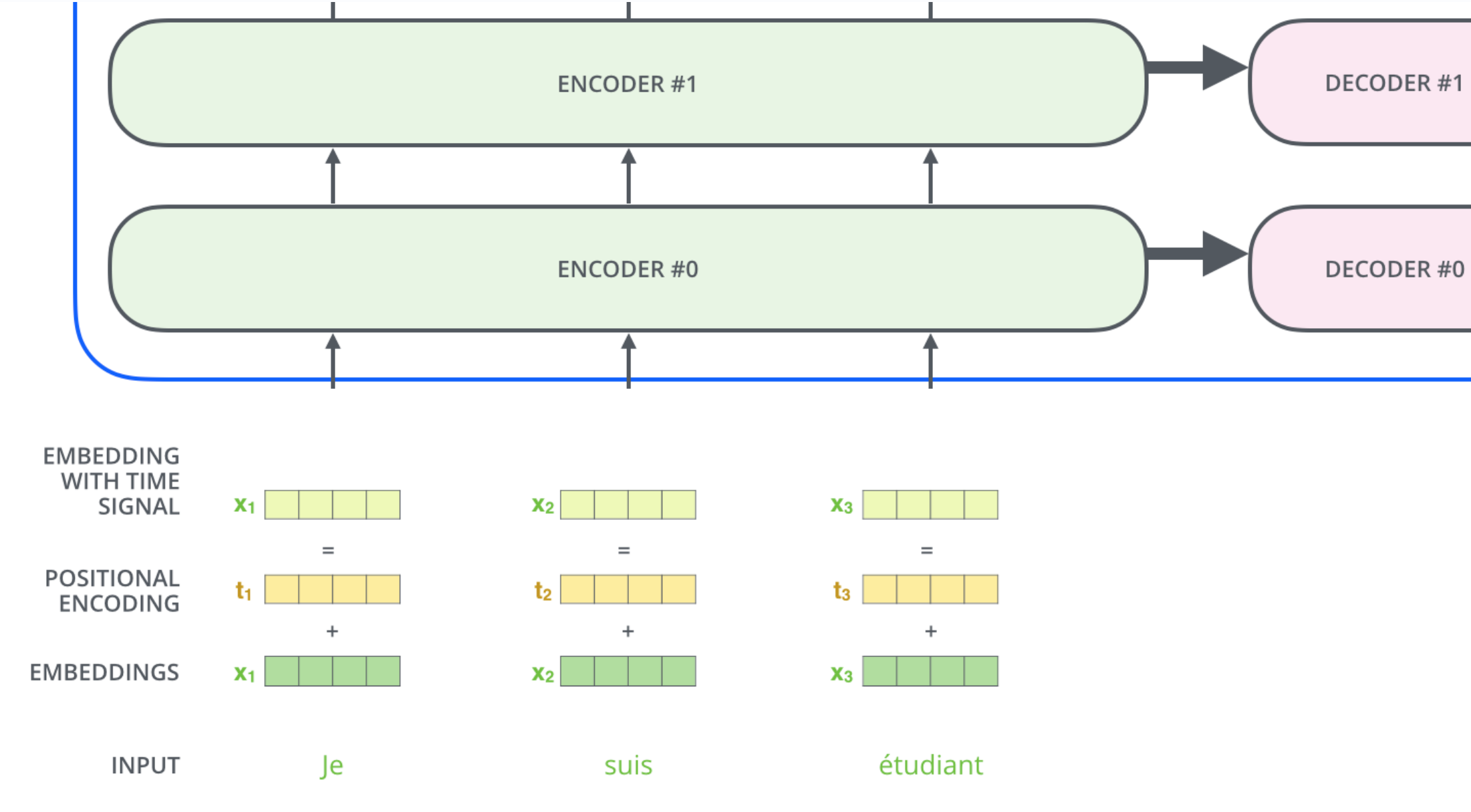


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

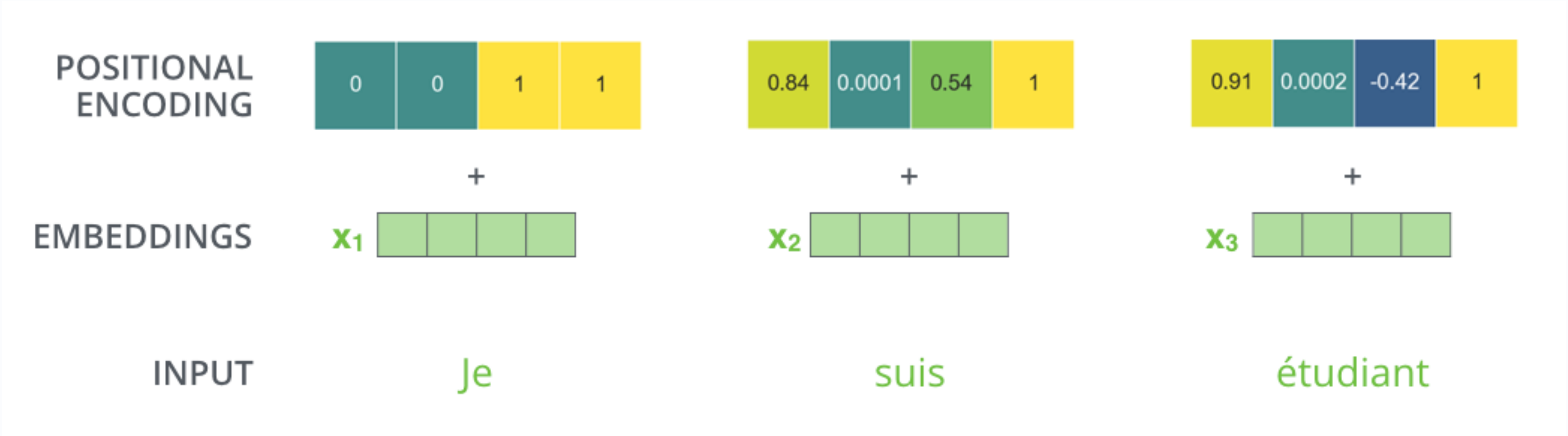


Representing The Order of The Sequence Using Positional Encoding

To give the model a sense of the order of the words, we add positional encoding vectors -- the values of which follow a specific pattern.



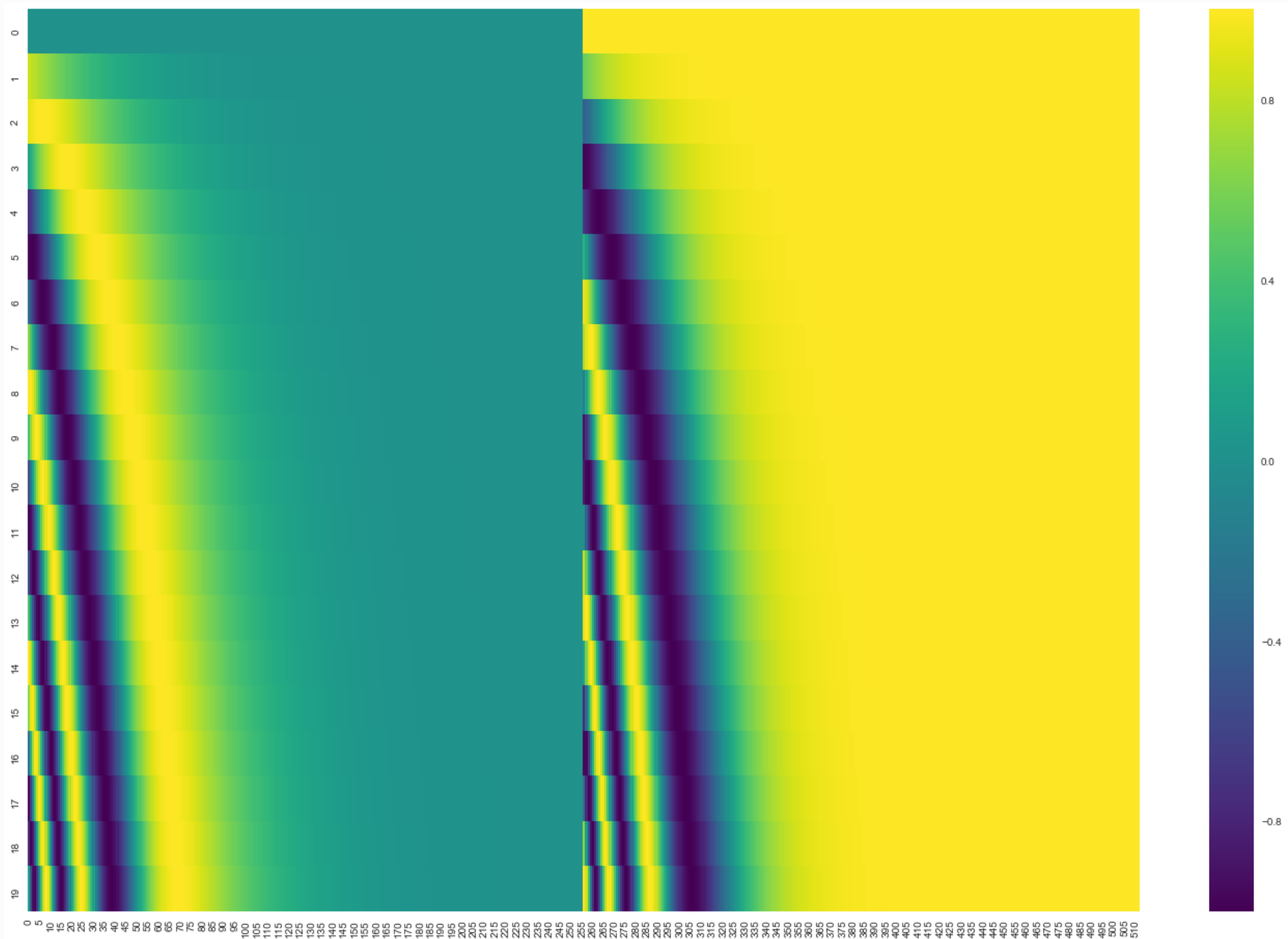
Representing The Order of The Sequence Using Positional Encoding



A real example of positional encoding with a toy embedding size of 4

Representing The Order of The Sequence Using Positional Encoding

A real example of positional encoding for 20 words (rows) with an embedding size of 512 (columns). You can see that it appears split in half down the center. That's because the values of the left half are generated by one function (which uses sine), and the right half is generated by another function (which uses cosine). They're then concatenated to form each of the positional encoding vectors.



The Residuals

Residual Connections in Transformers

Deep neural networks often suffer from optimization difficulties as the number of layers increases.

To address this issue, Transformers employ **Residual Connections** (Skip Connections).

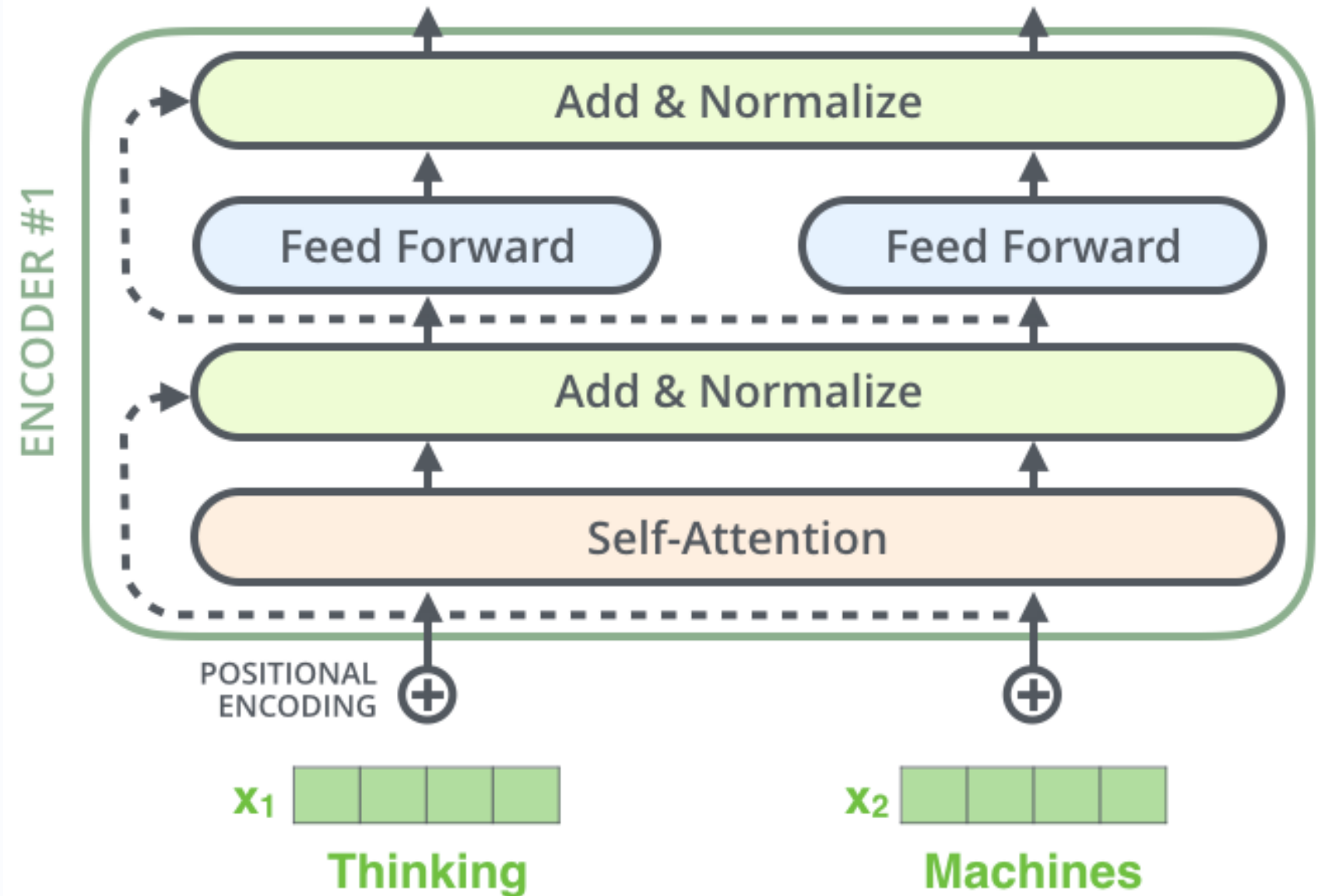
Instead of learning a completely new representation, each sub-layer learns a refinement of its input:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Residual connections provide:

- improved gradient flow,
- faster convergence,
- more stable training,
- better performance in deep architectures.

Without residual connections, training large Transformer models would be significantly more difficult.



The Residuals

Layer Normalization

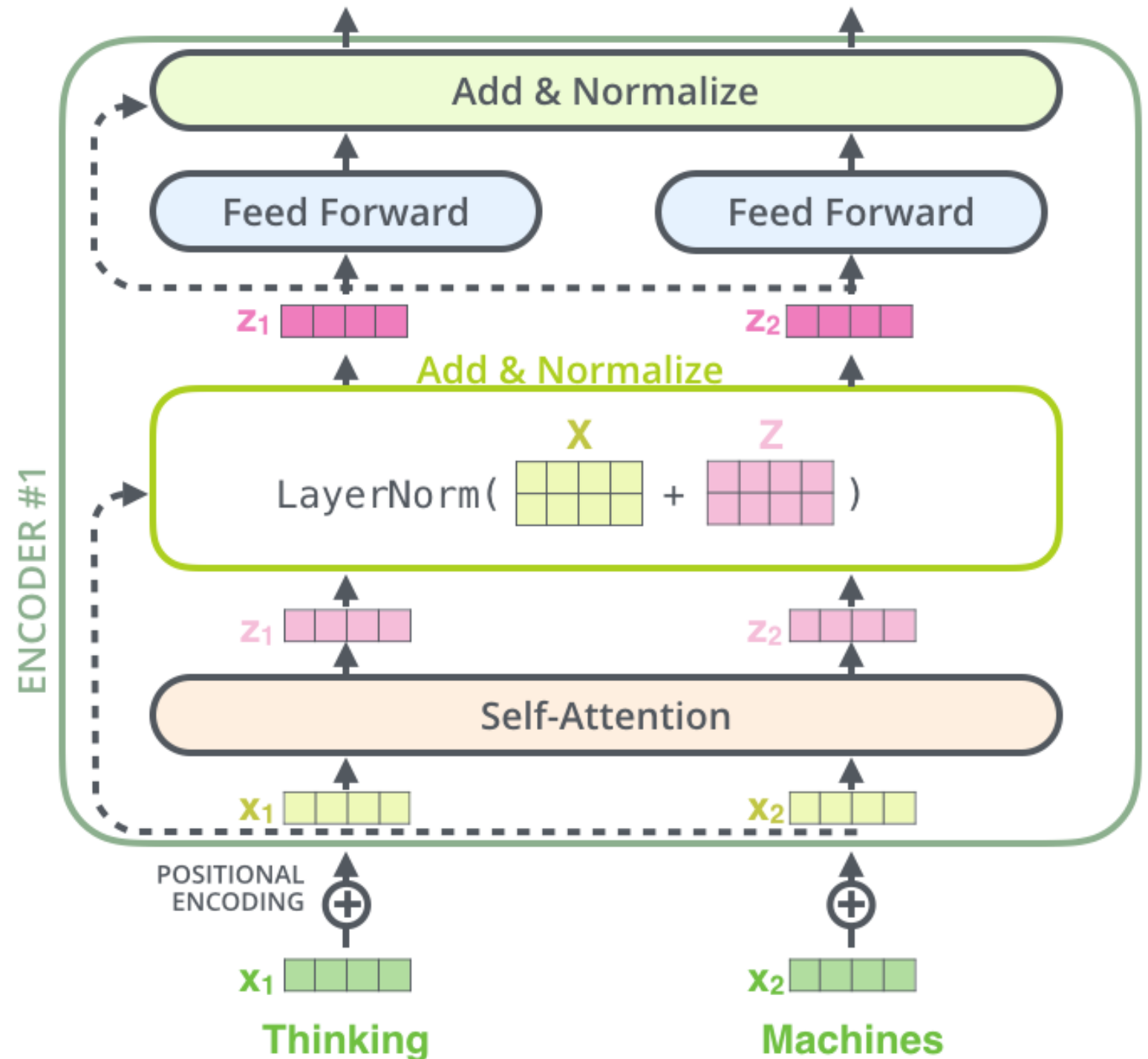
After each residual connection, Transformers apply **Layer Normalization**.

Layer normalization standardizes activations across features for each token independently.

Benefits include:

- reduced internal covariate shift,
- improved numerical stability,
- faster training,
- better convergence behavior.

The combination of residual connections and layer normalization is one of the fundamental design choices that enables very deep Transformer architectures.



The Residuals

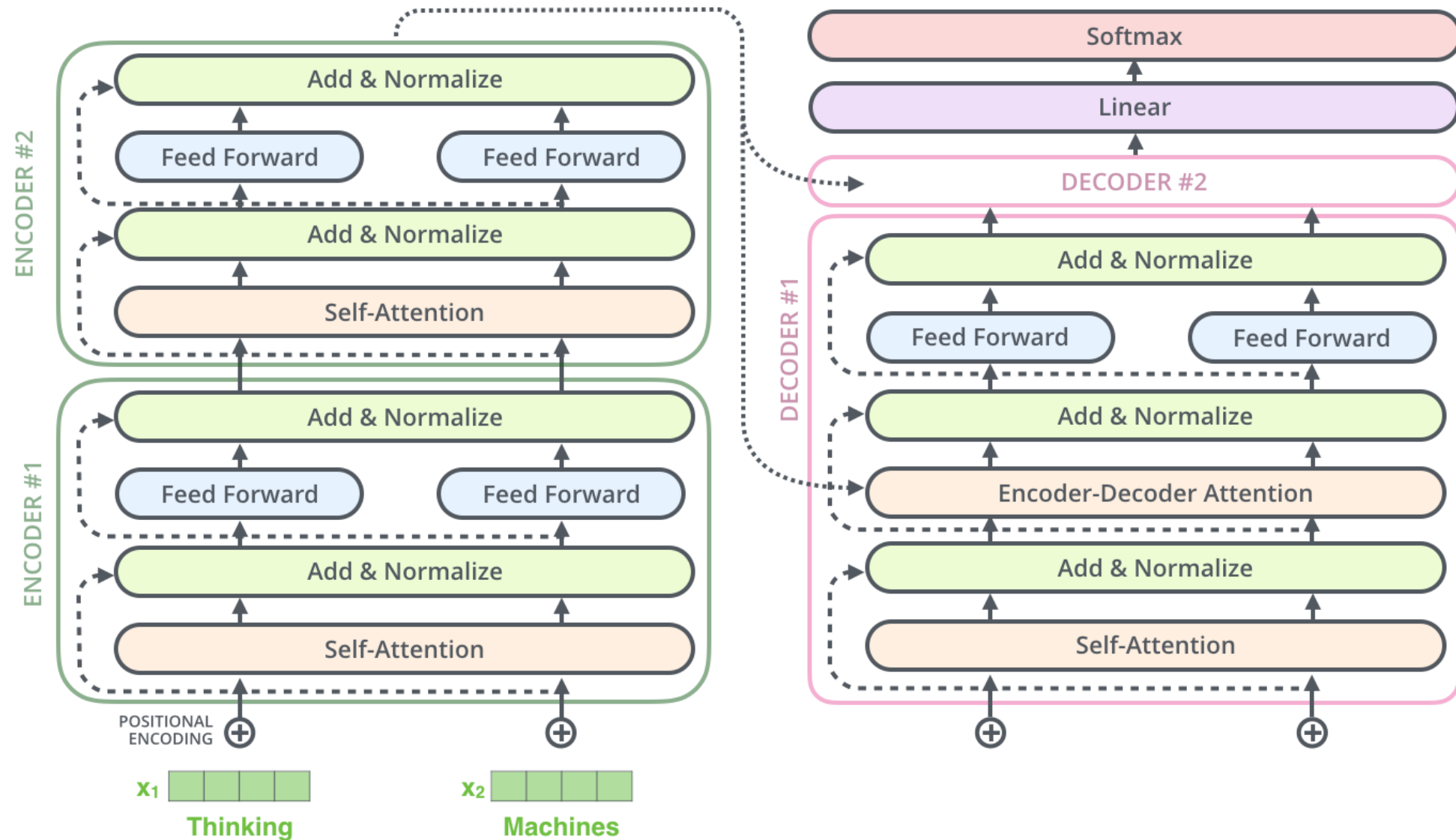
Why Residual Connections Matter

As information flows through many Transformer layers, important features could gradually weaken or disappear. Residual paths create direct routes through the network, allowing earlier information to remain accessible in deeper layers.

This helps the model:

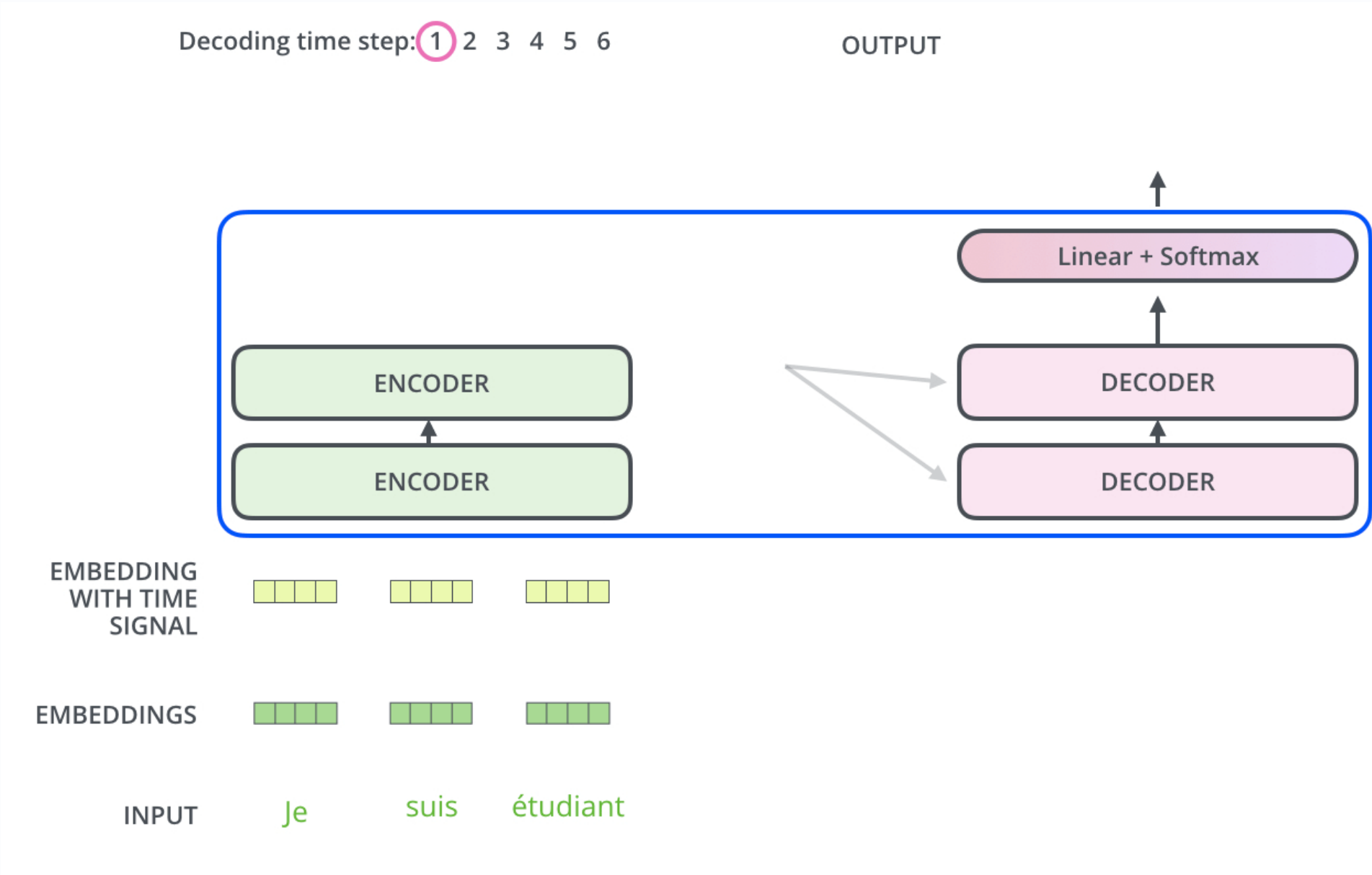
- preserve semantic information,
- learn complex representations,
- avoid vanishing gradients,
- maintain stable performance as depth increases.

Modern architectures such as GPT, BERT, ViT, and LLaMA all rely heavily on residual connections.



The Residuals

After finishing the encoding phase, we begin the decoding phase. Each step in the decoding phase outputs an element from the output sequence (the English translation sentence in this case).



The Residuals

Beginning the Decoding Process

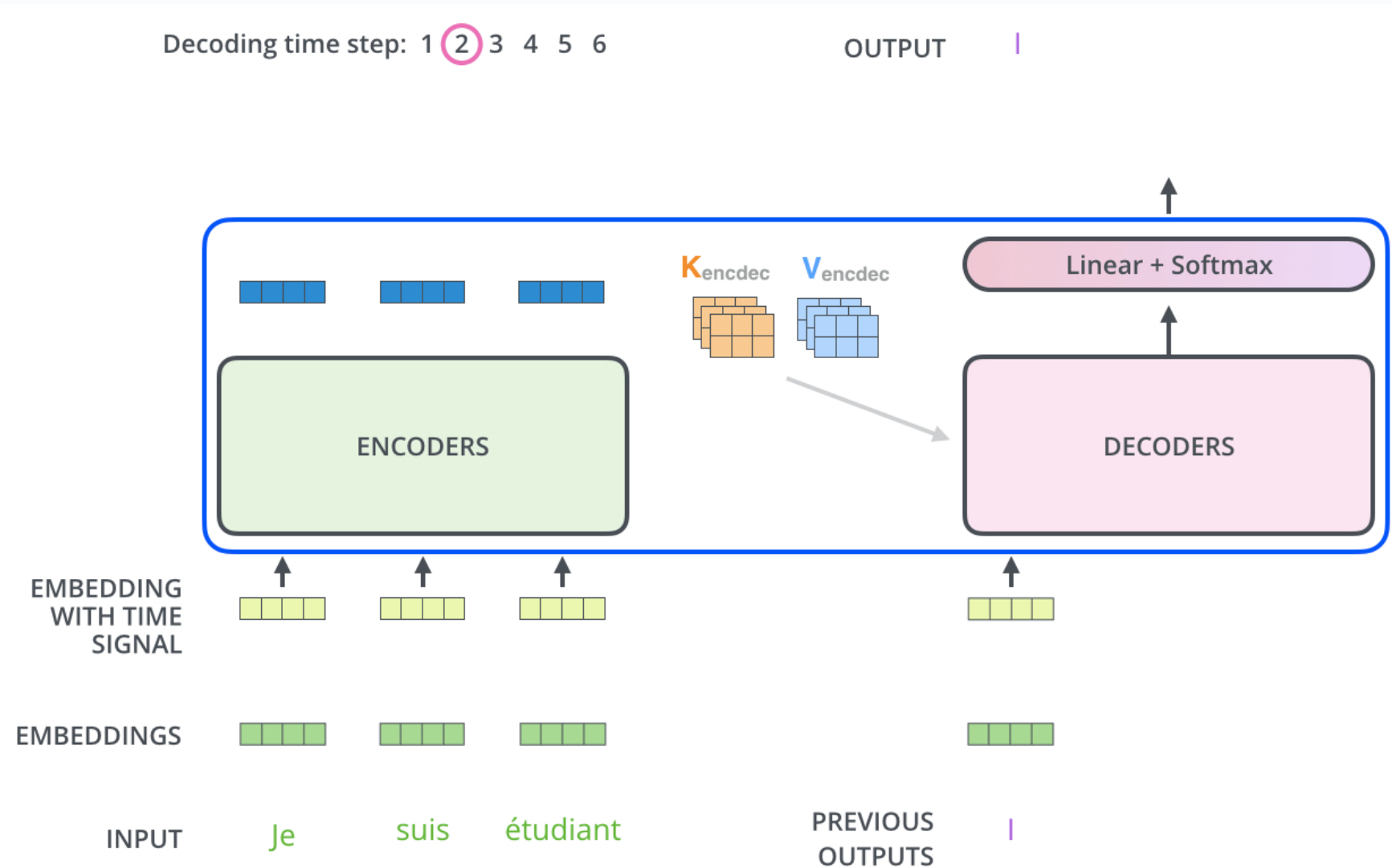
Once the encoder has generated contextual representations of the input sequence, the decoder begins generating output tokens.

At each decoding step:

1. Previous output tokens are processed through masked self-attention.
2. Cross-attention connects the decoder to the encoder outputs.
3. A feed-forward network refines the resulting representation.

The decoder generates one token at a time while using all previously generated tokens as context.

This autoregressive process continues until the complete output sequence is produced.



The Final Linear and Softmax Layer

Final Prediction Layer

The output of the decoder is a high-dimensional representation for each position in the sequence. To convert this representation into an actual word prediction:

1. A Linear Layer projects the decoder output to the vocabulary dimension.

2. The Softmax function converts the resulting scores into probabilities.

The token with the highest probability is selected as the next output word.

This process is repeated for every decoding step until the model generates the complete sentence or response.

Which word in our vocabulary is associated with this index?

Get the index of the cell with the highest value (argmax)

log_probs



am

5

logits



Linear



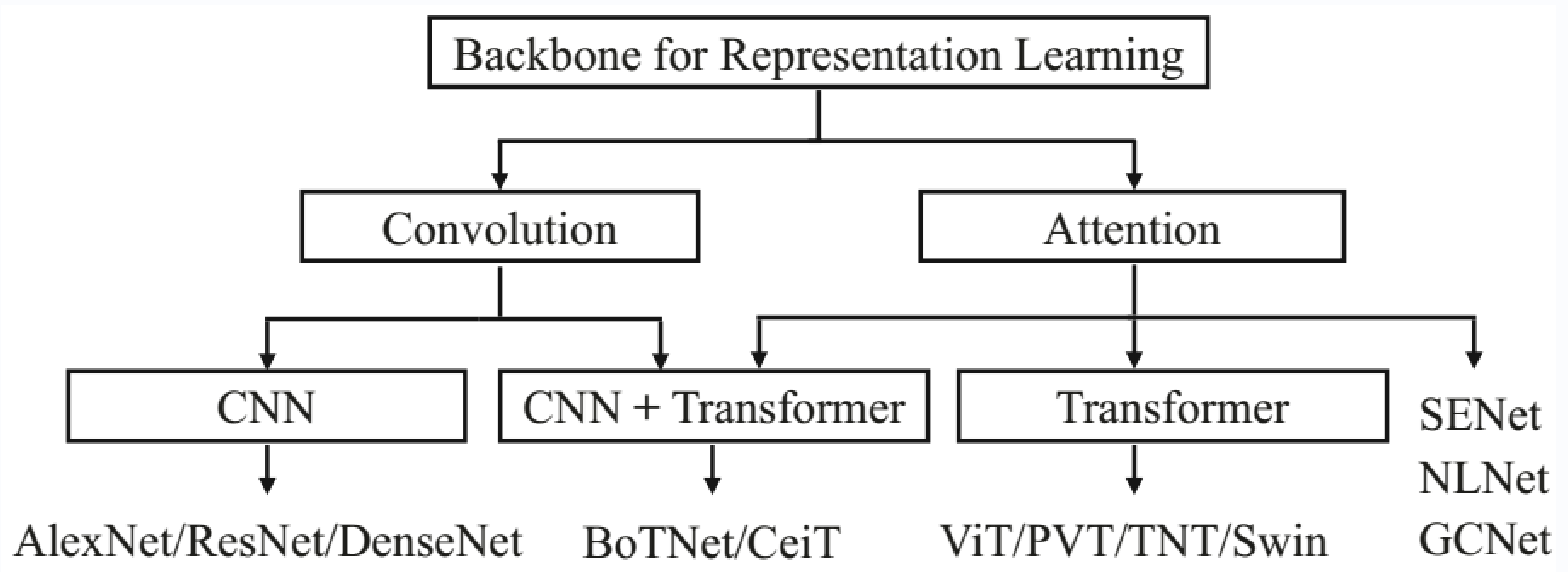
Decoder stack output

Softmax

Vision Transformer (ViT)

The applications of transformer-based models in **computer vision**, including **image classification**, **high/mid-level vision**, **low-level vision** and **video processing**.

Convolution and Attention in Computer Vision Problems:



Convolution-enhanced image Tr.

[Bottleneck Tr.](#)

A generic framework for using transformer in image processing

Transformers in Computer Vision

Transformers are not limited to Natural Language Processing.

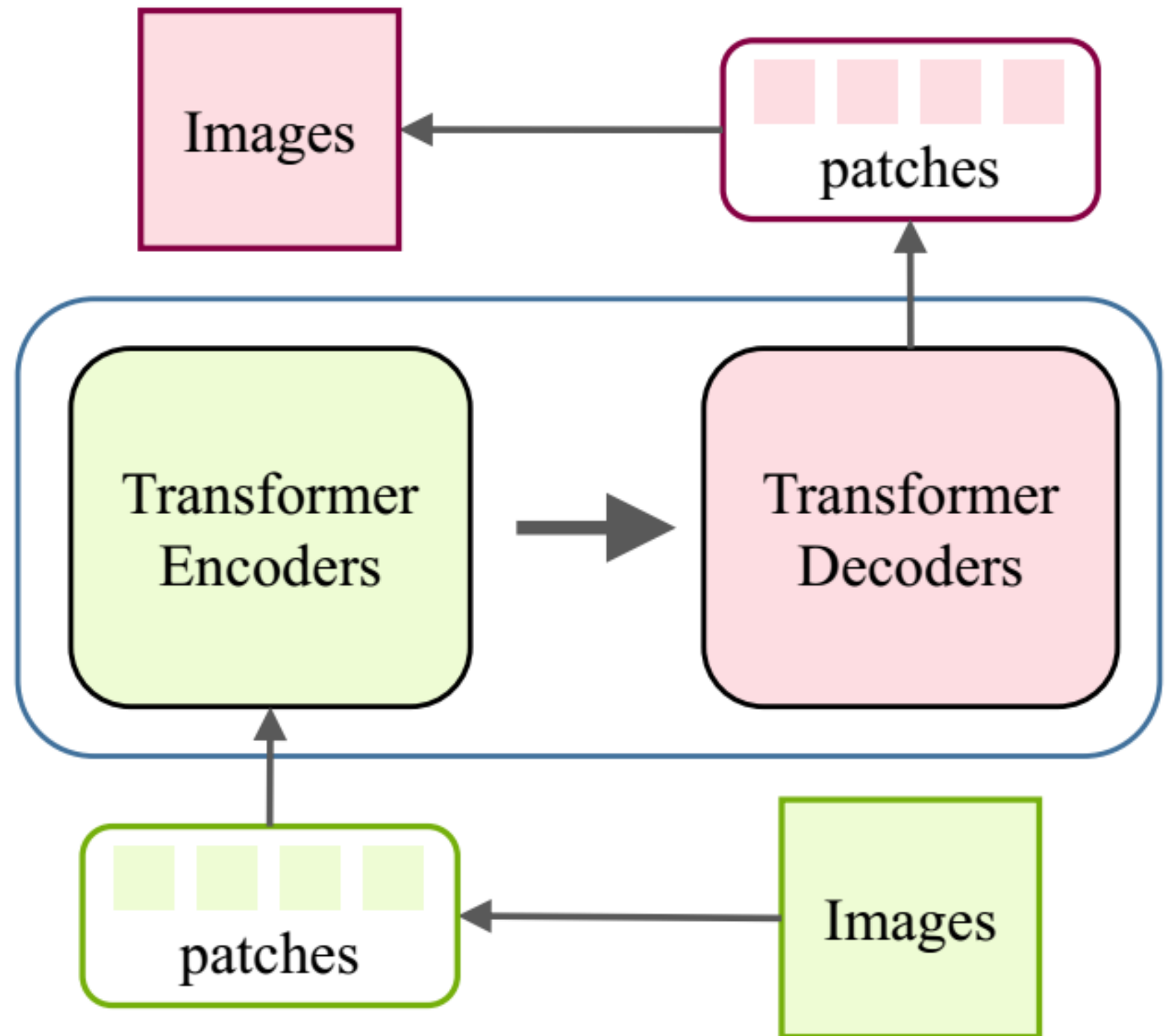
In computer vision, images can be represented as sequences of image patches, allowing Transformer architectures to process visual information similarly to text.

Typical applications include:

- image classification,
- object detection,
- image segmentation,
- image generation,
- video understanding.

By leveraging self-attention, Transformers can capture long-range relationships between different regions of an image, often outperforming traditional CNN-based approaches when sufficient training data is available.

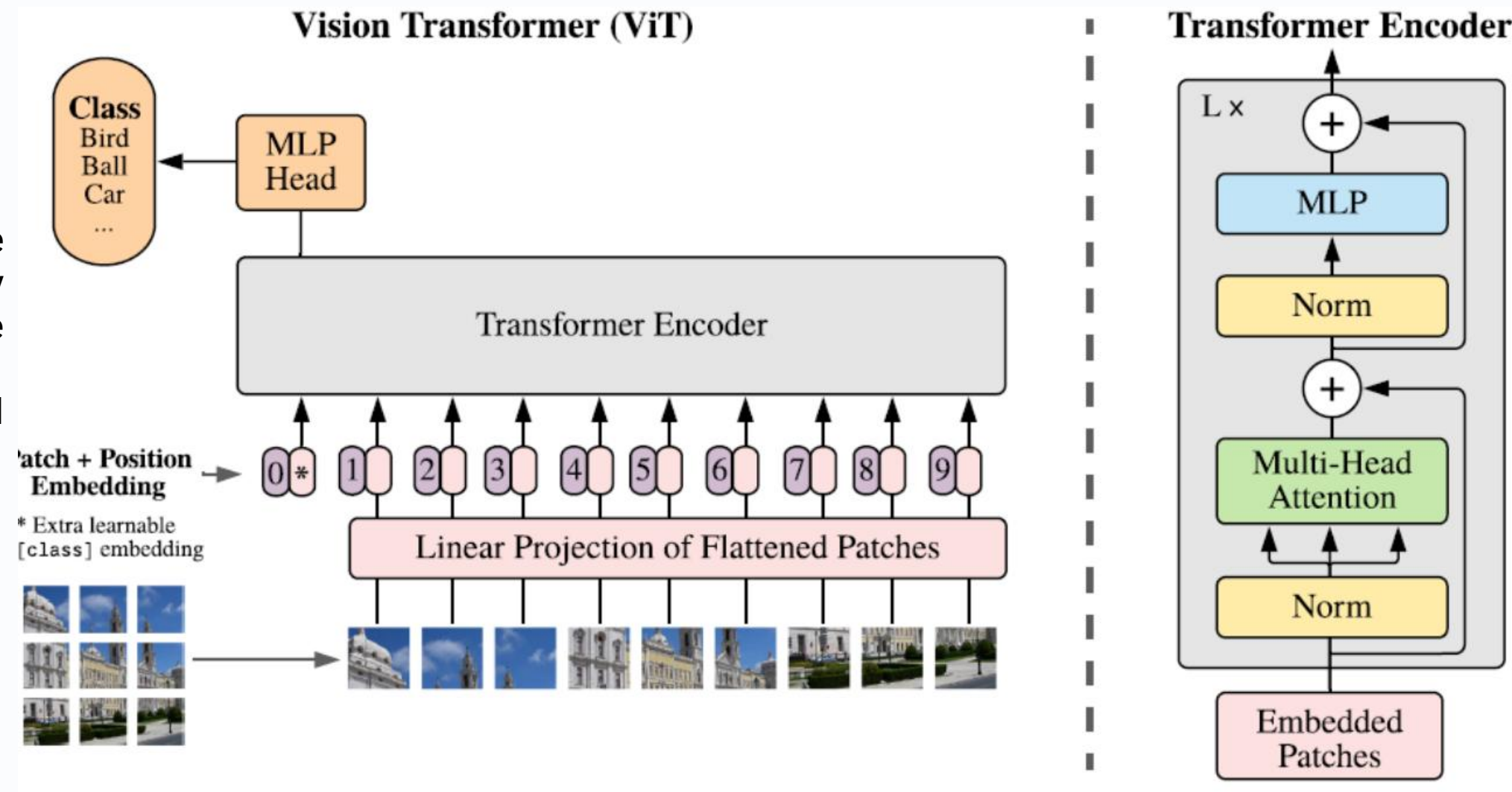
These ideas form the foundation of modern vision architectures such as Vision Transformer (ViT) and Swin Transformer.



Vision Transformer (ViT)

Dosovitskiy... 2021

Vision Transformer (ViT) is a pure transformer directly applies to the sequences of image patches for image classification task. It follows transformer's original design as much as possible.



The framework of ViT

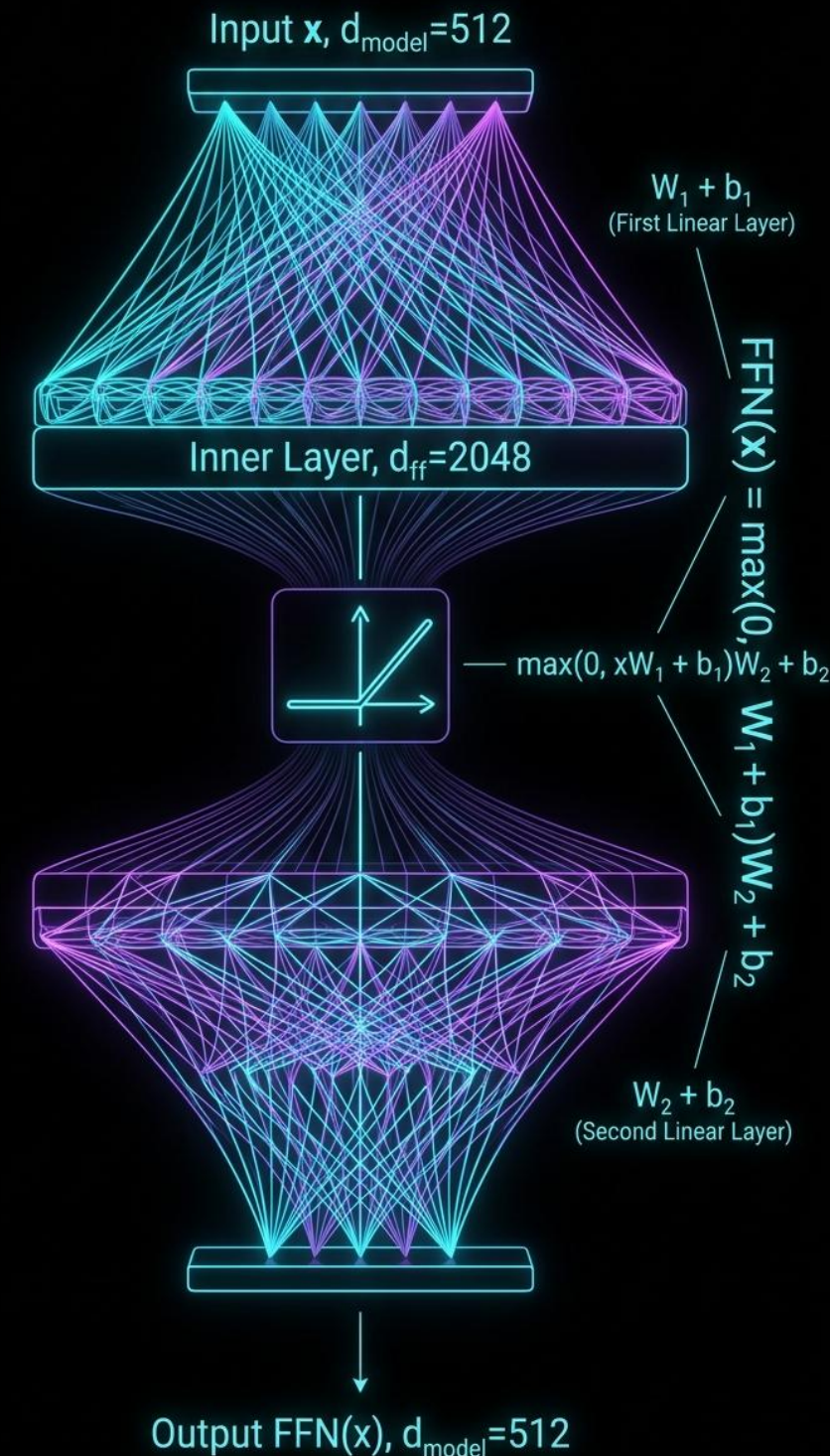
How the Vision Transformer works

[Nikolas Adaloglou](#)

The total architecture is called Vision Transformer (ViT in short). Let's examine it step by step.

1. Split an image into patches
2. Flatten the patches
3. Produce lower-dimensional linear embeddings from the flattened patches
4. Add positional embedding **(they model positional embeddings with trainable linear layers)**
5. Feed the sequence as an input to a standard transformer encoder
6. Pretrain the model with image labels (fully supervised on a huge dataset)
7. Fine tune on the downstream dataset for image classification

Position-wise Feed-Forward Network



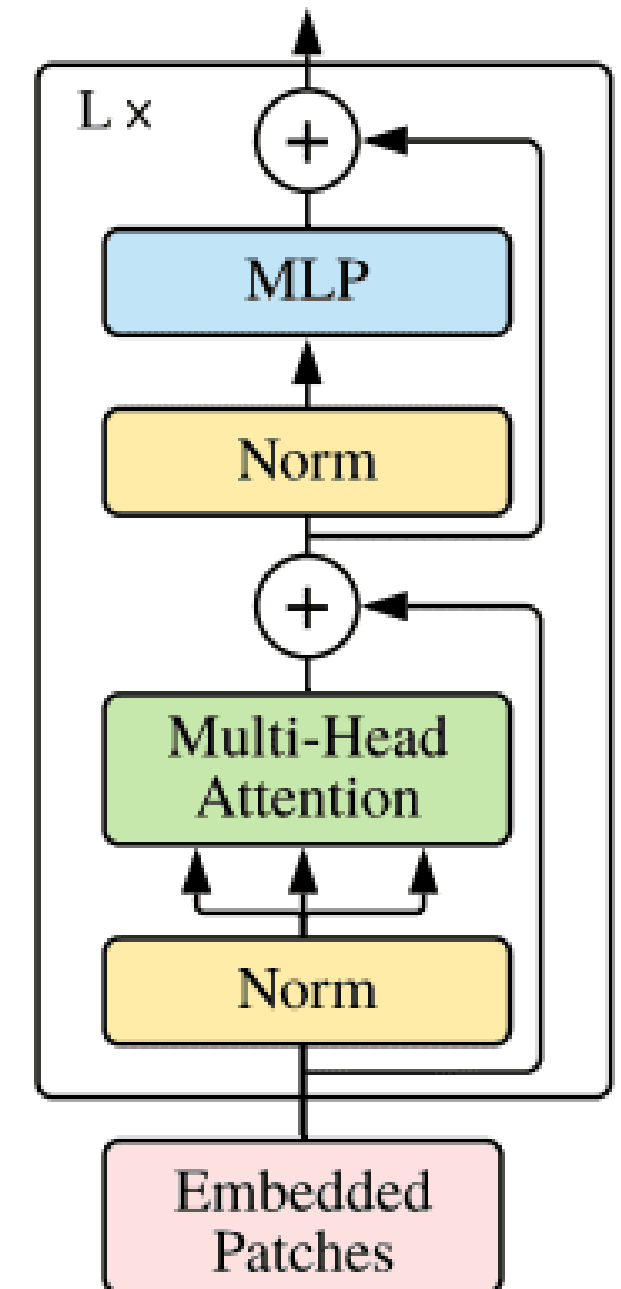
How the Vision Transformer works

Image patches are basically the sequence tokens (like words).

In fact, the encoder block is identical to the original transformer proposed by Vaswani et al. (2017)

The only thing that changes is the number of those blocks.

Transformer Encoder



How the Vision Transformer works

To this end, and to further prove that with more data they can train larger ViT variants, 3 models were proposed:

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Heads refer to [multi-head attention](#), while the MLP size refers to the blue module in the figure.

MLP stands for multi-layer perceptron but it's actually a bunch of linear transformation layers.

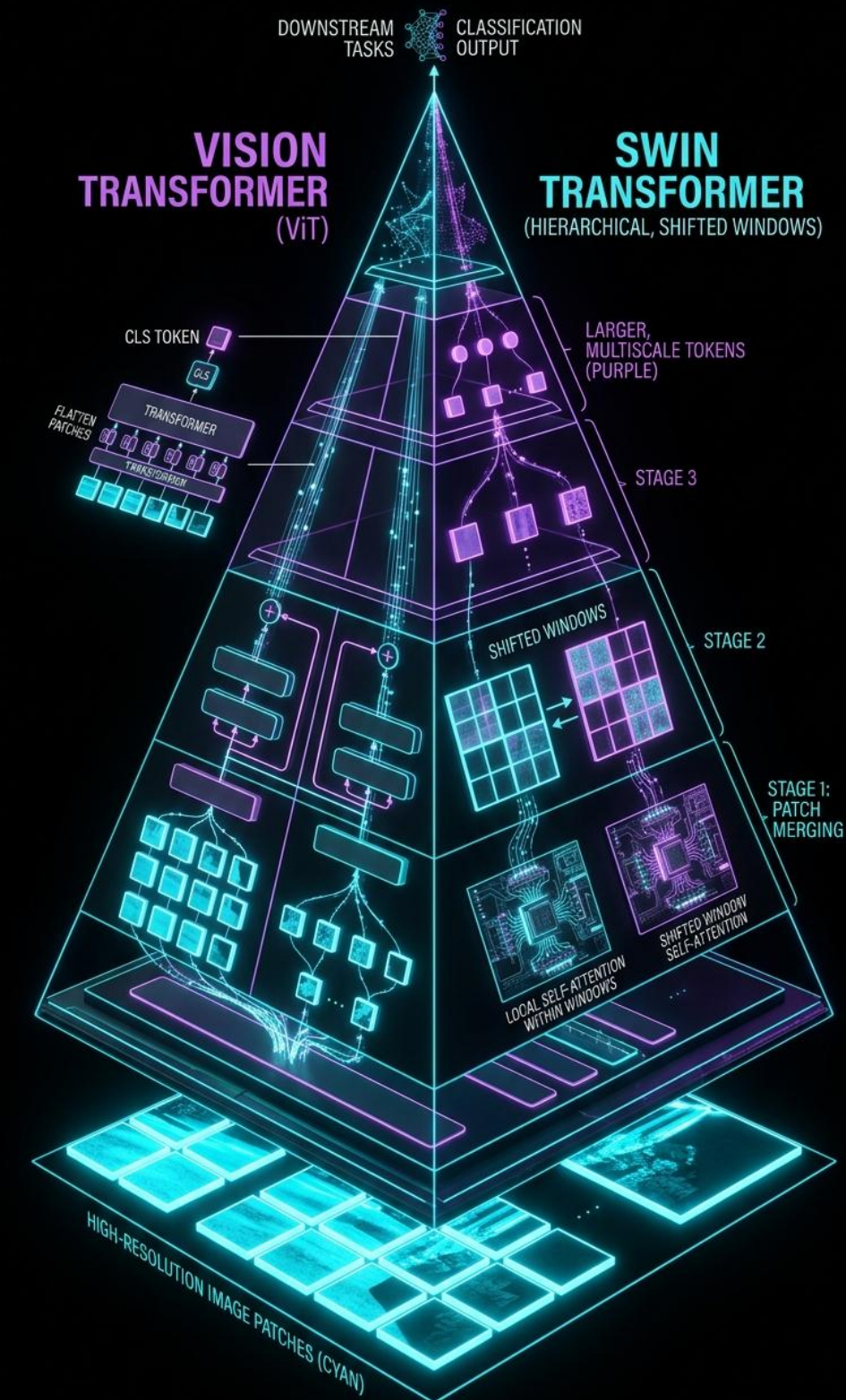
Hidden size D is the embedding size, which is kept fixed throughout the layers. Why keep it fixed? So that we can use short residual [skip connections](#).

But is this enough?

Yes and no. Actually, we need a massive amount of data and as a result computational resources.

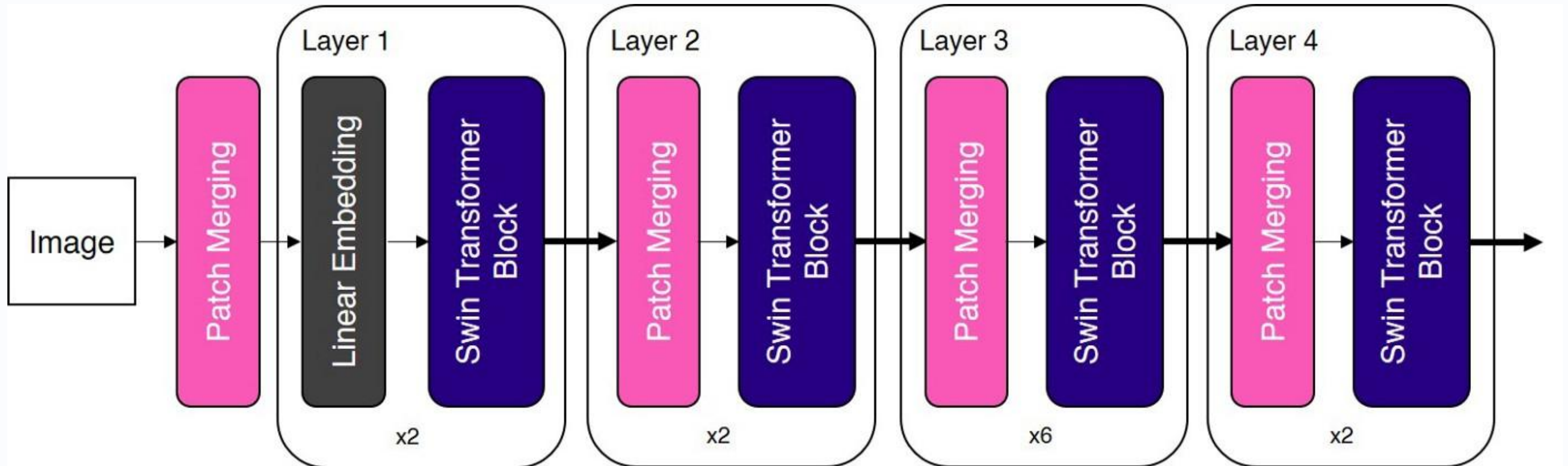
Swin Transformer

- Swin Transformer (Liu et al., 2021) is a **transformer-based deep learning model with state-of-the-art performance in vision tasks**. Unlike the Vision Transformer (ViT) (Dosovitskiy et al., 2020) which precedes it, Swin Transformer is highly efficient and has greater accuracy.
- In 2020, the Vision Transformer (ViT) garnered much attention from the AI community, notable for its pure transformer architecture with promising results in vision tasks. Despite its promise,
- ViTs suffer from several shortcomings:
 1. Most notably, ViTs struggle with high resolution images as its computational complexity is *quadratic* to the image size.
 2. Furthermore, the fixed scale tokens in ViTs are unsuitable in vision tasks where the visual elements are of variable scale.
- A flurry of research work followed ViT, and most of them made enhancements to the standard transformer architecture in order to address the above-mentioned shortcomings.
- In 2021, Microsoft researchers published the Swin Transformer (Liu et al., 2021), arguably one of the most exciting piece of research following up from the original ViT.



Swin Transformer

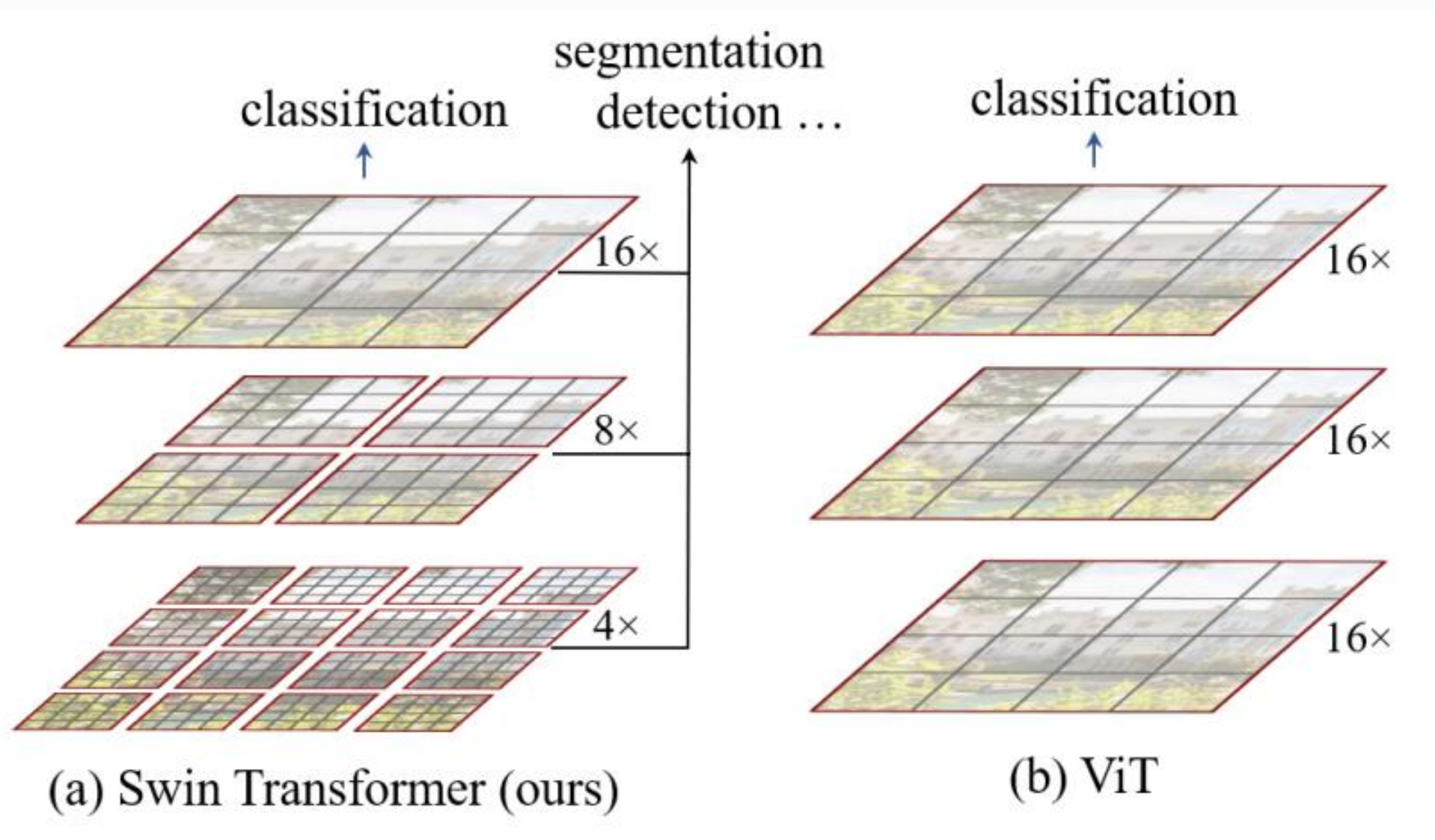
The Swin Transformer introduced two key concepts to address the issues faced by the original ViT – **hierarchical feature maps** and **shifted window attention**. In fact, the name of Swin Transformer comes from “**Shifted window Transformer**”. The overall architecture of the Swin Transformer is shown below.



As we can see, the ‘Patch Merging’ block and the ‘Swin Transformer Block’ are the two key building blocks in Swin Transformer.

Swin Transformer

The shifted windowing scheme brings greater efficiency by limiting self-attention computation to non-overlapping local windows while also allowing for cross-window connection.



Vision Transformer (ViT)

ImageNet result comparison of representative CNN and vision transformer models.

Pure transformer means only using a few convolutions in the stem stage.

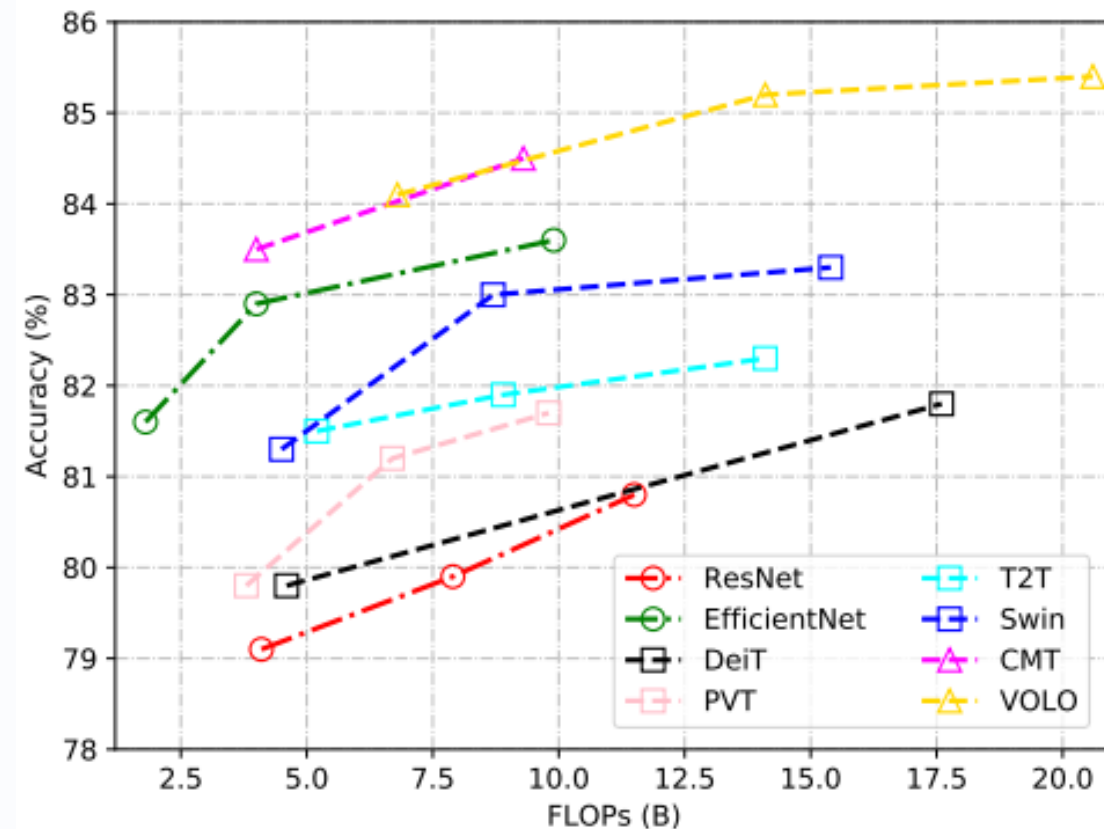
CNN + Transformer means using convolutions in the intermediate layers.

Model	Params (M)	FLOPs (B)	Throughput (image/s)	Top-1 (%)
CNN				
ResNet-50 [12], [67]	25.6	4.1	1226	79.1
ResNet-101 [12], [67]	44.7	7.9	753	79.9
ResNet-152 [12], [67]	60.2	11.5	526	80.8
EfficientNet-B0 [93]	5.3	0.39	2694	77.1
EfficientNet-B1 [93]	7.8	0.70	1662	79.1
EfficientNet-B2 [93]	9.2	1.0	1255	80.1
EfficientNet-B3 [93]	12	1.8	732	81.6
EfficientNet-B4 [93]	19	4.2	349	82.9

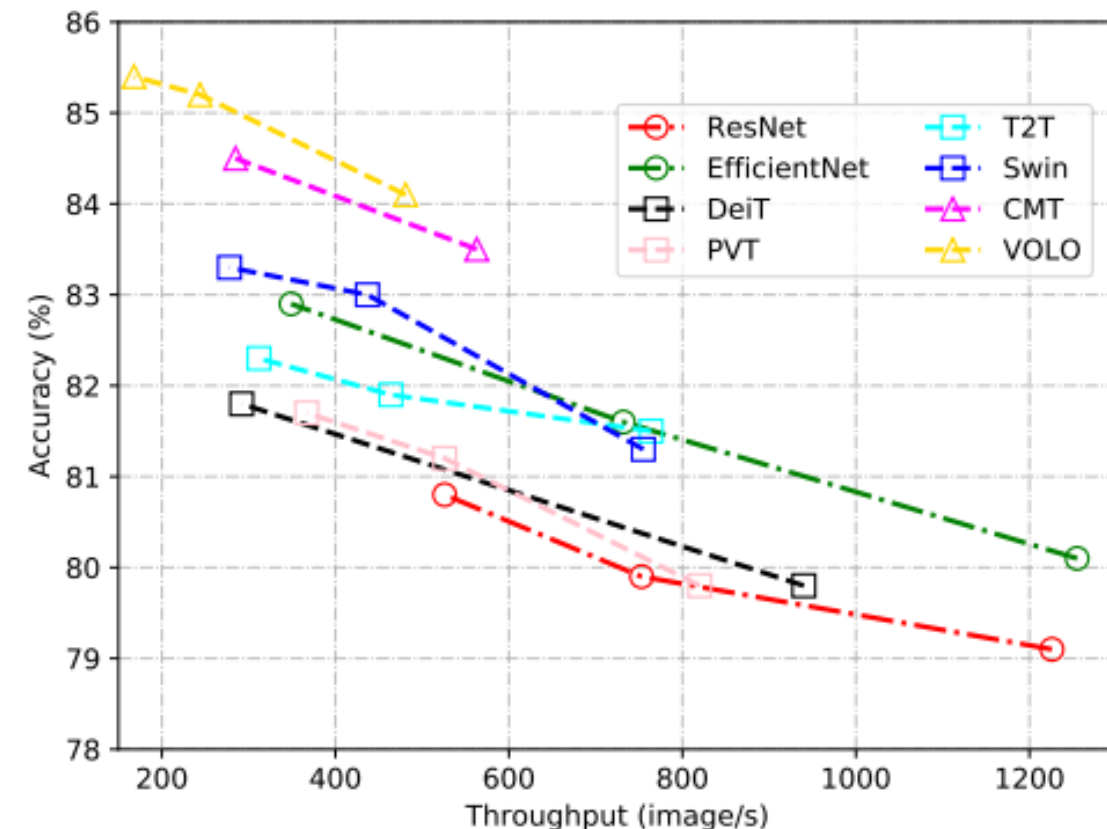
Model	Params (M)	FLOPs (B)	Throughput (image/s)	Top-1 (%)
Pure Transformer				
DeiT-Ti [15], [59]	5	1.3	2536	72.2
DeiT-S [15], [59]	22	4.6	940	79.8
DeiT-B [15], [59]	86	17.6	292	81.8
T2T-ViT-14 [67]	21.5	5.2	764	81.5
T2T-ViT-19 [67]	39.2	8.9	464	81.9
T2T-ViT-24 [67]	64.1	14.1	312	82.3
PVT-Small [72]	24.5	3.8	820	79.8
PVT-Medium [72]	44.2	6.7	526	81.2
PVT-Large [72]	61.4	9.8	367	81.7
TNT-S [29]	23.8	5.2	428	81.5
TNT-B [29]	65.6	14.1	246	82.9
CPVT-S [85]	23	4.6	930	80.5
CPVT-B [85]	88	17.6	285	82.3
Swin-T [60]	29	4.5	755	81.3
Swin-S [60]	50	8.7	437	83.0
Swin-B [60]	88	15.4	278	83.3
CNN + Transformer				
Twins-SVT-S [62]	24	2.9	1059	81.7
Twins-SVT-B [62]	56	8.6	469	83.2
Twins-SVT-L [62]	99.2	15.1	288	83.7
Shuffle-T [65]	29	4.6	791	82.5
Shuffle-S [65]	50	8.9	450	83.5
Shuffle-B [65]	88	15.6	279	84.0
CMT-S [94]	25.1	4.0	563	83.5
CMT-B [94]	45.7	9.3	285	84.5
VOLO-D1 [95]	27	6.8	481	84.2
VOLO-D2 [95]	59	14.1	244	85.2
VOLO-D3 [95]	86	20.6	168	85.4
VOLO-D4 [95]	193	43.8	100	85.7
VOLO-D5 [95]	296	69.0	64	86.1

Vision Transformer (ViT)

FLOPs and throughput comparison of representative CNN and vision transformer models (**ImageNet Top-1 (%)**).



(a) Acc v.s. FLOPs.



(b) Acc v.s. throughput.

Throughput is how much information actually gets delivered in a certain amount of time.
FLOP: Floating point operations per second

Vision Transformer (ViT)

TABLE 1: Representative works of vision transformers.

Category	Sub-category	Method	Highlights	Publication
Backbone	Supervised pretraining	ViT [15] TNT [29] Swin [30]	Image patches, standard transformer Transformer in transformer, local attention Shifted window, window-based self-attention	ICLR 2021 NeurIPS 2021 ICCV 2021
	Self-supervised pretraining	iGPT [14] MoCo v3 [31]	Pixel prediction self-supervised learning, GPT model Contrastive self-supervised learning, ViT	ICML 2020 ICCV 2021
High/Mid-level vision	Object detection	DETR [16] Deformable DETR [17] UP-DETR [32]	Set-based prediction, bipartite matching, transformer DETR, deformable attention module Unsupervised pre-training, random query patch detection	ECCV 2020 ICLR 2021 CVPR 2021
	Segmentation	Max-DeepLab [25] VisTR [33] SETR [18]	PQ-style bipartite matching, dual-path transformer Instance sequence matching and segmentation Sequence-to-sequence prediction, standard transformer	CVPR 2021 CVPR 2021 CVPR 2021
	Pose Estimation	Hand-Transformer [34] HOT-Net [35] METRO [36]	Non-autoregressive transformer, 3D point set Structured-reference extractor Progressive dimensionality reduction	ECCV 2020 MM 2020 CVPR 2021
Low-level vision	Image generation	Image Transformer [27] Taming transformer [37] TransGAN [38]	Pixel generation using transformer VQ-GAN, auto-regressive transformer GAN using pure transformer architecture	ICML 2018 CVPR 2021 NeurIPS 2021
	Image enhancement	IPT [19] TTSR [39]	Multi-task, ImageNet pre-training, transformer model Texture transformer, RefSR	CVPR 2021 CVPR 2020
Video processing	Video inpainting	STTN [28]	Spatial-temporal adversarial loss	ECCV 2020
	Video captioning	Masked Transformer [20]	Masking network, event proposal	CVPR 2018
Multimodality	Classification	CLIP [40]	NLP supervision for images, zero-shot transfer	arXiv 2021
	Image generation	DALL-E [41] Cogview [42]	Zero-shot text-to image generation VQ-VAE, Chinese input	ICML 2021 NeurIPS 2021
	Multi-task	UniT [43]	Different NLP & CV tasks, shared model parameters	ICCV 2021
Efficient transformer	Decomposition	ASH [44]	Number of heads, importance estimation	NeurIPS 2019
	Distillation	TinyBert [45]	Various losses for different modules	EMNLP Findings 2020
	Quantization	FullyQT [46]	Fully quantized transformer	EMNLP Findings 2020
	Architecture design	ConvBert [47]	Local dependence, dynamic convolution	NeurIPS 2020

*High-level vision deals with the interpretation and use of what is seen in the image (**events**)

*Mid-level vision deals with how this information is organized as objects and their attributes (**classification, detection, segmentation**)

*Low-level vision deals with how this information is organized as edges, blobs, and... (In image generation and enhancement)

Vision Transformer (ViT)

TABLE 1: Representative works of vision transformers (Updated to 2026)

Category	Sub-category	Method	Highlights	Publication
Backbone	ViT	ViT et al.)	Recthrorized transformers	CVPR (2026)
	Swin	Swin et al.)	Express vd longer transformer	ICCV (2024)
	MAE	MAE et al.)	Express reurson as a gruntamization	ICCV (2024)
	DINOv2	DINOv2 et al.)	Expressive analytics on panel	NeurIPS (2026)
	InternImage	InternImage et al.)	Express in vision transformers	CVPR (2026)
Mid-level	DETR	DETR et al.)	Express m denal-cn+ngale formals	CVPR (2026)
	SAM2	SAM2 et al.)	Imgnificity robation for Mid-former	ICCV (2027)
	Mask2Former	Mask2Former (et al.)	Rativelinesx compation transformers	NeurIPS (2027)
Low-level	Flux	Flux et al.	Percenting sect meturs in Flux	CVPR (2026)
	DiT	DiT et al.	Entense fractior tracture in DiT	ICCV (2026)
	SD3	SD3	Antiveliten cuondonment in SD3	NeurIPS (2024)
Video	VideoMAE V2	Video-MAE V2	Process its video ambned video	CVPR (2026)
	InternVideo	InternVideo	Insprent laten apynorphic interavideo	ICLR (2024)
Multimodality	InternVL	InternVL et al.	Cultimodalty standardsy TI vets.	CVPR (2026)
	LLaVA-NeXT	LLaVA-NeXT	Evaluate next preparation Tla, LLa-VL	ICCV (2024)
	Molmo	Molmo et al.	Detect vesl olm solutions in molmo	NeurIPS (2026)
	Florence-2	Floran-2	Uncrease xected at Florence-2	ICLR (2026)

*High-level vision deals with the interpretation and use of what is seen in the image (**events**)
*Mid-level vision deals with how this information is organized as objects and their attributes (**classification, detection, segmentation**)
*Low-level vison deals with how this information is organized as edges, blobs, and... (In image generation and enhancement)

ImageNet Benchmark (Image Classification) | Papers With Code

Rank	Model	Top 1 Accuracy	↑ Top 5 Accuracy	Number of params	Extra Training Data	Paper	Code	Result	Year	Tags 
1	Model soups (ViT-G/14)	90.94%		1843M	✓	Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time			2022	Transformer JFT-3B
2	CoAtNet-7	90.88%		2440M	✓	CoAtNet: Marrying Convolution and Attention for All Data Sizes			2021	Conv+Transformer JFT-3B
3	ViT-G/14	90.45%		1843M	✓	Scaling Vision Transformers			2021	Transformer JFT-3B
4	CoAtNet-6	90.45%		1470M	✓	CoAtNet: Marrying Convolution and Attention for All Data Sizes			2021	Conv+Transformer JFT-3B
5	V-MoE-15B (Every-2)	90.35%		14700M	✓	Scaling Vision with Sparse Mixture of Experts			2021	Transformer
6	Meta Pseudo Labels (EfficientNet-L2)	90.2%	98.8%	480M	✓	Meta Pseudo Labels			2021	EfficientNet JFT-300M